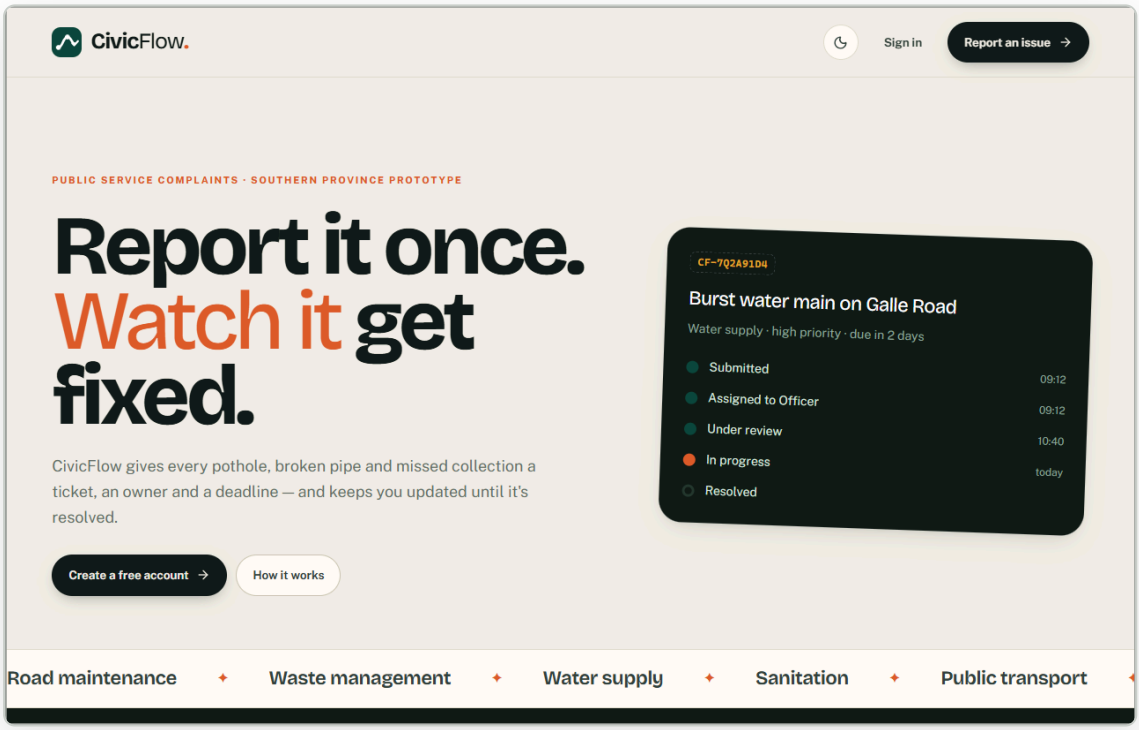
UNIVERSITY OF RUHUNA – FACULTY OF TECHNOLOGY

ICT2232 Software Engineering · Mini Project

CivicFlow

Public Service Complaint & Resolution System

Final Project Report · SDLC documentation · Group 15



GROUP 15 MEMBERS

TG/2023/1761	M.S.M. Sharfan
TG/2023/1731	H.M. Amadi Nethma
TG/2023/1757	N.M. Razim Mohamed
TG/2023/1710	M.F. Rishma
TG/2023/1776	V. Kayaththiry

SUBMISSION

Lecturer: [Lecturer name]

Date: [Submission date]

Live system: https://civic-flow-rho.vercel.app

Repository: github.com/razim2003/civic-flow

Declaration

We declare that this report and the CivicFlow software are the work of Group 15 for ICT2232 Software Engineering. Material from other sources is acknowledged in the References. All names, complaints and locations shown in this report, in the screenshots and in the demonstration are fictional.

Use of AI tools: as allowed by Section 4 of the project guidelines, we used AI-assisted tools to help with parts of the coding and documentation. The requirements, design decisions, priorities and trade-offs were decided by the team. We reviewed everything the tools produced, checked it with the 283 automated tests that run on every commit, and documented it. Each member can explain the parts of the system they worked on.

Executive summary

In Sri Lanka, people report problems with roads, waste collection, water supply, sanitation and public transport by telephone, by visiting an office or through people they know. These complaints are often lost, the person who complained cannot see what is happening, and the organisation cannot easily tell who is responsible or which complaints are overdue.

CivicFlow is a web application that handles each complaint from submission to resolution in one place. A Citizen submits a complaint with a category, a description, a location on the map and, if needed, photos or documents. The system sends it to the department responsible for that category and assigns it to the Officer there with the fewest open complaints. The Officer reviews it, sets a priority, works on it and resolves it. Each step is saved in a history that cannot be edited, and the Citizen is told about it in an Updates inbox. After resolution the Citizen can rate the service. Administrators manage users, invite Officers, maintain the service categories, move complaints that are stuck in departments without staff, and follow performance through analytics.

We followed Scrum over five sprints. The final system covers all 25 functional requirements (every Must and Should item, the one Could item and three requirements added through change request CR-006) and 11 measurable non-functional requirements. Data is stored in PostgreSQL with Row Level Security, every change goes through checked functions on the server, and evidence files are kept in private storage.

We wrote 283 automated tests (77 unit, 147 database integration and 59 system tests), and all of them pass. We also ran an automated layout check at four screen sizes. 19 defects were recorded and fixed. Eight security controls cover the eleven threats we assessed. The system is live at https://civic-flow-rho.vercel.app.

Table of contents

1 Introduction and Problem Domain 9
1.1 Report structure 9

2 Problem Statement 10

3 Stakeholders and Objectives 11
3.1 Stakeholders and their needs 11
3.2 Project objectives 11

4 Scope, Assumptions and Constraints 13
4.1 In scope 13
4.2 Out of scope 13
4.3 Assumptions 13
4.4 Constraints 14

5 Software Process Model and Justification 15
5.1 Selected model: Agile with Scrum 15
5.2 Justification 15
5.3 Development approach and Definition of Done 16
5.4 Project plan and tooling 16

6 Agile Planning 18
6.1 Product backlog 18
6.2 User stories 19
6.3 Acceptance criteria for the major user stories 20
6.4 Sprint plan and sprint history 21
6.5 Task allocation 22

7 Requirements Engineering 23
7.1 Actors 23
7.2 Functional requirements 23
7.3 Non-functional requirements 25
7.4 Use case diagram 27
7.5 Use case descriptions 28
7.6 Prioritisation (MoSCoW) 30
7.7 Requirements traceability matrix 31
7.8 Requirements validation 33

8 System Design 34
8.1 High-level system architecture 34
8.2 Frontend architecture 35
8.3 Class / domain model 36
8.4 Sequence diagrams 37
8.5 Activity diagram of the complaint workflow 41

8.6 State machine of the complaint status 42
8.7 Database design 42
8.8 User-interface prototype and wireframes 44
8.9 Design decisions and justification 46

9 Implementation 48
9.1 Technology stack 48
9.2 Module structure 48
9.3 Core workflows implemented end to end 49
9.4 Input validation 56
9.5 Error handling 57
9.6 Coding standards and version control 57

10 Testing and Quality Assurance 59
10.1 Test plan 59
10.2 Execution results 60
10.3 Representative test cases 60
10.4 Acceptance testing 61
10.5 Defects identified and corrected 63
10.6 Limitations of the test evidence 65

11 Deployment 66
11.1 Deployed architecture and database configuration 66
11.2 System requirements 67
11.3 Configuration 67
11.4 Installation and deployment steps 67
11.5 Deployment verification (smoke test) 68

12 Maintenance and Change Request 69
12.1 Change request register 69
12.2 Main change request CR-006: every complaint must reach a responsible Officer 70

13 Security 73
13.1 Threats and risk assessment 73
13.2 Security requirements 75
13.3 Implemented security controls 75
13.4 Security decisions 76

14 Project Management and Individual Contributions 77
14.1 Team coordination 77
14.2 Individual contributions 77
14.3 Contribution log 78
14.4 Version-control evidence 78

15 Conclusion 79
15.1 Lessons learned 79
15.2 Future work 79

16 References **80**

17 Appendices **81**

17.1 Appendix A: User guide (summary) 81

17.2 Appendix B: Complete test cases 81

17.3 Appendix C: Repository, deployment and demonstration 91

List of figures

Figure 5.1	Scrum cycle applied in every sprint	15
Figure 6.1	Five-sprint roadmap (from the proposal) with the delivered increments	21
Figure 7.1	UML use case diagram (UC-15 and UC-16 were added by change request CR-006)	27
Figure 8.1	High-level system architecture	34
Figure 8.2	Layered structure of every feature module	35
Figure 8.3	UML class diagram of the domain model (operations are the only way to change data)	36
Figure 8.4	SD-1: Citizen submits a complaint (UC-03)	37
Figure 8.5	SD-2: Assigned Officer resolves a complaint (UC-07)	38
Figure 8.6	SD-3: Automatic assignment and Administrator dispatch (CR-006, UC-15)	39
Figure 8.7	SD-4: Citizen gives feedback (UC-08)	40
Figure 8.8	UML activity diagram with swimlanes for the complete complaint workflow	41
Figure 8.9	UML state machine of the complaint status	42
Figure 8.10	Entity-relationship diagram (PostgreSQL 17)	43
Figure 8.11	Wireframe of the Citizen "Make a complaint" screen	44
Figure 8.12	Wireframe of the Officer complaint processing screen	45
Figure 8.13	Wireframe of the Administrator dispatch centre (CR-006)	45
Figure 8.14	Wireframe of the mobile complaint list and slide-out menu	46
Figure 9.1	Make a complaint screen (Citizen): category, description, OpenStreetMap location and typed coordinates	49
Figure 9.2	Complaint detail (Citizen) with the progress line, location, history and service-target badge	50
Figure 9.3	My complaints (Citizen): each complaint is shown as a ticket with its status and a due or overdue badge	50
Figure 9.4	Resolution details and the one-time service feedback form (Citizen)	51
Figure 9.5	Updates inbox (Citizen) with the unread count and mark-as-read	51
Figure 9.6	Department queue (Officer) with filters for status, category, priority and assignment	52
Figure 9.7	Processing a complaint (Officer): only the valid next actions are offered	52
Figure 9.8	Department map of open complaints (Officer)	53
Figure 9.9	Dispatch centre (Administrator) showing department coverage and complaints that need attention	53
Figure 9.10	Service analytics (Administrator): targets, coverage, workload, satisfaction and the six-month trend, with CSV export	54
Figure 9.11	Service category management (Administrator)	54
Figure 9.12	Citizen overview, feedback form and slide-out menu on a 360 px wide phone	55
Figure 9.13	Dispatch centre, user management and analytics on a 360 px wide phone	56
Figure 9.14	Continuous-integration pipeline that runs on every push	58
Figure 10.1	Automated test results by level (latest run, 30 October 2026)	60
Figure 11.1	UML deployment diagram of the production system	66
Figure 12.1	Maintenance process used for every change request	69
Figure 12.2	CR-006 impact across SDLC artefacts	71
Figure 13.1	Trust boundary: every decision is made on the server side	73

List of tables

Table 2.1	Problem analysis	10
Table 3.1	Stakeholders, needs and how CivicFlow addresses them	11
Table 3.2	Objectives and evidence of achievement	11
Table 4.1	Assumptions and their validation status	13
Table 5.1	Why Scrum fits this project better than a plan-driven model	15
Table 5.2	Definition of Done used for every backlog item	16
Table 5.3	Project plan by SDLC phase	16
Table 5.4	Tooling and configuration management	16
Table 6.1	Product backlog	18
Table 6.2	User stories	19
Table 6.3	Acceptance criteria (major stories)	20
Table 6.4	Sprint plan, delivered increments and review outcomes	21
Table 6.5	Task allocation by module	22
Table 7.1	Actors	23
Table 7.2	Functional requirements	23
Table 7.3	Non-functional requirements	25
Table 7.4	UC-03 Submit complaint (CF-04, 05, 06, 16, 23)	28
Table 7.5	UC-04 Track complaint (CF-07, 17, 18)	28
Table 7.6	UC-05 Manage complaint (CF-08, 09, 10)	28
Table 7.7	UC-06/07 Update and resolve complaint (CF-11, 12)	29
Table 7.8	UC-08 Provide feedback (CF-19)	29
Table 7.9	UC-09 Manage users and invite Officers (CF-14)	29
Table 7.10	UC-15 Dispatch complaint (CF-24)	29
Table 7.11	MoSCoW prioritisation	30
Table 7.12	Requirements traceability matrix	31
Table 7.13	Requirements validation activities	33
Table 8.1	Major tables and integrity rules	44
Table 8.2	Important design decisions	46
Table 9.1	Technology stack as built	48
Table 9.2	Module map	48
Table 9.3	Input validation in two layers	56
Table 10.1	Test levels	59
Table 10.2	Execution summary	60
Table 10.3	Representative test cases	60
Table 10.4	Acceptance test summary	61
Table 10.5	Defect log	63
Table 11.1	System requirements	67
Table 11.2	Configuration settings	67
Table 11.3	Smoke-test and acceptance checklist on the deployed URL	68
Table 12.1	Change request register	69
Table 12.2	CR-006 change request	70
Table 12.3	CR-006 impact analysis and estimate	71
Table 13.1	Assets and their sensitivity	73
Table 13.2	Threats and risk assessment	74
Table 13.3	Security requirements	75
Table 13.4	Implemented controls and how to demonstrate them	75
Table 14.1	Contribution matrix by SDLC activity	77

Table 14.2 Contribution log (major tasks) 78
Table 17.1 User guide by role 81
Table 17.2 Authentication and access (CF-01…03, CF-15) 82
Table 17.3 Complaint submission and tracking (CF-04…07, 16…18) 83
Table 17.4 Officer processing and resolution (CF-08…12) 84
Table 17.5 Withdrawal (CF-22) 85
Table 17.6 Sprint 4: feedback, inbox, categories, monitoring (CF-13, 19, 20, 21) 86
Table 17.7 Dispatch, coverage and service targets (CR-006: CF-23, 24, 25, 21) 87
Table 17.8 Sri Lankan demonstration data (supabase/demo) 89
Table 17.9 Administration (CF-14) 89
Table 17.10 HTTP integration with real JWTs (last executed 2026-10-21) 90
Table 17.11 Key links and commands 91
Table 17.12 Demonstration script for the final evaluation 91

1 Introduction and Problem Domain

Road maintenance, waste collection, water supply, sanitation and public transport affect people every day. When something goes wrong, such as a pothole, an overflowing bin, a burst water main or a bus that does not stop at a halt, people need an easy way to report it and to know that someone is dealing with it.

Today most complaints are made by telephone, by visiting a council or divisional office, by letter or e-mail, or through personal contacts. None of these channels keeps a shared record, so complaints get lost or are reported twice. The person who complained cannot see any progress and has to call again to follow up. The service organisations, on the other hand, have no reliable way to record, categorise, prioritise, assign and monitor complaints, or to show how quickly they responded.

Our group (Group 15) was given the domain "Public Service Complaint & Resolution System: a system for citizens to submit service complaints, track progress and receive resolution updates". We named the product CivicFlow. This report describes how we took it through the ten stages of the Software Development Life Cycle (SDLC) and how the output of each stage was used in the next.

1.1 Report structure

The chapters follow the structure given in Section 16 of the project guidelines. Chapters 2 to 5 cover problem definition and process planning (Milestone 1), Chapter 6 covers Agile planning (Milestone 2) and Chapter 7 covers requirements engineering (Milestone 3). Chapter 8 describes the design (Milestone 4), Chapters 9 and 10 the implementation and testing (Milestone 5), and Chapter 11 the deployment (Milestone 6). Chapters 12 and 13 deal with maintenance and security (Milestone 7), and Chapter 14 with project management and each member's contribution.

2 Problem Statement

Problem statement

Citizens find it difficult to report public service issues and to follow up on them, because complaint information is handled through separate, manual channels. This leads to poor visibility of complaint status, slow communication, unclear responsibility and difficulty in monitoring unresolved complaints. Public service organisations also lack an organised way to record, categorise, prioritise, assign and monitor complaints. A centralised, role-based Public Service Complaint & Resolution System is therefore required to improve accessibility, transparency, accountability and efficiency.

Table 2.1: Problem analysis

Aspect	Description
Who is affected	Citizens, front-line public service Officers, department managers and public service organisations
Current situation	Phone, walk-in, e-mail and informal reports; no shared record; no status visibility; manual follow-up
Impact	Lost or duplicated complaints, no audit trail, unclear accountability, slow resolution and low public trust
Root causes	No single system of record; no defined workflow or ownership; no notification of progress
Proposed solution	CivicFlow: a web application with a defined complaint lifecycle (Submitted → Under review → In progress → Resolved, or Withdrawn by the owner), automatic assignment, history, updates and feedback
Success measure	Every complaint has a reference, a responsible Officer, a service target and a complete history visible to the Citizen

3 Stakeholders and Objectives

3.1 Stakeholders and their needs

We identified the stakeholders from the problem domain and refined the list while writing the proposal. Three of them use the system directly, as the roles Citizen, Officer and Administrator. Department managers are stakeholders too, but we decided not to give them a separate login role; their needs are covered by the department queues and the Administrator's analytics.

Table 3.1: Stakeholders, needs and how CivicFlow addresses them

Stakeholder	Type	Needs	How CivicFlow addresses them
Citizen	Primary user (application role)	Easy submission, tracking, status visibility, updates and feedback	Complaint form with map, My Complaints, timeline, Updates inbox, feedback (CF-04…07, 13, 18, 19)
Public Service Officer	Primary user (application role)	Centralised complaints, search/filter, assignment, prioritisation and status management	Department queue, Assigned to me, filters, protected workflow actions (CF-08…12, 17)
Administrator	Primary user (application role)	User and category management, monitoring and secure administration	User management, Officer invitations, Service categories, monitoring overview (CF-14, 20, 21)
Department manager	Secondary stakeholder (not an application role)	Visibility of department complaints, workload and unresolved cases	Served by department-scoped queues and admin monitoring. Not a fourth login role (decision recorded in requirements).
Public service organisations	Indirect stakeholder	Accountability, organised records, monitoring	Immutable history, audit events, data-driven categories and departments
Community	Indirect stakeholder	Transparent, responsive handling	Status and history visible to the complainant; resolution date recorded
Lecturer (ICT2232)	Project stakeholder / reviewer	Evidence of each SDLC stage	This evidence pack

3.2 Project objectives

We kept the ten objectives from the proposal. The table below gives the evidence for each one.

Table 3.2: Objectives and evidence of achievement

#	Objective	Evidence of achievement
O1	Convenient digital complaint submission	UC-03 implemented; browser test TC-SUB-01
O2	Citizens can track status and progress	UC-04, timeline, Updates inbox; TC-TRK-01, TC-NTF-01
O3	Officers can manage, assign, prioritise and resolve	UC-05…07; TC-OFF-01
O4	Accurate, centralised records	Database constraints, atomic history and rollback; TC-DB-SUB-05, TC-DB-OFF-08

#	Objective	Evidence of achievement
O5	Timely progress and resolution updates	Durable notifications created in the same transaction as every change; TC-NTF-*
O6	Feedback after resolution	UC-08; TC-FB-*
O7	Administrators manage users and categories	UC-09, UC-10; TC-ADM-, TC-CAT-
O8	Transparency, accountability, efficiency	Append-only history, admin audit log, department queues
O9	Authentication and role-based authorisation	RLS and protected RPCs; security
O10	Extensible to more public services	Categories and departments are data. CF-20 lets an Administrator add services without a code change.

4 Scope, Assumptions and Constraints

4.1 In scope

- Citizen registration, login, logout and profile management; role-based access for Citizen, Officer and Administrator.
- Complaint submission with category, description, map location (or typed coordinates) and private evidence files.
- Routing to the category's department, automatic workload-balanced assignment, and Administrator dispatch of stuck complaints.
- Officer processing: review, assign, prioritise, transfer, start, progress remarks and resolution, with service-level targets.
- Complaint tracking, an append-only history, an Updates inbox with read state, citizen withdrawal and post-resolution feedback.
- Administration: user management, Officer invitations, service-category management, monitoring and analytics with CSV export.

4.2 Out of scope

- Online payments; real-time GPS tracking of field Officers; AI-based complaint classification.
- Automatic legal case management; emergency response and disaster management; automated field dispatch of crews.
- Integration with every government information system; SMS, e-mail or push notifications for complaint updates (updates are delivered in-system).

4.3 Assumptions

Table 4.1: Assumptions and their validation status

Assumption	Status
Users have an internet-connected device and browser	Accepted; responsive UI tested at 375 px and desktop widths
Citizens provide truthful, sufficient information	Accepted. The system validates form and bounds but can't verify truth. Officers review each complaint.
Officers have access and update complaints	Accepted; Officer accounts are invited by an Administrator
Categories and departments are system data	Implemented as database tables; six prototype categories chosen by the team
Evidence follows supported types and size	Enforced: JPEG/PNG/PDF, ≤ 5 MiB, ≤ 5 files per complaint
Demonstrations use fictional data	Enforced for all automated tests and screenshots
No external government integration	Accepted

4.4 Constraints

- Limited academic duration and a five-member student team.
- No access to real government databases or systems; demonstrations use fictional data only.
- Free-tier hosting (Supabase and Vercel), so no paid services, custom servers or SMS gateways.
- Security and privacy must still be designed in, even though this is a prototype.
- The system cannot represent every organisation's workflow; categories and departments are therefore configuration data.

5 Software Process Model and Justification

5.1 Selected model: Agile with Scrum

We used an Agile approach based on Scrum. We kept a prioritised product backlog and refined it during the project. The work was split into five time-boxed sprints. Each sprint ended with a working increment that we could demonstrate, followed by a sprint review and a retrospective, and what we learned in these meetings went back into the backlog.

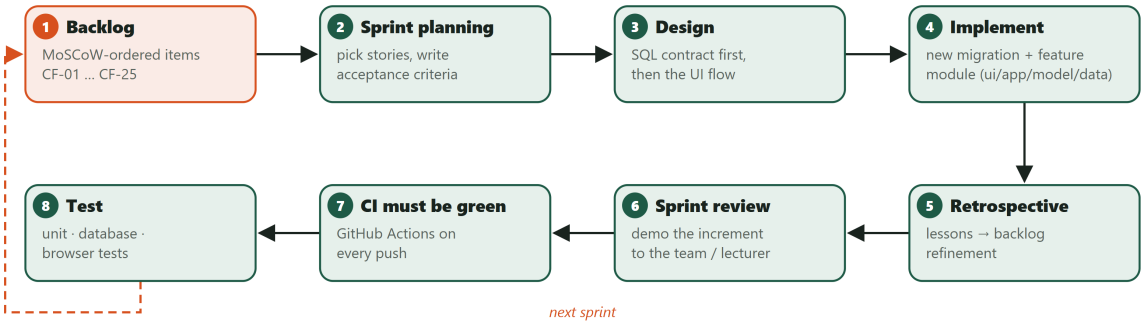


Figure 5.1: Scrum cycle applied in every sprint

5.2 Justification

Table 5.1: Why Scrum fits this project better than a plan-driven model

Project characteristic	Why Scrum fits	Why a plan-driven model (Waterfall/V-model) fits worse
Requirements refined through lecturer and stakeholder feedback	Backlog refinement at the end of every sprint absorbs change. Three changes (CF-22 withdrawal, CF-04 location, CF-14 invitations) were absorbed this way.	Changes after the requirements phase are expensive and need re-baselining
Five-member student team with a fixed deadline	Time-boxed sprints give regular checkpoints and visible progress	One big delivery at the end gives risky feedback very late
Security-sensitive, role-based system	Each sprint adds tests with its features, so defects are found early. The PT409 defect, for example, was found and fixed within Sprint 3.	Testing at the end finds design-level defects late
Needs early demonstrations (Milestones 1–3)	Working increments can be shown at every milestone	Working software appears only near the end

We did not use prototyping on its own because the module asks for documented requirements, design and testing. We also decided against the Spiral model, because its formal risk analysis is too heavy for a team of five working for one semester.

5.3 Development approach and Definition of Done

Every sprint followed the same steps. First we agreed the database side of a feature: the tables, security rules, function signatures and error codes. Then we designed the screens. The implementation added a new SQL migration (applied migrations are never edited) and a feature module with four layers. We wrote tests at three levels and ran the full set of checks locally and in continuous integration before demonstrating the increment.

Table 5.2: Definition of Done used for every backlog item

#	A backlog item is done only when…
1	its acceptance criteria are demonstrably met;
2	authorisation is enforced in the database (grants, Row Level Security and function checks), not only in the user interface;
3	input limits are identical in the client schema and in SQL;
4	allow and deny tests exist (unit, database and a browser journey where there is UI);
5	loading, empty and error states are accessible;
6	lint (0 warnings), strict typecheck, all tests and the build pass locally and in CI;
7	documentation and traceability are updated.

5.4 Project plan and tooling

Table 5.3: Project plan by SDLC phase

Phase	Main activities	Output	Status
Project definition	Problem, stakeholders, objectives, scope	Approved proposal (Milestone 1)	Done
Process planning	Select and justify Scrum; plan	Chapter 5	Done
Agile planning	Backlog, user stories, acceptance criteria, sprint plan	Chapter 6 (Milestone 2)	Done, refined
Requirements	FRs, NFRs, use cases, prioritisation, traceability	Chapter 7 (Milestone 3)	Done, refined by CRs
Design	Architecture, UML, database, UI	Chapter 8 (Milestone 4)	Done
Implementation & testing	Sprints 1–5, unit / database / system tests	Working software, Chapter 9–10	Done
Deployment	Vercel + Supabase, configuration, smoke test	Live URL, Chapter 11	Done
Maintenance & security	Change requests, threat model, controls	Chapters 12–13	Done; lecturer CR pending

Table 5.4: Tooling and configuration management

Concern	Tool / practice
Version control	Git and GitHub (https://github.com/razim2003/civic-flow); feature branches merged to main

Concern	Tool / practice
Continuous integration	GitHub Actions runs install, lint, typecheck, unit tests, database tests, a type-drift check, the build and the Playwright tests on every push
Database change control	Ordered, timestamped SQL migrations; never edited after being applied; hosted rollout only after explicit approval with data checksums
Backlog tracking	Product backlog, roadmap and traceability matrix kept in the repository (docs/)
Code quality	ESLint (0 warnings), strict TypeScript, database types generated from the migrations

6 Agile Planning

6.1 Product backlog

The product backlog lists every requirement as a backlog item with its MoSCoW priority, the sprint that delivered it and its final status. Items CF-22 to CF-25 were added during the project through change requests (Chapter 12).

Table 6.1: Product backlog

CF ID	Backlog item	MoSCoW	Sprint	Status (2026-10-29)
CF-01	User registration (Citizen)	Must	1	Done
CF-02	Secure login/logout	Must	1	Done
CF-03	Role-based access control	Must	1	Done
CF-04	Submit complaint (description, location)	Must	2	Done
CF-05	Complaint category selection	Must	2	Done
CF-06	Unique complaint ID, initial Submitted status	Must	2	Done
CF-07	View submitted complaints / track status	Must	2	Done
CF-08	Officer review and complaint queue	Must	3	Done
CF-09	Assign complaint (officer/department)	Must	3	Done
CF-10	Set complaint priority	Must	3	Done
CF-11	Update status and progress remarks	Must	3	Done
CF-12	Resolve complaint	Must	4	Done
CF-13	Complaint notifications/updates	Must	4	Done (inbox and read state added 2026-10-29)
CF-14	Admin user management	Must	1 (+3)	Done
CF-15	Citizen profile management	Should	1	Done (display name)
CF-16	Upload supporting evidence	Should	2	Done
CF-17	Search and filter complaints	Should	2–3	Done
CF-18	Complaint history	Should	2–3	Done
CF-19	Citizen feedback	Should	4	Done (2026-10-29)
CF-20	Complaint category management	Should	2→4	Done (2026-10-29)
CF-21	Dashboard/reports	Could	4	Basic monitoring done; enhanced reports not planned
CF-22	Citizen withdrawal (approved scope extension)	Should (team-assigned)	2	Done

CF ID	Backlog item	MoSCoW	Sprint	Status (2026-10-29)
CF-23	Automatic workload-balanced assignment (CR-006)	Must	5	Done
CF-24	Administrator dispatch centre (CR-006)	Must	5	Done
CF-25	Service-level targets and overdue flags (CR-006)	Should	5	Done
—	Integration/system testing, security, deployment	Sprint 5 scope	5	In progress

6.2 User stories

We expanded the story titles from the proposal into 24 user stories written as "As a ‹role›, I want ‹capability›, so that ‹benefit›". Story points are our own relative estimates on a scale of 1 to 8.

Table 6.2: User stories

ID	User story	CF	MoSCoW	Pts
US-01	As a citizen, I want to create an account with my name, email and a password, so that I can submit and follow my complaints.	CF-01	Must	3
US-02	As a registered user, I want to log in and log out securely, so that only I can use my account.	CF-02	Must	3
US-03	As any user, I want to see only the pages and data my role allows, so that private information stays protected.	CF-03	Must	5
US-04	As a citizen, I want to describe a public service problem and mark its location on a map or by coordinates, so that the service team can find it.	CF-04	Must	8
US-05	As a citizen, I want to choose a service category, so that my complaint reaches the right department.	CF-05	Must	2
US-06	As a citizen, I want a unique reference and confirmation after submitting, so that I know my complaint was recorded.	CF-06	Must	2
US-07	As a citizen, I want to list, search and open my complaints and see their status, so that I know what is happening.	CF-07, CF-17	Must	5
US-08	As an officer, I want a queue of my department's complaints with filters, so that I can see and pick up work.	CF-08, CF-17	Must	5
US-09	As an officer, I want to assign a complaint to an officer in my department, so that responsibility is clear.	CF-09	Must	3
US-10	As an officer, I want to set a complaint's priority, so that urgent problems are handled first.	CF-10	Must	2
US-11	As an assigned officer, I want to start work and add progress remarks, so that the citizen sees what is being done.	CF-11	Must	5
US-12	As an assigned officer, I want to record resolution details and mark the complaint resolved, so that the case is closed with a record.	CF-12	Must	3
US-13	As a citizen, I want an Updates inbox that shows what changed and which updates I haven't read, so that I don't miss progress on my complaints.	CF-13	Must	5
US-14	As an administrator, I want to view users and change their role, department or active status, and invite officers, so that the team has the right access.	CF-14	Must	8
US-15	As a citizen, I want to change my display name, so that my profile is correct.	CF-15	Should	1

ID	User story	CF	MoSCoW	Pts
US-16	As a citizen, I want to attach photos or PDF documents privately, so that officers have evidence of the problem.	CF-16	Should	8
US-17	As a citizen or officer, I want to see the full history of a complaint, so that every change is transparent.	CF-18	Should	3
US-18	As a citizen, I want to rate and comment on how my resolved complaint was handled, so that the service can improve.	CF-19	Should	3
US-19	As an administrator, I want to add, rename, route and deactivate service categories, so that the system can grow to new services without code changes.	CF-20	Should	5
US-20	As a citizen, I want to withdraw my unresolved complaint with a reason, so that officers don't spend time on a problem that no longer needs action.	CF-22	Should*	5
US-21	As an administrator, I want summary counts of complaints by status and the average feedback rating, so that I can monitor service performance.	CF-21	Could	2
US-22	As a citizen, I want my complaint to be given to a responsible Officer as soon as I submit it, so that it never waits unnoticed.	CF-23	Must	5
US-23	As an administrator, I want to route and assign complaints that are stuck in departments without Officers, so that every complaint has an owner.	CF-24	Must	5
US-24	As an officer, I want to see which complaints are due soon or overdue, so that I handle urgent work first.	CF-25	Should	3

6.3 Acceptance criteria for the major user stories

The acceptance criteria are written in Given/When/Then form, and each one is linked to the automated test that checks it.

Table 6.3: Acceptance criteria (major stories)

Story	Acceptance criteria	Verified by
US-04/05/06 Submit complaint	AC1 Given a signed-in Citizen, when a valid category, description (10–5,000 chars), location (1–300) and coordinates are submitted, then the complaint is saved with a unique ID and status Submitted, and a confirmation shows the reference. AC2 Missing or out-of-range input shows field errors and nothing is saved; the database rejects the same input sent directly. AC3 A retried lost request returns the same complaint (no duplicate). AC4 Non-citizens, inactive and anonymous callers are refused.	TC-SUB-01…03, TC-DB-SUB-01…04
US-07 Track complaint	AC1 My Complaints shows only my own complaints with reference, category and status. AC2 Opening one shows progress steps, history and, if resolved, the resolution text and date. AC3 A reference that is not mine shows no details.	TC-TRK-01…03
US-11 Update status	AC1 Only an Officer of the complaint's department can review it; only the assigned Officer can start, add progress or resolve. AC2 Only Submitted → Under review → In progress → Resolved is accepted. AC3 Each change writes the new status, one history event and one Citizen update atomically. AC4 A stale change is rejected and the Officer's input is kept.	TC-DB-OFF-01…05, TC-OFF-03, TC-OFF-05

Story	Acceptance criteria	Verified by
US-12 Resolve complaint	AC1 Resolution needs 10–5,000 characters and confirmation; status becomes Resolved with date and Officer stored. AC2 The Citizen sees the resolution and receives an update. AC3 Feedback becomes available. AC4 Resolved and withdrawn complaints reject further processing.	TC-OFF-01, TC-DB-OFF-06…07, TC-FB-01
US-13 Updates inbox	AC1 One item per change, newest first, unread items counted in the menu badge. AC2 Mark one or all as read changes only my notifications. AC3 I can never see another user's notifications.	TC-NTF-01, TC-DB-NTF-01…02
US-14 Manage users	AC1 Only an active Administrator can manage users. AC2 Role, department and active state can change; every change is audited. AC3 The last active Administrator cannot be removed, even concurrently. AC4 A deactivated user loses access immediately.	TC-ADM-01…02, TC-DB-ADM-01…04
US-18 Feedback	AC1 On a Resolved complaint the owner submits a 1–5 rating and optional comment once; it is then read-only. AC2 A different second submission is refused. AC3 Feedback on unresolved complaints or by others is refused.	TC-FB-01…02, TC-DB-FB-01…04
US-22/23 Every complaint has an owner	AC1 If the department has active Officers, the least-loaded one is assigned and notified on submission. AC2 Otherwise the complaint is flagged in the Dispatch centre. AC3 An Administrator dispatch with a reason changes route and assignee in one versioned transaction with history and notifications.	TC-DB-DSP-01…07, TC-DSP-01

6.4 Sprint plan and sprint history

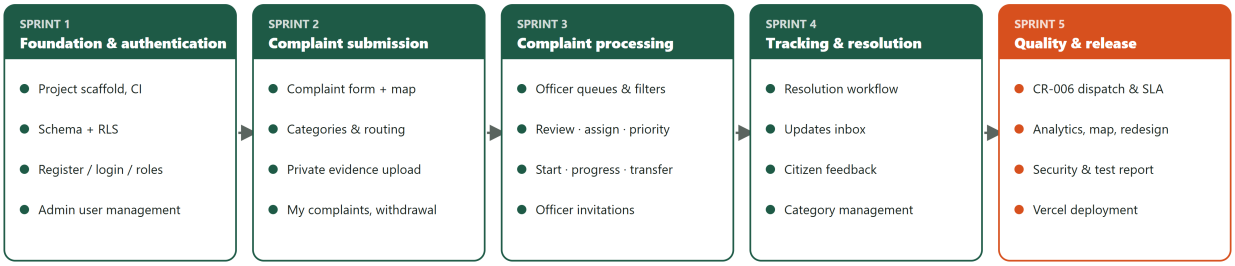


Figure 6.1: Five-sprint roadmap (from the proposal) with the delivered increments

Table 6.4: Sprint plan, delivered increments and review outcomes

Sprint	Goal	Delivered increment	Review / retrospective outcome
1	Foundation & authentication	React/Vite scaffold; Supabase schema with Row Level Security; sign-up, login, recovery; role workspaces; admin user management; CI	Decided to keep sessions in memory only, choosing security over convenience
2	Complaint submission	Complaint form with map; six categories with routing; idempotent submission; private evidence; My Complaints; withdrawal (CR-001)	An empty category list in the demo led to a clear "missing setup" message and a database check before each demo
3	Complaint processing	Officer queues and filters; review, assign, priority, start, progress, transfer; Officer invitations (CR-003)	A concurrency test found an endless retry loop (DEF-01), fixed with a conflict code that is not retried (CR-002)
4	Tracking & resolution	Resolution; Updates inbox with read state; feedback; category management; monitoring	The inbox work was tracked as a separate item so that its progress was reported accurately

Sprint	Goal	Delivered increment	Review / retrospective outcome
5	Quality & release	CR-006 auto-assignment, dispatch centre and service targets; analytics; map; redesign; security and test report; deployment	Hosted use showed that 4 of 5 open complaints had no owner, so CR-006 was done before release; the layout audit found 4 layout defects, which were fixed

6.5 Task allocation

We kept the task allocation from the proposal. Each member owned one module from start to finish and worked on its requirements, design, implementation and testing, as Section 13 of the guidelines requires. Chapter 14 gives the details.

Table 6.5: Task allocation by module

Member	Main area	Backlog items owned
TG/2023/1731 H.M. Amadi Nethma	Requirements & Citizen module	CF-01, 02, 04, 07, 13 (UI), 15, 19 (UI), 22
TG/2023/1710 M.F. Rishma	Complaint management module	CF-05, 06, 09, 10, 16, 25
TG/2023/1757 N.M. Razim Mohamed	Officer module	CF-08, 11, 12, 17 (Officer), Officer map
TG/2023/1761 M.S.M. Sharfan	Administration & database	CF-03 (database), 14, 18, 20, 21, 23, 24
TG/2023/1776 V. Kayaththiry	Quality, security & deployment	Test plan and suites, NFRs, security controls, deployment, maintenance CR

7 Requirements Engineering

We gathered the requirements from the allocated domain, the stakeholder analysis and what we know about how local authorities in Sri Lanka handle complaints, and refined them later through sprint reviews and change requests. Every requirement has a fixed identifier (CF-xx for functional and NFR-xx for non-functional requirements), which is used in the backlog, the design, the code, the tests and this report.

7.1 Actors

Table 7.1: Actors

Actor	Kind	Description
Citizen	Primary, application role	Registers, submits and tracks own complaints, receives updates, gives feedback, may withdraw an unresolved complaint
Public Service Officer	Primary, application role	Belongs to one department; reviews, assigns, prioritises and transfers its complaints; when assigned, starts work, adds progress and resolves
Administrator	Primary, application role	Manages users and Officer invitations, service categories, dispatch of stuck complaints, monitoring and analytics; does not process complaints
Supabase Auth	Secondary system actor	Stores credentials, verifies e-mail, issues sessions, sends invitation and recovery e-mails
OpenStreetMap	Secondary system actor	Supplies map tiles for location selection

7.2 Functional requirements

The specification has 25 functional requirements (the guidelines ask for at least 12). Each one describes a single behaviour that can be tested, and the last column lists the test cases that check it.

Table 7.2: Functional requirements

ID	Requirement ("The system shall …")	Priority	Source	Verified by
CF-01	… allow a visitor to register a Citizen account with name, email and a password of at least 12 characters, and shall assign the Citizen role only.	Must	p. 11	TC-AUTH-02, TC-DB-FND-01
CF-02	… allow registered users to log in and log out, and shall deny protected pages and data after logout.	Must	p. 11	TC-AUTH-03, TC-AUTH-06
CF-03	… enforce role-based access to every protected function in the database, independently of the user interface.	Must	p. 11	TC-AUTH-04, TC-RBAC-01, TC-DB-FND-02…04
CF-04	… allow a Citizen to submit a complaint with a description (10–5,000 characters), location text (1–300) and a latitude/longitude selected on a map or entered as numbers.	Must	p. 11	TC-SUB-01, TC-SUB-02, TC-DB-SUB-03
CF-05	… require the Citizen to select one active service category and shall route the complaint to that category's department.	Must	p. 11	TC-SUB-04, TC-DB-SUB-01, TC-DB-SUB-07

ID	Requirement ("The system shall …")	Priority	Source	Verified by
CF-06	… generate a unique identifier for each complaint, set its initial status to Submitted and show a confirmation with its reference.	Must	p. 11	TC-SUB-01, TC-DB-SUB-01
CF-07	… allow a Citizen to list, open and track only their own complaints, including status, progress and resolution.	Must	p. 11	TC-TRK-01…03, TC-DB-SUB-06
CF-08	… allow an active Officer to view and open the complaints of their own department, and complaints assigned to them.	Must	p. 11	TC-OFF-02, TC-DB-SUB-06
CF-09	… allow an Officer to assign a department complaint to an active Officer of the same department, and to transfer an unstarted complaint to another department with a reason.	Must	p. 11	TC-OFF-01, TC-OFF-04, TC-DB-OFF-04, TC-DB-OFF-09
CF-10	… allow an Officer to set complaint priority to Low, Normal, High or Urgent.	Must	p. 11	TC-OFF-01, TC-OFF-08
CF-11	… allow only valid status transitions (Submitted → Under review → In progress → Resolved) and allow only the assigned Officer to start work and record progress remarks.	Must	p. 11	TC-OFF-01, TC-OFF-05, TC-DB-OFF-05
CF-12	… allow the assigned Officer to resolve an In progress complaint with resolution details (10–5,000 characters), storing the resolution date and Officer.	Must	p. 11	TC-OFF-01, TC-DB-OFF-06
CF-13	… record a durable update for the Citizen with every complaint change, and shall let the Citizen list updates and mark them read.	Must	p. 11	TC-NTF-01, TC-DB-NTF-01…05
CF-14	… allow an Administrator to view users, change role, department and active state, and invite Officer accounts. It shall audit every change and protect the last active Administrator.	Must	p. 11	TC-ADM-01…03, TC-DB-ADM-01…04
CF-15	… allow every user to change their own display name only.	Should	p. 6	TC-AUTH-08
CF-16	… allow a Citizen to attach up to five private JPEG/PNG/PDF files (≤ 5 MiB each) to their Submitted complaint, downloadable only by authorised users.	Should	p. 11	TC-SUB-01, TC-DB-EV-01…06
CF-17	… allow searching by reference or description and filtering by status (Citizen), and by status, category, priority and assignment (Officer).	Should	p. 6	TC-TRK-01, TC-OFF-02, TC-OFF-09
CF-18	… maintain an append-only history of every complaint change, visible to authorised users.	Should	p. 11	TC-SUB-01, TC-DB-OFF-01
CF-19	… allow the owning Citizen to submit exactly one rating (1–5) with an optional comment (≤ 2,000 characters) on a Resolved complaint.	Should	p. 11	TC-FB-01, TC-FB-02, TC-DB-FB-01…05
CF-20	… allow an Administrator to create, edit, route and deactivate service categories without deleting them, auditing every change.	Should	p. 11	TC-CAT-01, TC-CAT-02, TC-DB-CAT-01…06
CF-21	… provide Administrators with read-only complaint monitoring information (counts by status, average rating).	Could	p. 11	TC-MON-01, TC-U-MON-01
CF-22	… allow the owning Citizen to withdraw an unresolved complaint with a 10–2,000 character reason, retaining its record, evidence and history. (approved scope extension)	Should	change record	TC-WD-01…04, TC-DB-WD-01…05
CF-23	… automatically assign each new complaint to the active Officer with the fewest open assignments in the routed department, and notify that Officer. (CR-006)	Must	CR-006	TC-DB-DSP-01…03, TC-DSP-03

ID	Requirement ("The system shall …")	Priority	Source	Verified by
CF-24	… allow an active Administrator to dispatch an open complaint: route it to another active department/category and/or assign an active Officer of that department, with a recorded 10–2,000 character reason. (CR-006)	Must	CR-006	TC-DSP-01, TC-DB-DSP-04…09
CF-25	… apply a service-level target per priority (urgent 1 day, high 3, normal 7, low 14), flag open complaints that are due soon or overdue, and report on-target resolution. (CR-006)	Should	CR-006	TC-U-SLA-01, TC-DSP-02, TC-DSP-03

7.3 Non-functional requirements

The quality goals in the proposal were turned into eleven non-functional requirements, each with a measurable acceptance threshold. They cover performance, usability, security, reliability, integrity, availability, maintainability, scalability and privacy.

Table 7.3: Non-functional requirements

ID	Category	Requirement	Measure / acceptance threshold	Verification method	Status
NFR-01	Performance	List screens shall load a bounded page of results, never the whole table.	Citizen/Officer lists ≤ 10 rows per request; admin users ≤ 20; ownership, department and status columns indexed	Code review of range() calls; indexes in migration 001	Verified
NFR-02	Performance	Common screens should be usable quickly on the demo deployment.	Proposed: first meaningful content ≤ 3 s and page navigation ≤ 1.5 s on a typical broadband connection with demo data	Manual timing in browser DevTools on the deployed URL	Pending (deployment)
NFR-03	Usability / accessibility	Forms shall be usable by keyboard and screen reader, and pages shall fit a phone screen.	Every input has a programmatic label; errors announced (role="alert"); skip link; no horizontal scroll at 375 px width	Playwright getByLabel/role queries; mobile overflow tests TC-UX-01, TC-TRK-01, TC-OFF-06	Verified
NFR-04	Security – authentication	Protected functions shall require an authenticated, active account.	100% of RPCs deny the anonymous role; inactive accounts denied immediately	TC-DB-FND-03, TC-DB-ADM-04, security advisor	Verified
NFR-05	Security – authorisation	Users shall access only data and actions permitted by role, ownership and department.	RLS enabled on 9/9 application tables; 0 direct browser INSERT/UPDATE/DELETE grants	TC-DB-FND-02, TC-DB-OFF-03, TC-DB-FB-05	Verified
NFR-06	Reliability / integrity	Every complaint change shall be atomic and concurrency-safe.	Each version change writes exactly 1 event and 1 notification, or nothing; stale writes rejected (HTTP 409); retries are idempotent	TC-DB-SUB-05, TC-DB-OFF-08, TC-DB-WD-05, TC-INT-04…06	Verified
NFR-07	Data integrity / validation	Invalid or incomplete records shall be rejected in both client and database.	Identical bounds in Zod schemas and SQL CHECK/RPC validation	Unit TC-U- and database TC-DB--bounds tests	Verified

ID	Category	Requirement	Measure / acceptance threshold	Verification method	Status
NFR-08	Availability	The deployed system shall be reachable during the evaluation.	Smoke-test checklist (08 §8.6) passes within 24 h before the demo	Manual smoke test	Pending (deployment)
NFR-09	Maintainability	Code shall be modular and consistently checked.	Feature modules with ui/application/model/data layers; ESLint 0 warnings; strict TypeScript; CI green on main	CI workflow	Verified
NFR-10	Scalability / extensibility	New categories, departments and users shall be added without code changes.	Category added via the UI appears in the Citizen form	TC-CAT-01, TC-DB-CAT-02	Verified
NFR-11	Privacy	Complaint data and evidence shall be visible only to authorised users; demo data is fictional.	Evidence bucket private; no public complaint feed; tests use fictional identities	TC-DB-EV-01, TC-TRK-03, security review	Verified

7.4 Use case diagram

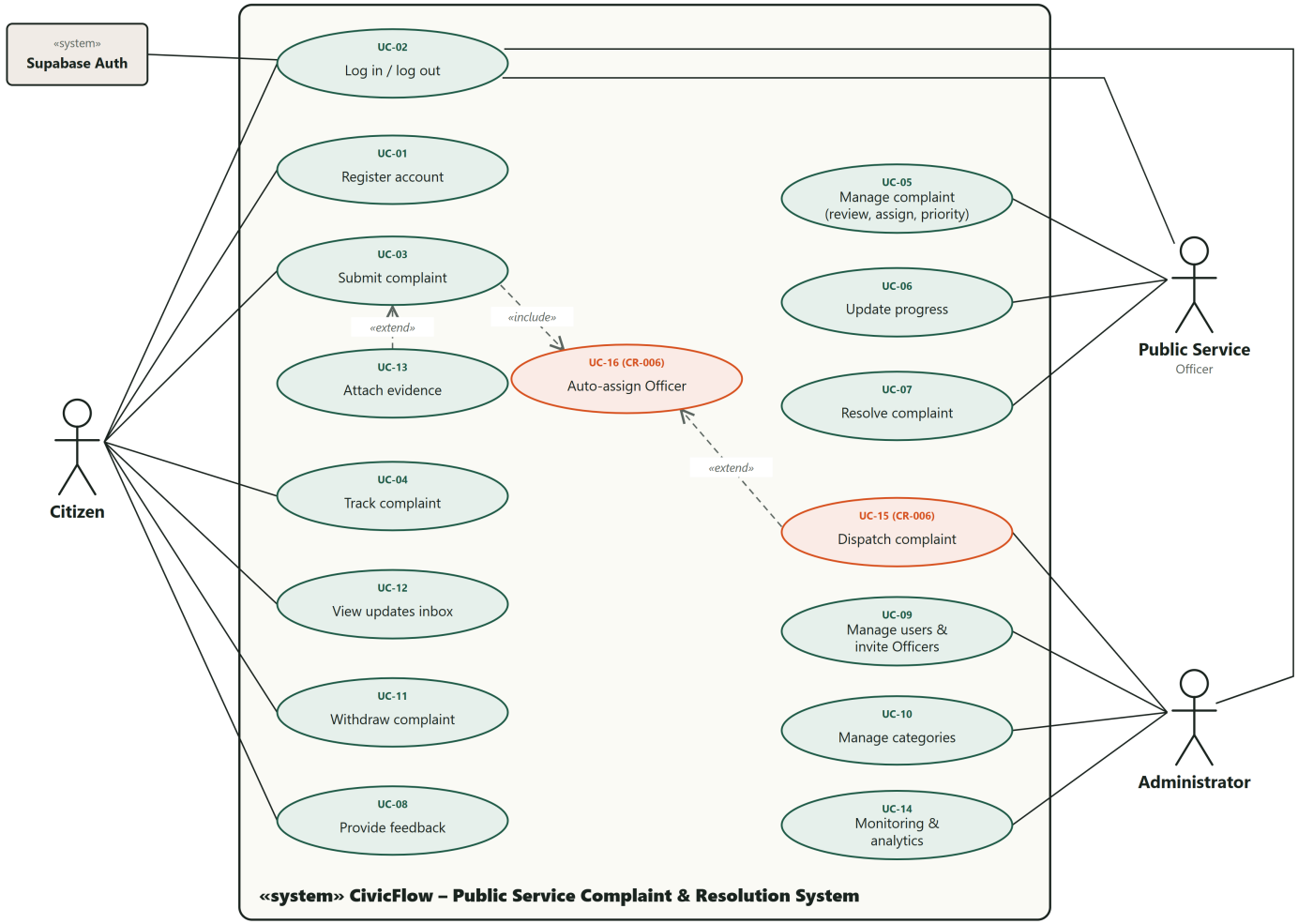


Figure 7.1: UML use case diagram (UC-15 and UC-16 were added by change request CR-006)

7.5 Use case descriptions

Table 7.4: UC-03 Submit complaint (CF-04, 05, 06, 16, 23)

Element	Description
Primary actor	Citizen
Preconditions	Signed in with an active Citizen account; at least one active category exists
Main flow	1) Open Make a complaint. 2) Select a service category. 3) Describe the issue. 4) Select the location on the map, type coordinates or use "Use my current location", and describe the location. 5) Submit. 6) The system validates the input, routes the complaint to the category's department, assigns the least-loaded active Officer, records history event 1 and updates, and shows the reference. 7) Optionally attach evidence (UC-13).
Alternative flows / exceptions	3a/4a Invalid or missing fields: field errors, nothing saved. 5a Network failure: the same request key is retried and no duplicate is created. 6a No Officer in the department: the complaint is left unassigned and flagged for dispatch. 7a Upload fails: the complaint stays saved and the file can be retried.
Postcondition	The complaint is Submitted, visible only to its owner, the department's Officers and Administrators.

Table 7.5: UC-04 Track complaint (CF-07, 17, 18)

Element	Description
Primary actor	Citizen
Preconditions	Signed in
Main flow	1) Open My complaints. 2) Search by reference or description and filter by status. 3) Open a complaint. 4) View the progress line, details, location, evidence, resolution and history.
Alternative flows / exceptions	The reference does not exist or belongs to someone else: "Complaint unavailable" with no details.
Postcondition	No data is changed.

Table 7.6: UC-05 Manage complaint (CF-08, 09, 10)

Element	Description
Primary actor	Officer
Preconditions	Active Officer of an active department; complaint in that department and not closed
Main flow	1) Open the department queue and filter. 2) Open a complaint. 3) Begin review (Submitted → Under review). 4) Assign an Officer of the department. 5) Set the priority. Each step saves one history event and one Citizen update.
Alternative flows / exceptions	4a Transfer to another department with a reason (Submitted or Under review only). Exceptions: complaint changed meanwhile (stale-version message, input kept); target Officer inactive or in another department (refused).
Postcondition	Assignment, priority and status are saved with history.

Table 7.7: UC-06/07 Update and resolve complaint (CF-11, 12)

Element	Description
Primary actor	Assigned Officer
Preconditions	Status Under review (to start) or In progress
Main flow	1) Start work, optionally with a remark. 2) Add progress updates (1–2,000 characters). 3) Resolve with resolution details (10–5,000 characters) and confirmation.
Alternative flows / exceptions	Not the assignee, invalid transition, empty remark or short resolution: refused atomically with no partial update.
Postcondition	Status Resolved; date and Officer stored; the Citizen is notified and can give feedback.

Table 7.8: UC-08 Provide feedback (CF-19)

Element	Description
Primary actor	Citizen (owner)
Preconditions	Complaint Resolved with no feedback yet
Main flow	1) Open the resolved complaint. 2) Choose a 1–5 rating. 3) Optionally add a comment. 4) Submit.
Alternative flows / exceptions	No rating: client error. Already submitted: "Feedback was already submitted". Not resolved: refused.
Postcondition	Feedback shown read-only to the owner, the department's Officers and Administrators.

Table 7.9: UC-09 Manage users and invite Officers (CF-14)

Element	Description
Primary actor	Administrator
Preconditions	Active Administrator
Main flow	1) Open User management. 2) Search. 3) Edit role, department or active state, or Create Officer account (name, e-mail, department). 4) Save; the change is audited and the invitee sets their own password.
Alternative flows / exceptions	Removing the last active Administrator: refused. An Officer with open assignments cannot be moved or deactivated until the work is reassigned.
Postcondition	Access changed and audited.

Table 7.10: UC-15 Dispatch complaint (CF-24)

Element	Description
Primary actor	Administrator
Preconditions	Complaint is Submitted, Under review or In progress
Main flow	1) Open the Dispatch centre; coverage cards show departments with no Officers and the Needs-attention filter lists unstaffed, unassigned or overdue complaints. 2) Choose Dispatch. 3) Select department, category and optionally an Officer. 4) Enter a reason and confirm.
Alternative flows / exceptions	Work in progress cannot change department or lose its assignee. Changed meanwhile: refresh. Closed complaints: refused.

Element	Description
Postcondition	Route and/or assignee changed; history records the Administrator's reason; Citizen and Officer notified.

The other use cases (UC-01 Register, UC-02 Log in/out, UC-10 Manage categories, UC-11 Withdraw complaint, UC-12 View updates inbox, UC-13 Attach evidence and UC-14 Monitoring) use the same format and are described in the repository specification (docs/sdlc/04-requirements-specification.md).

7.6 Prioritisation (MoSCoW)

We used MoSCoW because it fits the sprint order well: within each sprint the Must items were scheduled before the Should items, and the Could item was done last.

Table 7.11: MoSCoW prioritisation

Priority	Requirements	Rationale
Must have	CF-01 – CF-14, CF-23, CF-24	Without these the core complaint lifecycle, its security or its ownership does not work
Should have	CF-15 – CF-20, CF-22, CF-25	Important for usability, evidence, transparency and administration, but the lifecycle works without them
Could have	CF-21 (monitoring and analytics)	Valuable for management; delivered after all Must and Should items
Won't have (this release)	AI classification, GPS Officer tracking, payments, emergency response, broad government integration, SMS/e-mail alerts	Out of scope (Chapter 4) or beyond the free-tier constraints

7.7 Requirements traceability matrix

The matrix links each requirement to its user story, use case, design artefact, implementation (database migration and frontend module) and tests. Migration numbers are shortened: 001 is 20261019000100_foundation.sql and 015 is 20261029000400_dispatch_and_coverage.sql.

Table 7.12: Requirements traceability matrix

Req	Story	Use case	Design	Implementation (DB → frontend)	Tests	Status
CF-01	US-01	UC-01	Auth flow, handle_new_user trigger	001 trigger → features/auth	TC-AUTH-02, TC-U-AUTH-01, TC-DB-FND-01	Done
CF-02	US-02	UC-02	Memory-only session decision	Supabase Auth → features/auth/AuthProvider	TC-AUTH-03, TC-AUTH-06, TC-U-AUTH-02	Done
CF-03	US-03	all	Access matrix (DB design)	001–002 RLS, private.active_role → route guards	TC-AUTH-04, TC-RBAC-01, TC-DB-FND-02…04	Done
CF-04	US-04	UC-03	Seq. SD-1, shared/location	004 submit_complaint → complaints/NewComplaintPage	TC-SUB-01…02, TC-LOC-01…02, TC-U-CMP-01, TC-DB-SUB-03	Done
CF-05	US-05	UC-03	Category routing	004, 006 → category selector	TC-SUB-04…05, TC-DB-SUB-07	Done
CF-06	US-06	UC-03	SD-1	004 → submission success view	TC-SUB-01, TC-SUB-03, TC-DB-SUB-01…02	Done
CF-07	US-07	UC-04	Citizen screens	004 RLS → MyComplaintsPage, ComplaintDetailPage	TC-TRK-01…03	Done
CF-08	US-08	UC-05	Officer screens	004 RLS → officer/OfficerQueue	TC-OFF-02, TC-OFF-07	Done
CF-09	US-09	UC-05	State diagram, SD-2	009 assign_complaint, transfer_complaint → OfficerActions	TC-OFF-01, TC-OFF-04, TC-DB-OFF-04, TC-DB-OFF-09	Done
CF-10	US-10	UC-05	—	009 set_complaint_priority	TC-OFF-01, TC-OFF-08	Done
CF-11	US-11	UC-06	Activity diagram, state diagram	009 review/start/add_progress	TC-OFF-01, TC-OFF-03, TC-OFF-05, TC-DB-OFF-01…05	Done
CF-12	US-12	UC-07	SD-2	010 resolve_complaint	TC-OFF-01, TC-DB-OFF-06…07	Done
CF-13	US-13	UC-12	notifications table	004/009 writes, 013 read policy + mark_notifications_read → features/notifications	TC-NTF-01, TC-U-NTF-01, TC-DB-NTF-01…05	Done
CF-14	US-14	UC-09	Admin audit, Edge Function	002 admin_update_user, provision-officer → features/admin	TC-ADM-01…03, TC-U-INV-01, TC-DB-ADM-01…04, TC-INT-08	Done
CF-15	US-15	—	—	002 update_my_profile → features/profiles	TC-AUTH-08	Done

Req	Story	Use case	Design	Implementation (DB → frontend)	Tests	Status
CF-16	US-16	UC-13	Evidence storage design	005 bucket, policies, reserve/finalize → features/evidence	TC-SUB-01, TC-U-EV-01, TC-DB-EV-01…06	Done
CF-17	US-07, US-08	UC-04, UC-05	—	Data adapters (bounded ilike, filters)	TC-TRK-01, TC-OFF-02, TC-OFF-09	Done
CF-18	US-17	UC-04	complaint_events	004–011 event writes → timeline	TC-SUB-01, TC-DB-OFF-01	Done
CF-19	US-18	UC-08	SD-3, complaint_feedback	012 submit_feedback → features/feedback	TC-FB-01…02, TC-U-FB-01, TC-DB-FB-01…05	Done
CF-20	US-19	UC-10	Category rules	014 admin_upsert_category → features/categories	TC-CAT-01…02, TC-U-CAT-01, TC-DB-CAT-01…06	Done
CF-21	US-21	UC-14	Analytics view	015 admin_analytics, community_impact → features/analytics, admin/AdminOverview	TC-MON-01, TC-DSP-02, TC-U-MON-01, TC-U-ANL-01, TC-DB-DSP-10…12	Done (enhanced)
CF-23	US-22	UC-03	SD-4	015 submit_complaint, private.pick_officer	TC-DB-DSP-01…03, TC-DSP-03	Done
CF-24	US-23	UC-15	SD-4	015 admin_dispatch_complaint → features/dispatch	TC-DSP-01, TC-U-DSP-01, TC-DB-DSP-04…09	Done
CF-25	US-24	UC-05, UC-15	SLA table	shared/sla.ts + private.sla_hours	TC-U-SLA-01, TC-DSP-02, TC-DSP-03	Done
CF-22	US-20	UC-11	State diagram	007–008, 011 withdraw_complaint → WithdrawalPanel	TC-WD-01…04, TC-U-WD-01, TC-DB-WD-01…05	Done
NFR-04/05	US-03	all	Security	Grants, RLS, definer RPCs	TC-DB-FND-, TC-INT-	Done
NFR-06	—	—	Lock order, version checks	All workflow RPCs	TC-DB-*-rollback, TC-INT-04…06	Done

7.8 Requirements validation

Table 7.13: Requirements validation activities

Activity	What was done	Outcome
Proposal review	The proposal (Milestones 1–3) was submitted as the requirements baseline	Accepted as the baseline for development
Stakeholder refinement	The project owner requested withdrawal, Administrator-created Officers and the location refinement; hosted testing revealed unowned complaints	Recorded as change requests CR-001 – CR-006 with impact analysis (Chapter 12)
Prototype walkthroughs	Each increment was demonstrated in the browser and screenshots were reviewed	Defects such as the empty category list were found and fixed
Consistency checks	Client and database input limits compared; tests assert both	No mismatches outstanding
Not yet done	Validation with real citizens or Officers	Labelled as an open assumption

8 System Design

8.1 High-level system architecture

CivicFlow uses a three-tier client-server architecture. The presentation tier is a React single-page application served from Vercel's CDN. Supabase provides the application tier (authentication, a REST/RPC API and the business functions that run on the server) and the data tier (PostgreSQL and private file storage).

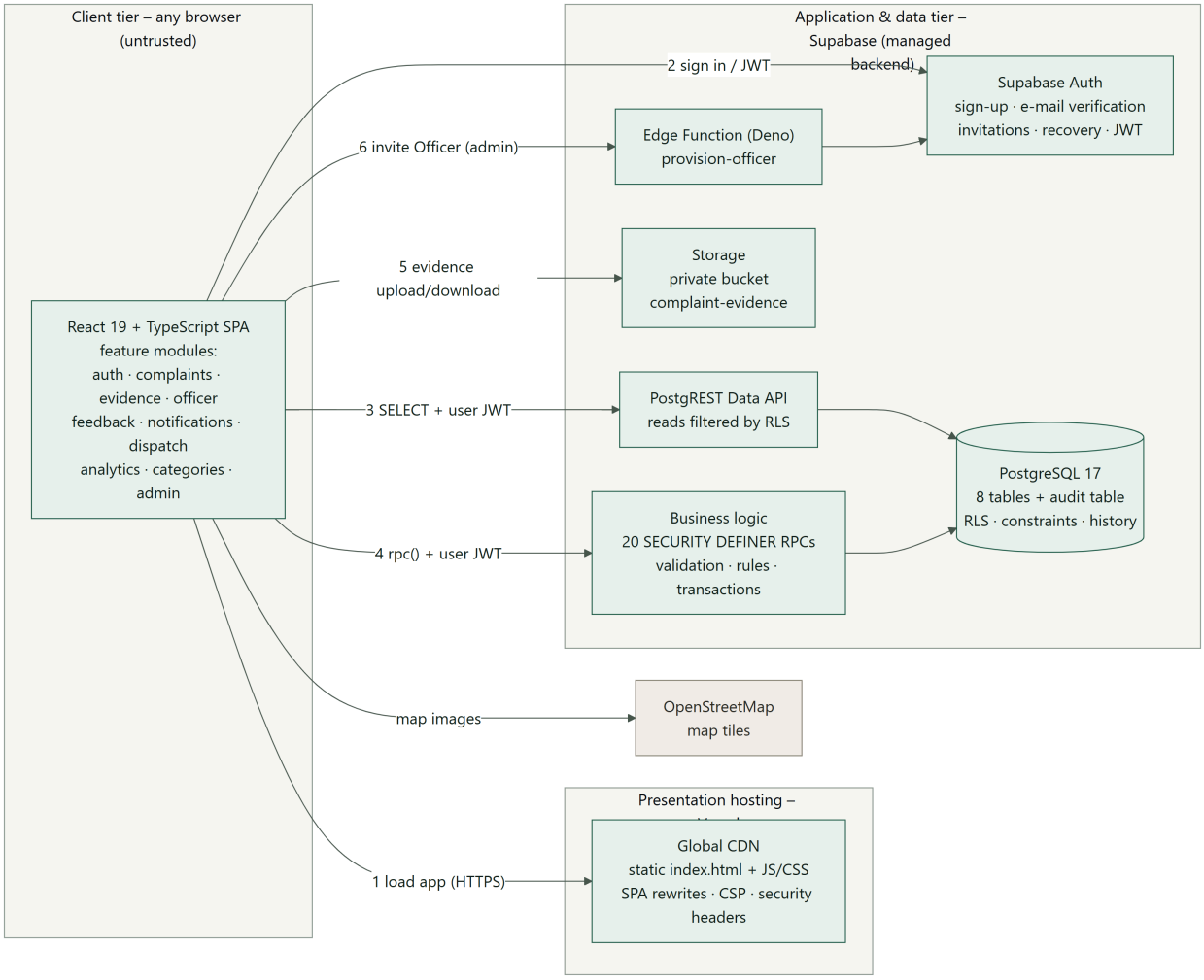


Figure 8.1: High-level system architecture

Trust boundary

We treat the browser as untrusted, so every authorisation decision is made on the server by PostgreSQL grants, Row Level Security policies, checks inside SECURITY DEFINER functions and Storage policies. Hiding buttons and guarding routes in the interface only makes the application easier to use; it does not protect any data.

8.2 Frontend architecture

The frontend code is grouped by feature (auth, complaints, evidence, officer, feedback, notifications, dispatch, analytics, categories, admin and landing). Every feature has the same four layers, shown below. Screens never call the backend directly, and the business rules can be unit-tested without React or a network connection.

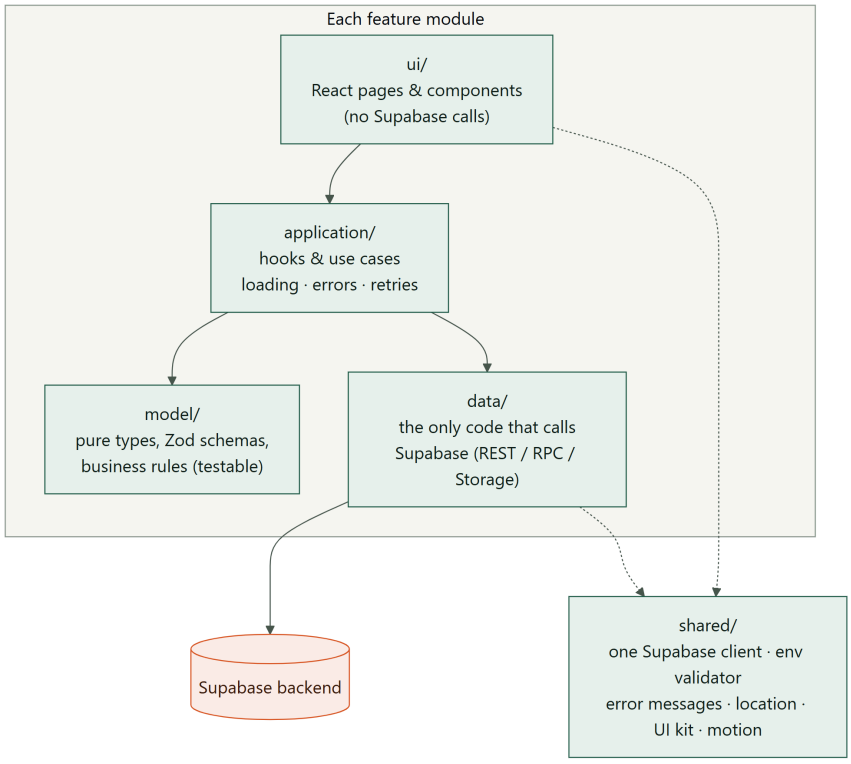


Figure 8.2: Layered structure of every feature module

8.3 Class / domain model

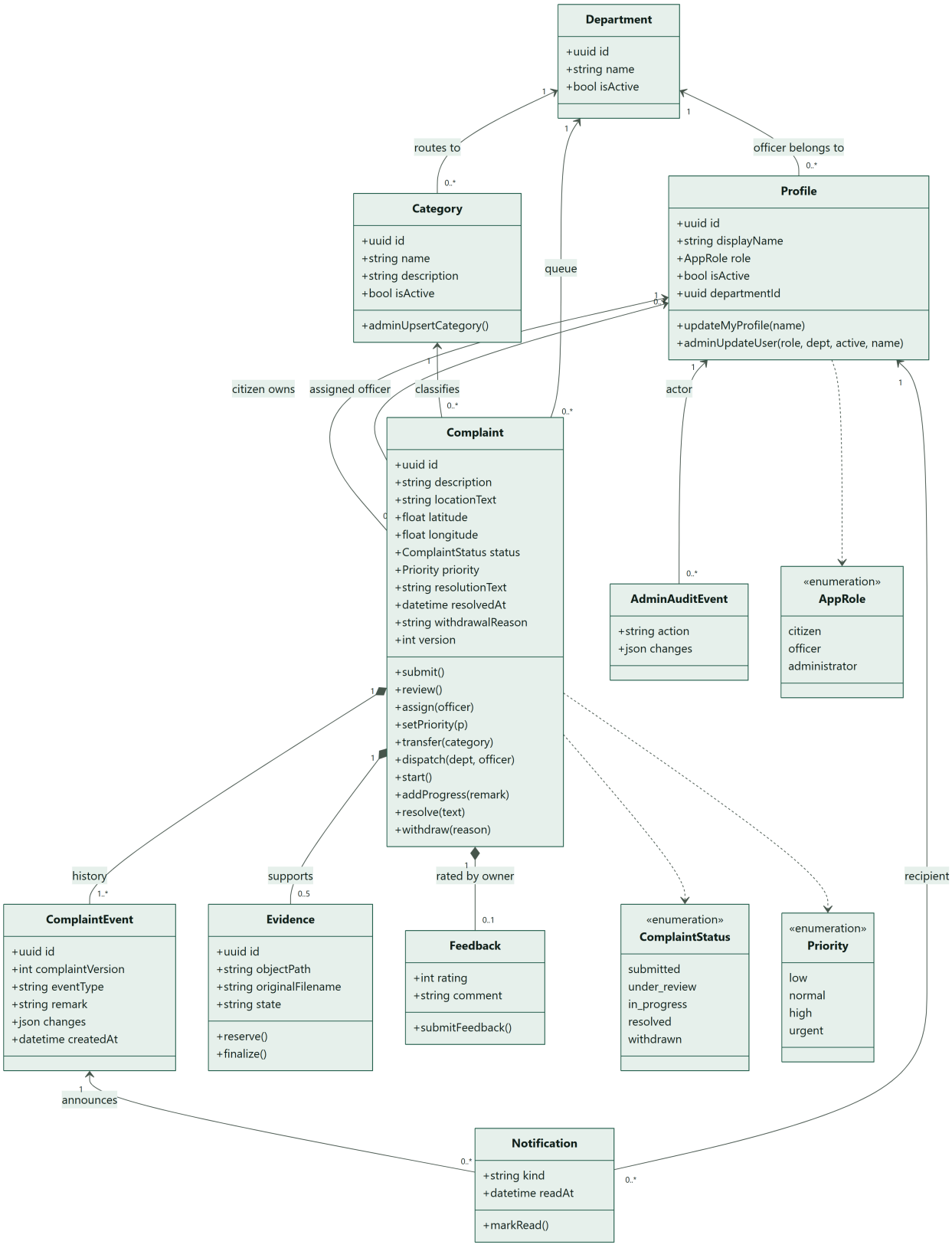


Figure 8.3: UML class diagram of the domain model (operations are the only way to change data)

Browsers have no direct insert, update or delete rights on any table. Every change goes through one of the operations shown in the diagram, each implemented as a protected database function.

8.4 Sequence diagrams

We drew sequence diagrams for the four most important interactions. SD-1 covers complaint submission, including validation, protection against duplicate submissions and automatic assignment. SD-2 shows an Officer resolving a complaint with row locking and version checks. SD-3 shows Administrator dispatch, which was added by CR-006, and SD-4 shows a Citizen giving feedback.

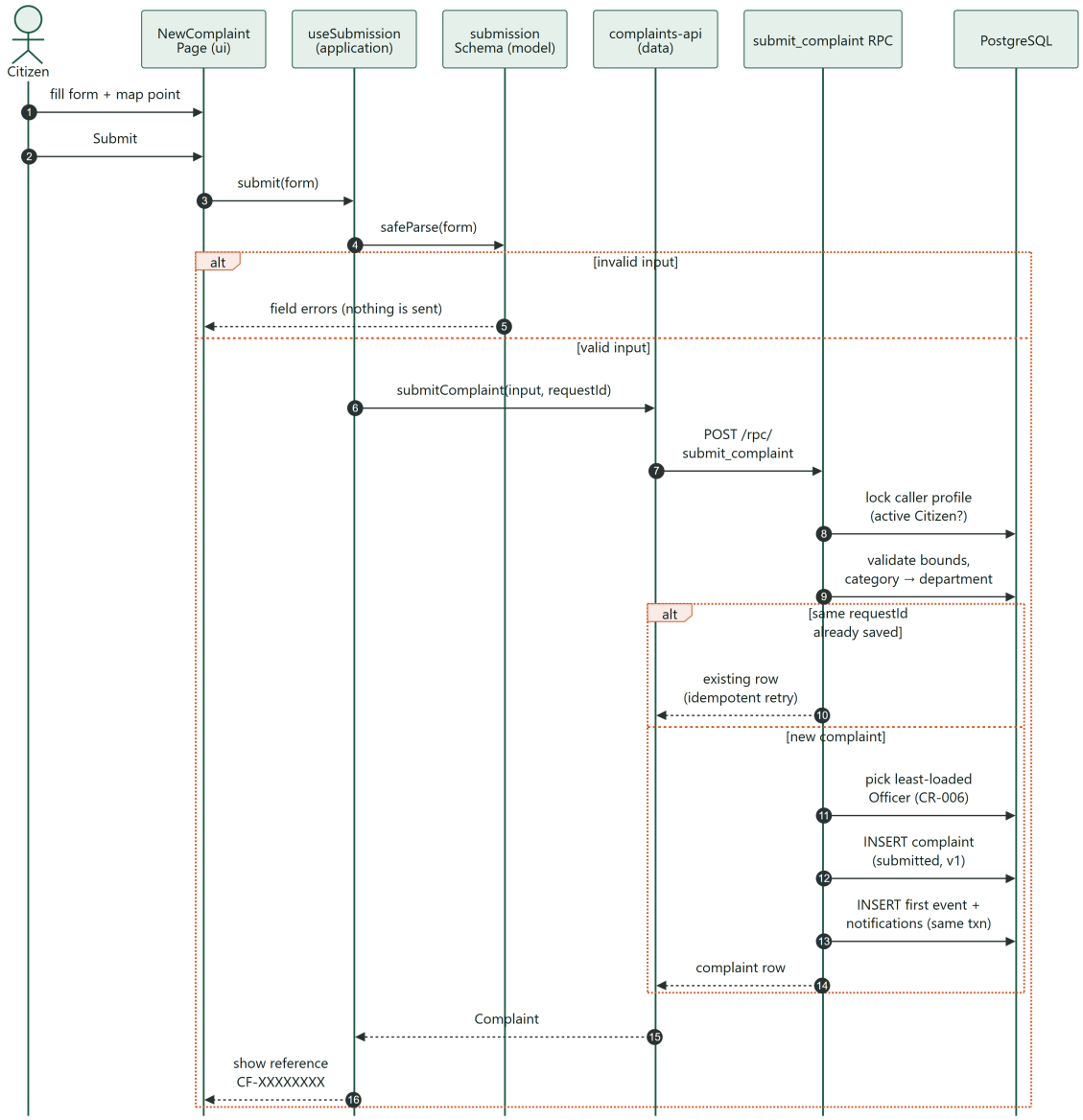


Figure 8.4: SD-1: Citizen submits a complaint (UC-03)

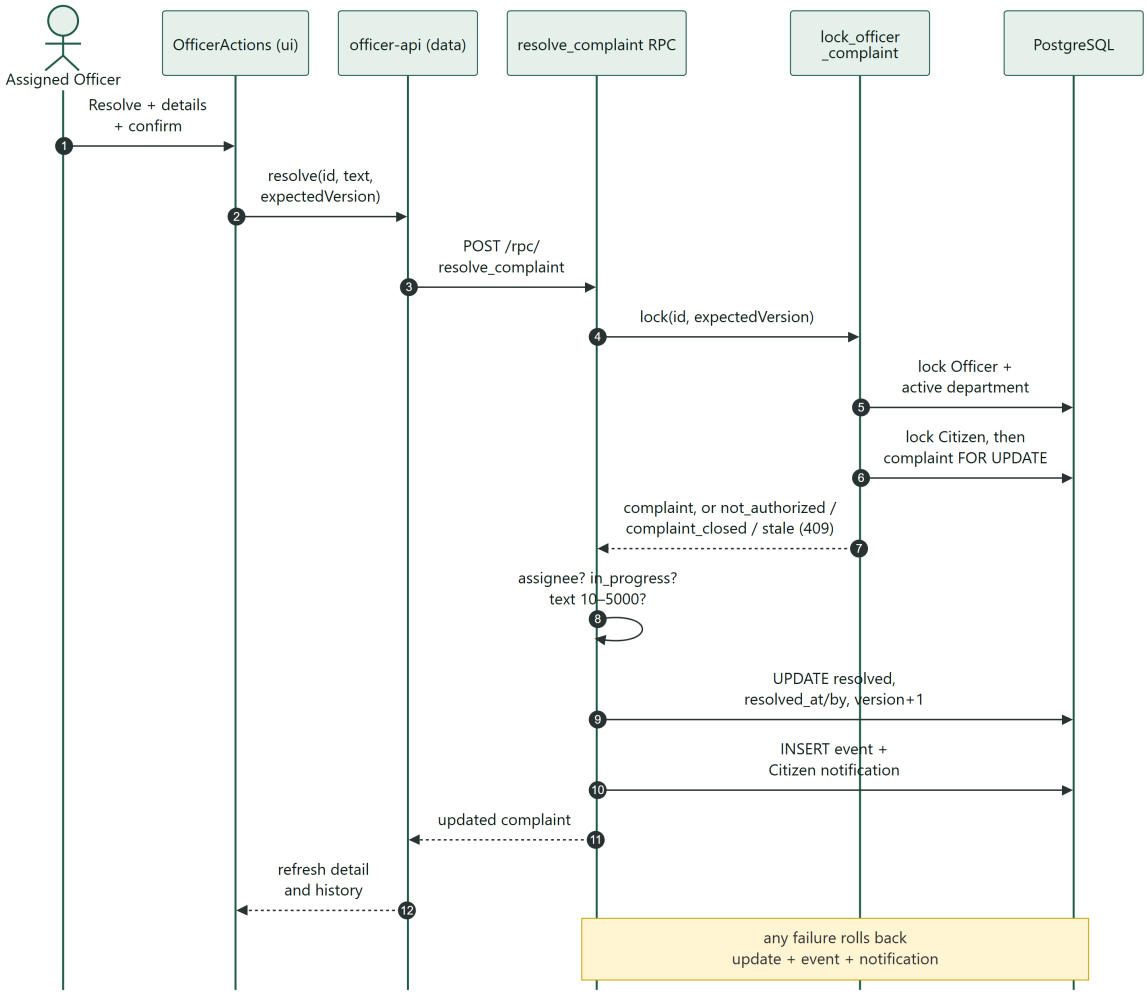


Figure 8.5: SD-2: Assigned Officer resolves a complaint (UC-07)

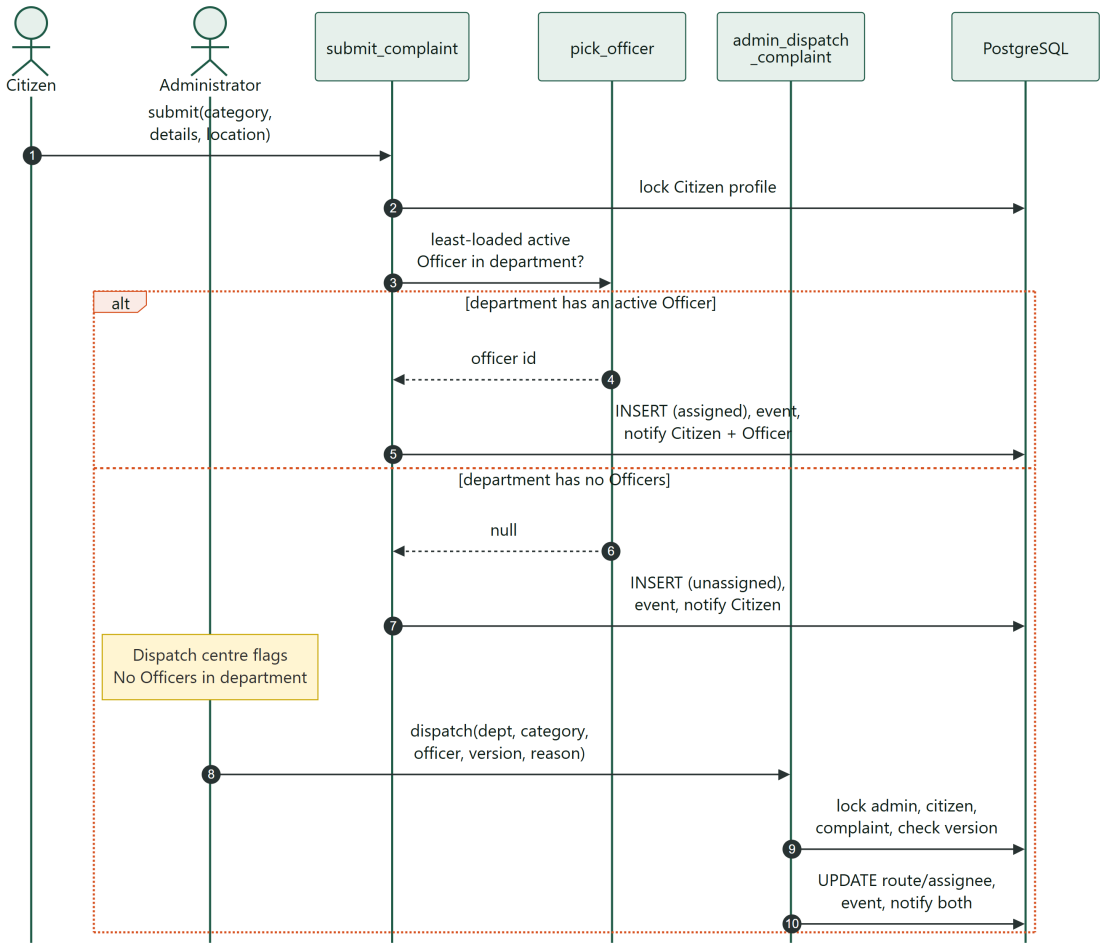


Figure 8.6: SD-3: Automatic assignment and Administrator dispatch (CR-006, UC-15)

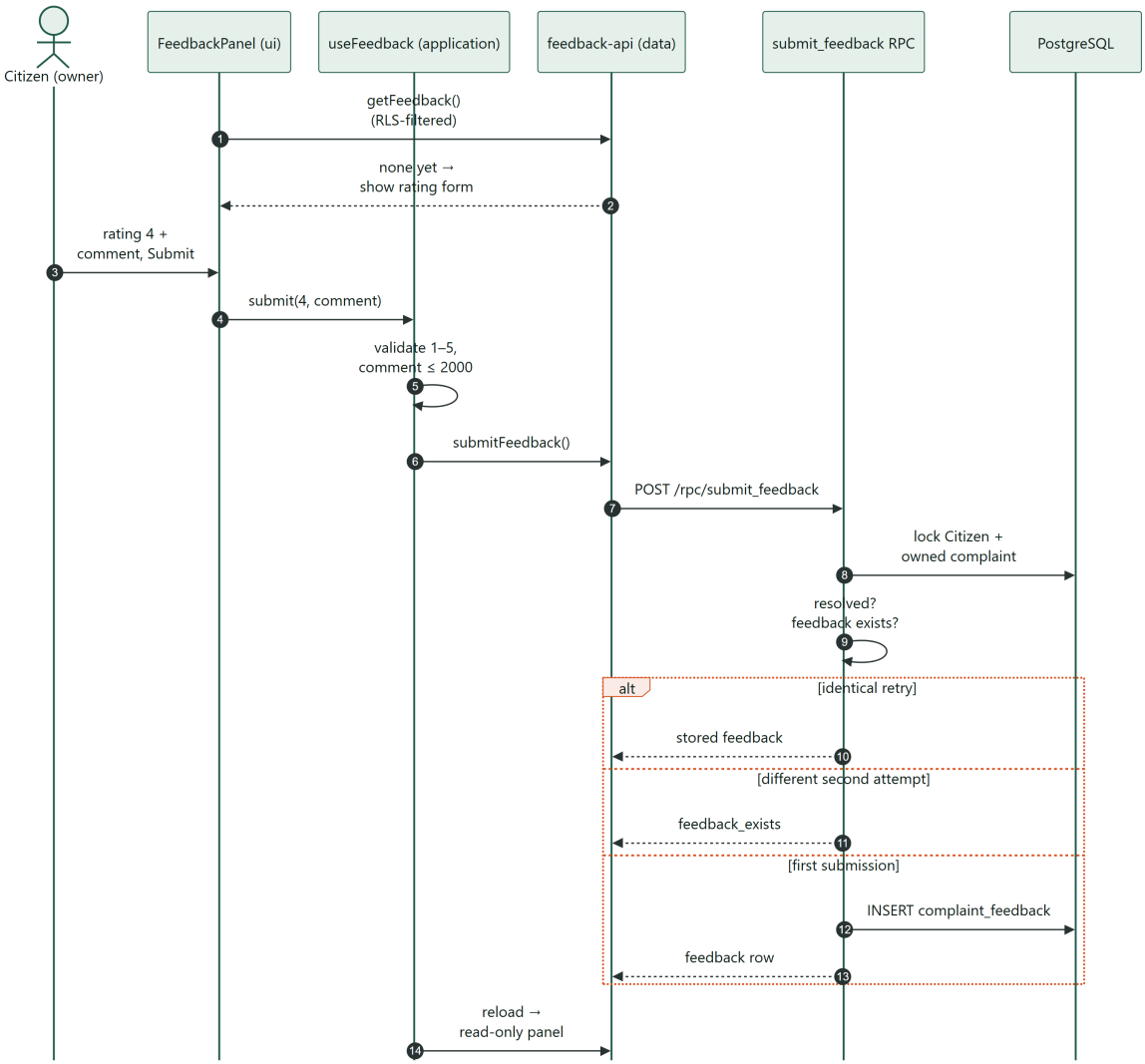


Figure 8.7: SD-4: Citizen gives feedback (UC-08)

8.5 Activity diagram of the complaint workflow

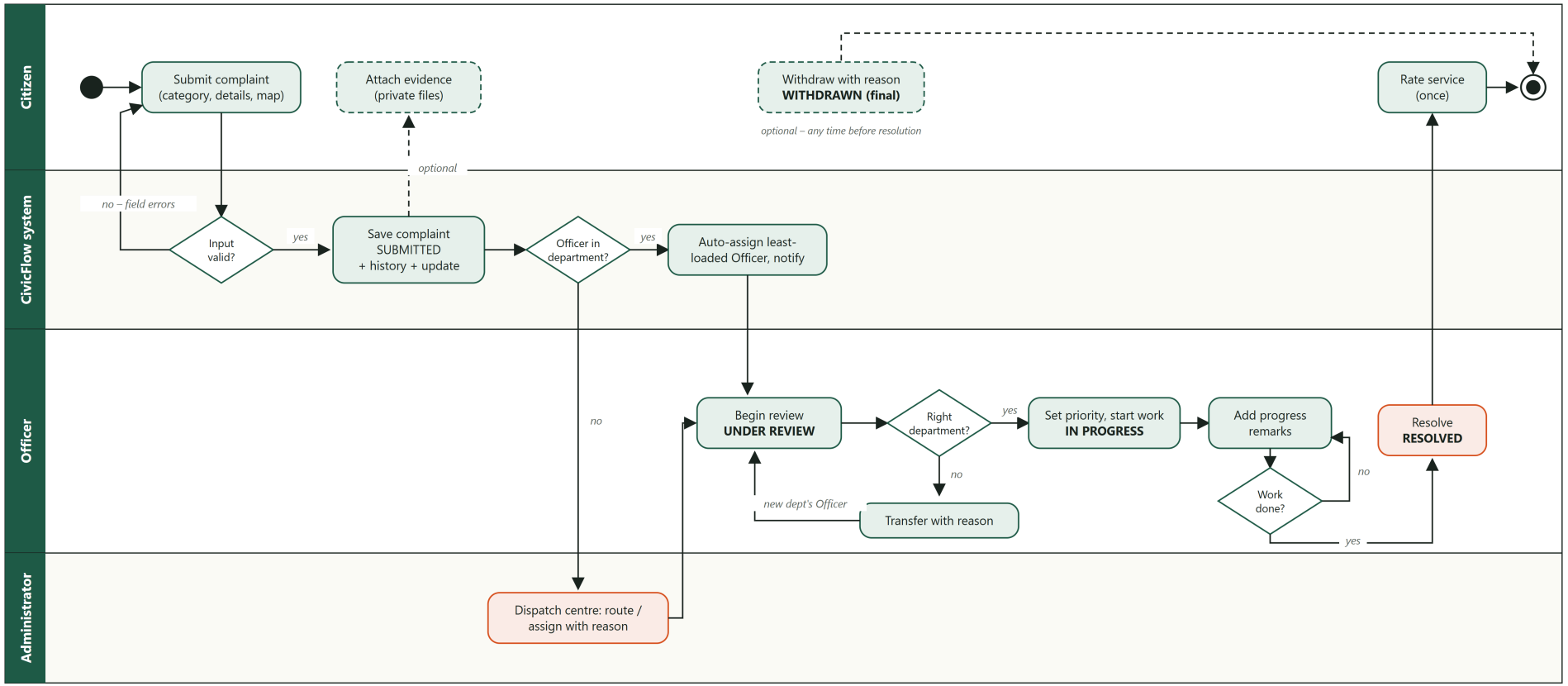


Figure 8.8: UML activity diagram with swimlanes for the complete complaint workflow

8.6 State machine of the complaint status

The status of a complaint follows the state machine below, which is enforced inside the database functions. Assigning an Officer, changing the priority and adding a progress remark leave the status unchanged but increase the version number, which we use for optimistic concurrency control.

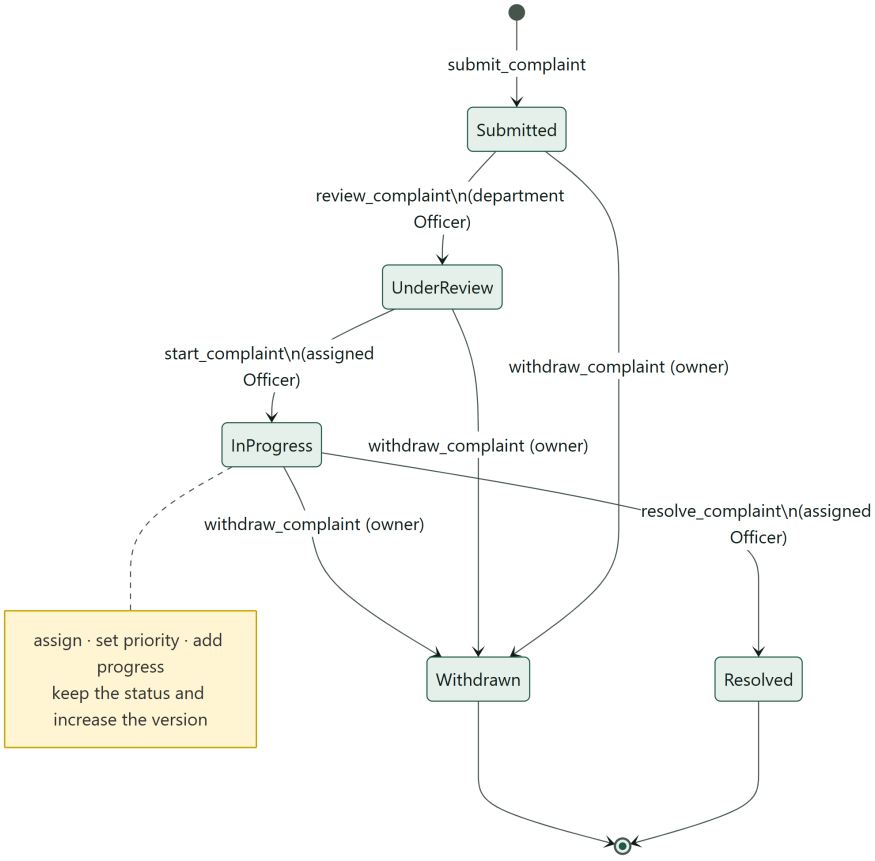


Figure 8.9: UML state machine of the complaint status

8.7 Database design

The database has eight application tables in the public schema and one audit table in a private schema. The entity-relationship diagram below shows their keys and relationships, and the table after it lists the main integrity rules.

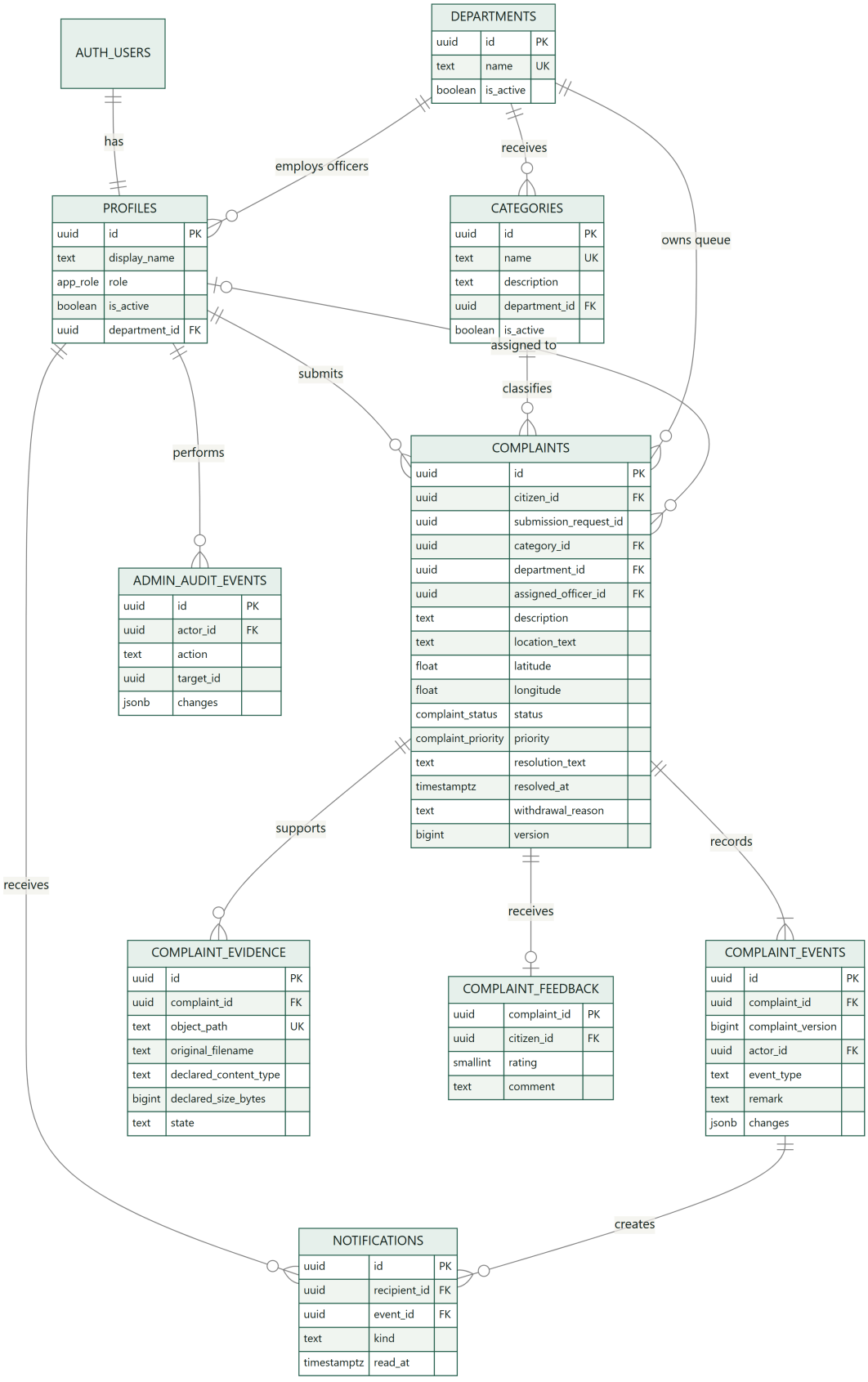


Figure 8.10: Entity-relationship diagram (PostgreSQL 17)

Table 8.1: Major tables and integrity rules

Table	Purpose	Key rules
profiles	One row per Auth user: display name, role, active flag, department	Signup always creates a Citizen; an Officer must have a department and only Officers may have one
departments / categories	Service departments and the categories that route to them	Unique names; categories are deactivated, never deleted; a used category keeps its department
complaints	The complaint and its current state	Description 10–5,000; location 1–300; resolution fields set if and only if Resolved; In progress/Resolved require an assignee; version ≥ 1; (citizen, request id) unique for idempotency
complaint_events	Append-only history	Exactly one event per complaint version (unique complaint_id, complaint_version); progress events need a remark
complaint_evidence	Private file metadata	JPEG/PNG/PDF, 1 byte – 5 MiB, ≤ 5 files; server-reserved unique object path; pending → attached
complaint_feedback	One rating per complaint	Primary key = complaint; rating 1–5; comment ≤ 2,000
notifications	Durable in-system updates	Unique (recipient, event); read_at for the inbox
private.admin_audit_events	Audit of administrative changes	Not exposed to browsers; before/after values in JSON

All foreign keys use ON DELETE RESTRICT, so records are never deleted by the application. Row Level Security is switched on for every table and browsers have no INSERT, UPDATE or DELETE rights. Every change goes through one of twenty narrowly scoped functions.

8.8 User-interface prototype and wireframes

Before building the screens we drew low-fidelity wireframes of the main pages to agree on the layout. The working screens were then built and improved after each sprint review; Chapter 9 shows the final versions.

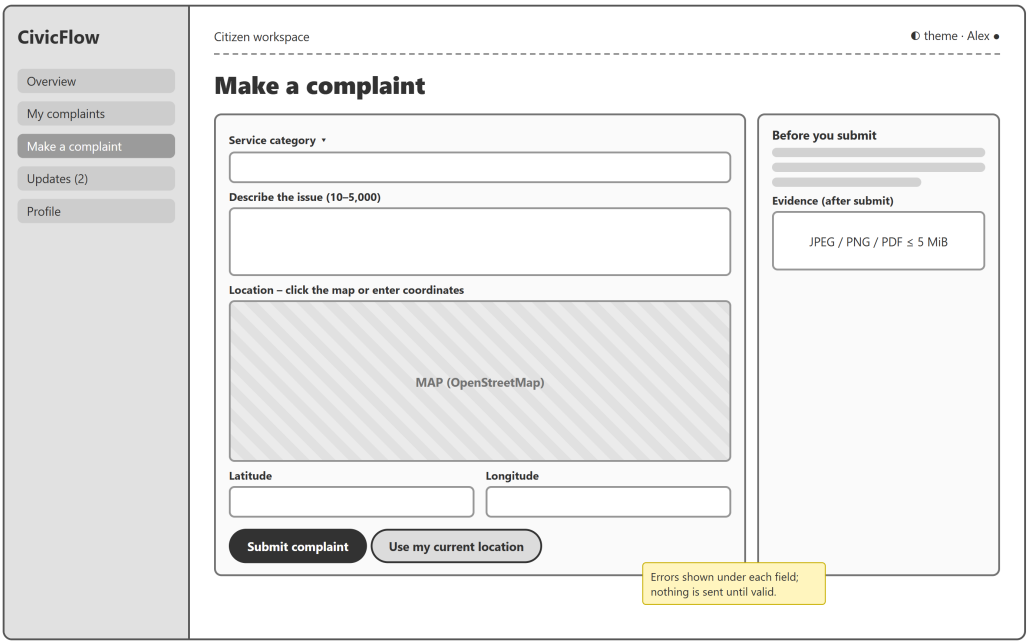


Figure 8.11: Wireframe of the Citizen "Make a complaint" screen

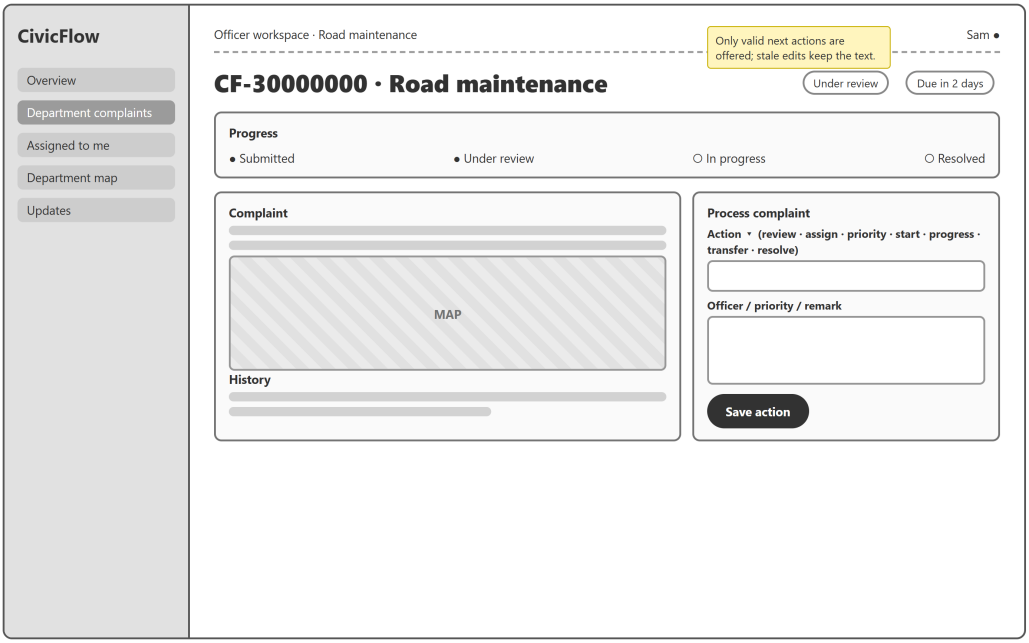


Figure 8.12: Wireframe of the Officer complaint processing screen

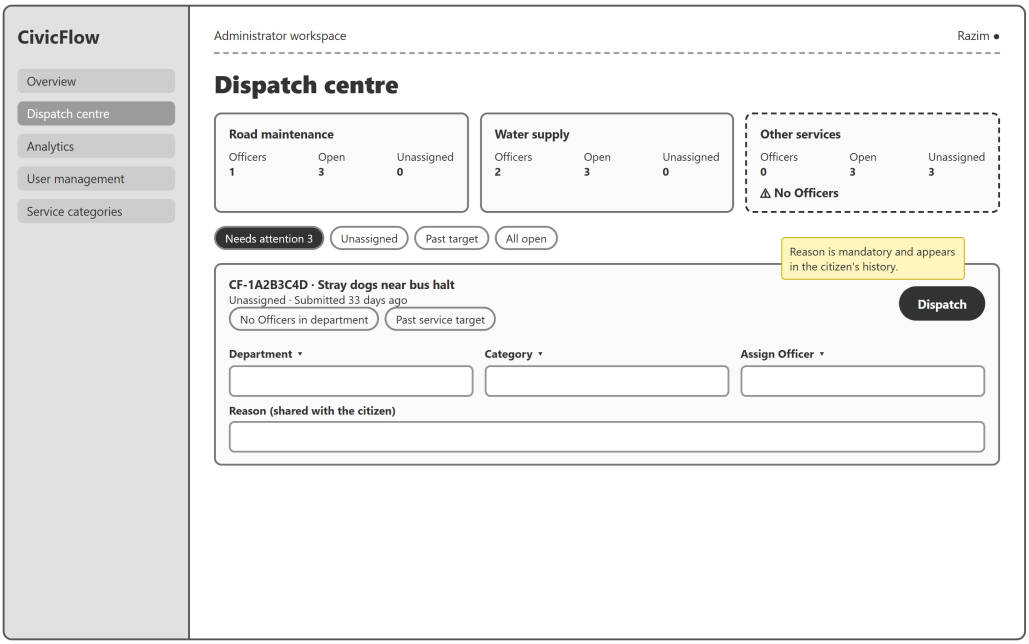


Figure 8.13: Wireframe of the Administrator dispatch centre (CR-006)

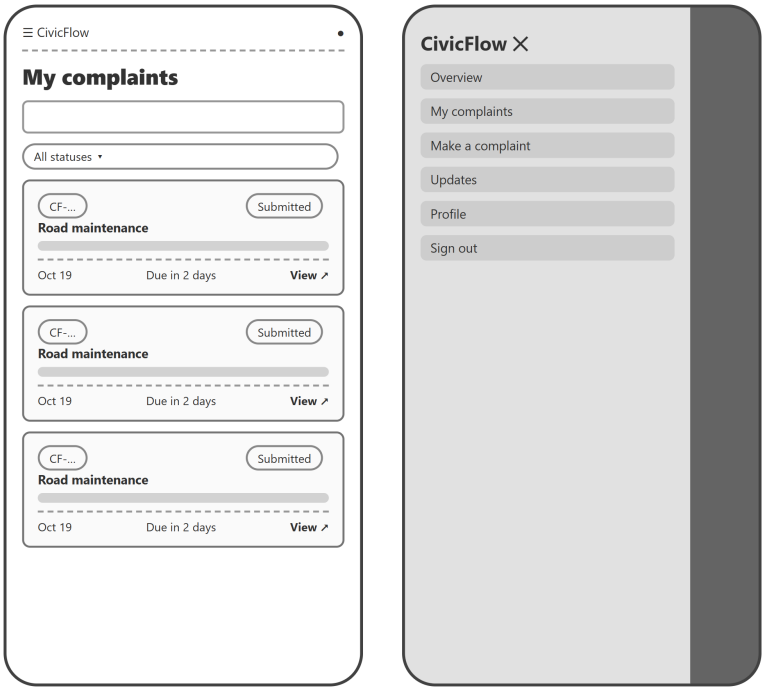


Figure 8.14: Wireframe of the mobile complaint list and slide-out menu

The interface follows a few simple rules. Each screen has one main action. Status names are written in plain language and shown with both text and colour. Every form field has a label, a hint and an error message. There is a skip link and a visible keyboard focus, a light and a dark theme, and animations are turned off when the device asks for reduced motion. Every page works on screens from 360 px phones up to desktops.

8.9 Design decisions and justification

Table 8.2: Important design decisions

Decision	Alternatives considered	Reason
React + TypeScript + Vite SPA	Server-rendered (Next.js), plain JS	Typed domain models catch errors at build time. Vite gives fast builds. The team already knew React. An SPA is enough because there is no SEO requirement.
Supabase (PostgreSQL, Auth, Storage) instead of a custom API server	Node/Express + MySQL; Firebase	Relational integrity (FKs, CHECKs, transactions) suits a workflow with history. RLS puts authorisation next to the data. Managed Auth avoids storing passwords ourselves. The free tier fits the project.
Business writes only through SECURITY DEFINER RPCs; direct table DML revoked	Direct table updates with RLS UPDATE policies	A status change must update the complaint, history and notification atomically with transition rules and locks. Generic UPDATE policies can't express "only valid transitions".
Optimistic concurrency (expected_version, error PT409)	Last-write-wins	Two Officers editing the same complaint must not overwrite each other silently. PT409 maps to HTTP 409 without PostgREST retry loops (defect DEF-01).
Append-only history + durable notifications in PostgreSQL	Realtime push, email/SMS	Meets CF-13/18 within the scope constraint on external notifications. History is the audit trail.

Decision	Alternatives considered	Reason
Memory-only sessions (persistSession: false)	localStorage tokens	Avoids leaving bearer tokens in persistent storage on shared computers. Trade-off: a page refresh requires signing in again (documented).
Private evidence bucket with server-reserved paths	Public bucket, signed URLs	Complaint evidence may contain personal information. Paths can't be chosen by users, and downloads are authorised per request.
Department-scoped Officers; assigned-Officer-only progress/resolution	Any Officer can edit any complaint	Least privilege and clear accountability, as the problem statement asks ("identifying responsible officers").
Categories deactivated, never deleted; referenced department immutable	Hard delete / free edit	Historic complaints must keep their original routing and meaning.
Leaflet + OpenStreetMap with numeric coordinate fallback	Google Maps	No API key or cost. Accessible fallback when tiles fail. The OSM attribution policy is respected.
Vercel static hosting	Netlify, GitHub Pages	Simple Vite deployment, SPA rewrites and security headers configured in vercel.json.
Auto-assignment + Administrator dispatch (CR-006)	Manual pick-up only; giving Administrators full workflow rights	Guarantees an owner whenever staff exist, and gives a narrow, audited escape hatch when they don't, without making Administrators process complaints.
motion library + CSS tokens	Hand-written CSS only; heavy UI kits	Declarative, interruptible animations that respect reduced motion. Design tokens keep light and dark themes consistent.
Vitest + PGlite + Playwright	Jest only, manual testing	Tests each layer: pure rules, real SQL/RLS inside PostgreSQL, and user journeys in a real browser.

9 Implementation

9.1 Technology stack

Table 9.1: Technology stack as built

Layer	Technology (pinned in package.json)
UI	React 19, React Router 7, Tailwind CSS 4, lucide-react icons
Language / build	TypeScript 6 (strict), Vite 8 (Rolldown)
Validation	Zod 4 (client); SQL CHECK constraints and RPC validation (server)
Maps	Leaflet 1.9 + React Leaflet 5, OpenStreetMap tiles
Backend	Supabase: PostgreSQL 17, Auth, Storage, one Deno Edge Function
Tests	Vitest 5 (unit), PGlite (embedded PostgreSQL for database tests), Playwright (browser)
CI / hosting	GitHub Actions; Vercel configuration (vercel.json)

We chose these technologies because they suit the requirements. TypeScript finds many errors before the code runs. PostgreSQL gives us relational integrity and transactions, which a workflow with a full history needs. Row Level Security keeps the access rules next to the data. Because Supabase manages authentication, we never store passwords ourselves, and the free tiers fit the budget constraint. Section 8.9 lists the alternatives we considered.

9.2 Module structure

Table 9.2: Module map

Module	Path	Responsibility	Requirements
App shell	src/app/App.tsx	Routes, role navigation, protected layout, unread badge	CF-02, CF-03
Auth	src/features/auth	Register, login/logout, recovery, invitation password, session/profile loading	CF-01–03
Profiles	src/features/profiles	Own display name	CF-15
Complaints	src/features/complaints	Form, submission, list, detail, withdrawal	CF-04–07, 17, 18, 22
Evidence	src/features/evidence	Private reserve → upload → finalize, download	CF-16
Officer	src/features/officer	Queues, filters, review/assign/priority/start/progress/transfer/resolve	CF-08–12, 17
Feedback	src/features/feedback	Post-resolution rating	CF-19
Notifications	src/features/notifications	Updates inbox, read state, unread badge	CF-13
Categories	src/features/categories	Admin category maintenance	CF-20

Module	Path	Responsibility	Requirements
Admin	src/features/admin	User management, Officer invitation, monitoring overview	CF-14, 21
Shared	src/shared	One Supabase client, environment validation, errors, location, UI primitives	—
Database	supabase/migrations (15 files)	Schema, RLS, RPCs, Storage policies	All
Edge Function	supabase/functions/provision-officer	Admin-only Officer invitation using the Auth Admin API	CF-14

9.3 Core workflows implemented end to end

Each core use case works end to end: from the screen, through the application hook and the data adapter, to a protected database function, with tests at every level. The screenshots below were taken from the running system using fictional data.

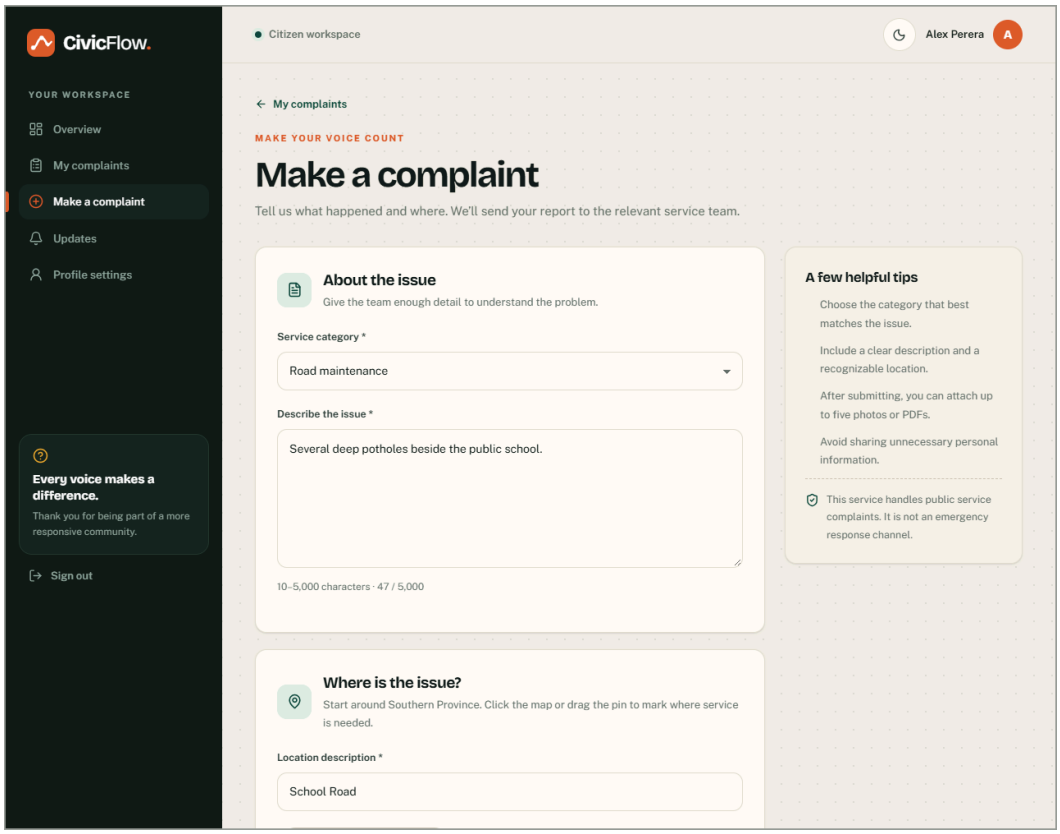


Figure 9.1: Make a complaint screen (Citizen): category, description, OpenStreetMap location and typed coordinates

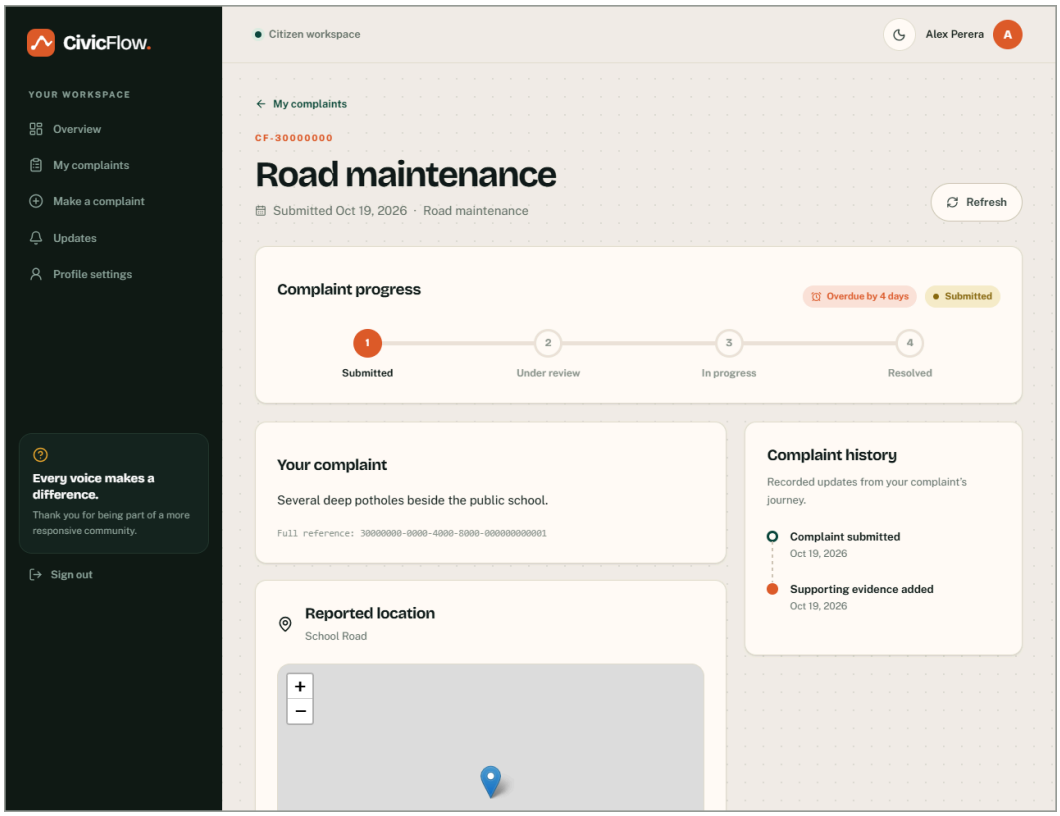


Figure 9.2: Complaint detail (Citizen) with the progress line, location, history and service-target badge

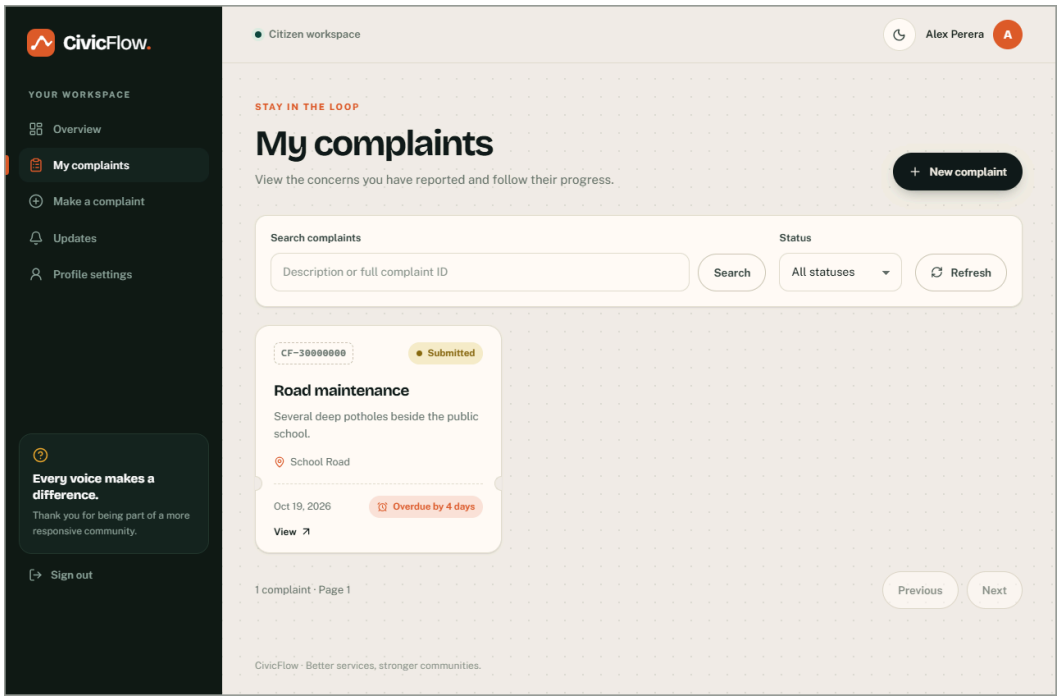


Figure 9.3: My complaints (Citizen): each complaint is shown as a ticket with its status and a due or overdue badge

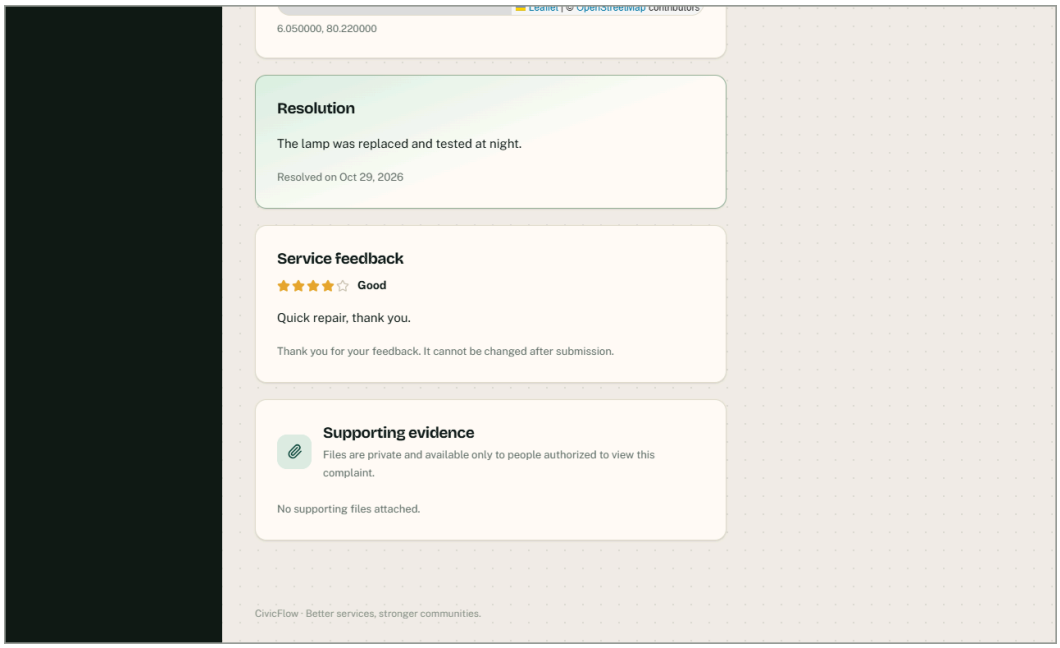


Figure 9.4: Resolution details and the one-time service feedback form (Citizen)

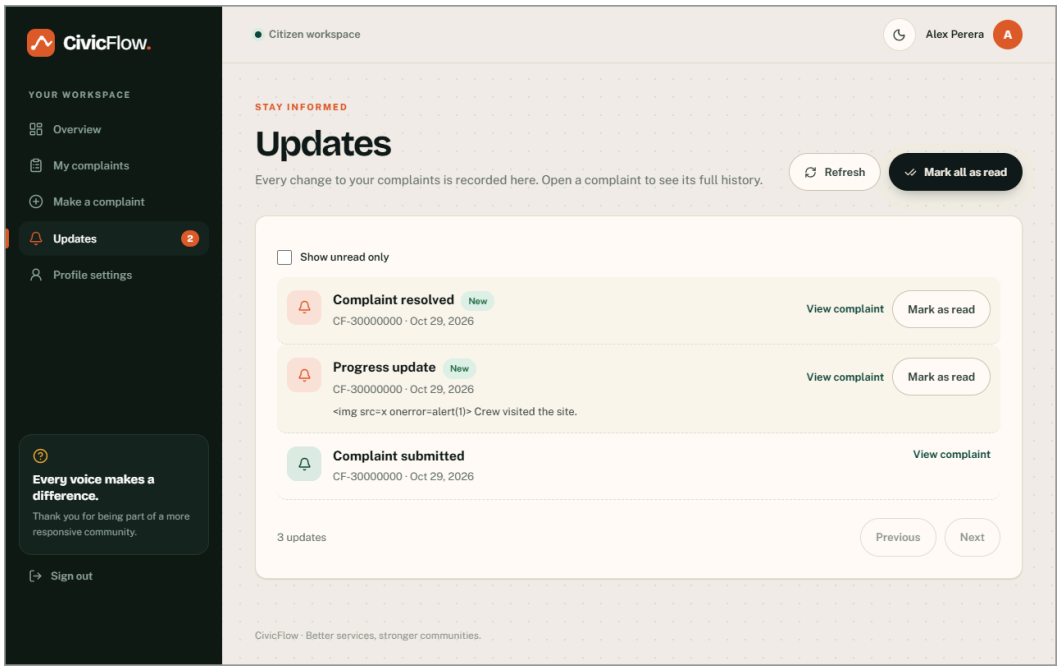


Figure 9.5: Updates inbox (Citizen) with the unread count and mark-as-read

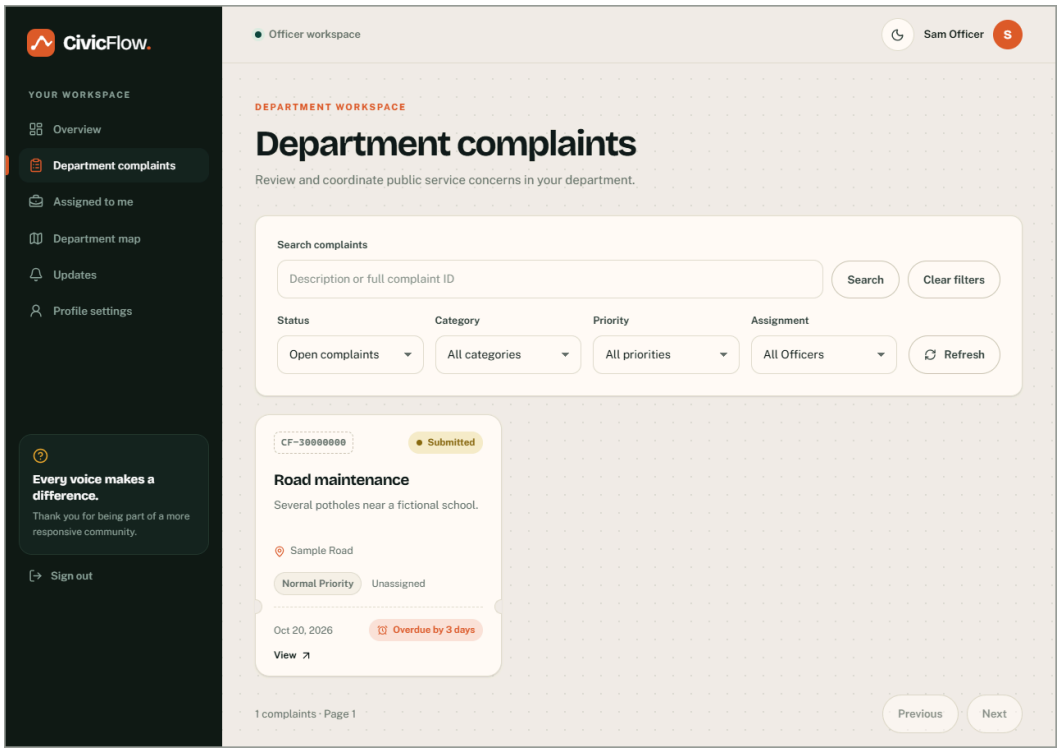


Figure 9.6: Department queue (Officer) with filters for status, category, priority and assignment

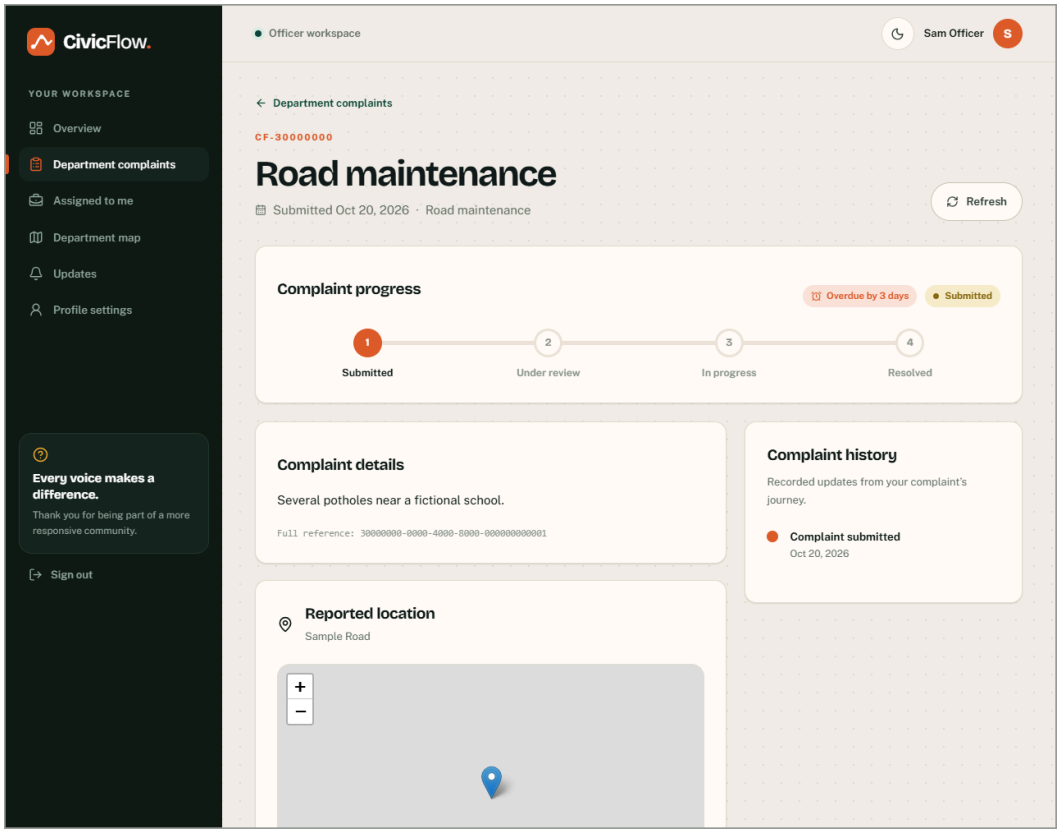


Figure 9.7: Processing a complaint (Officer): only the valid next actions are offered

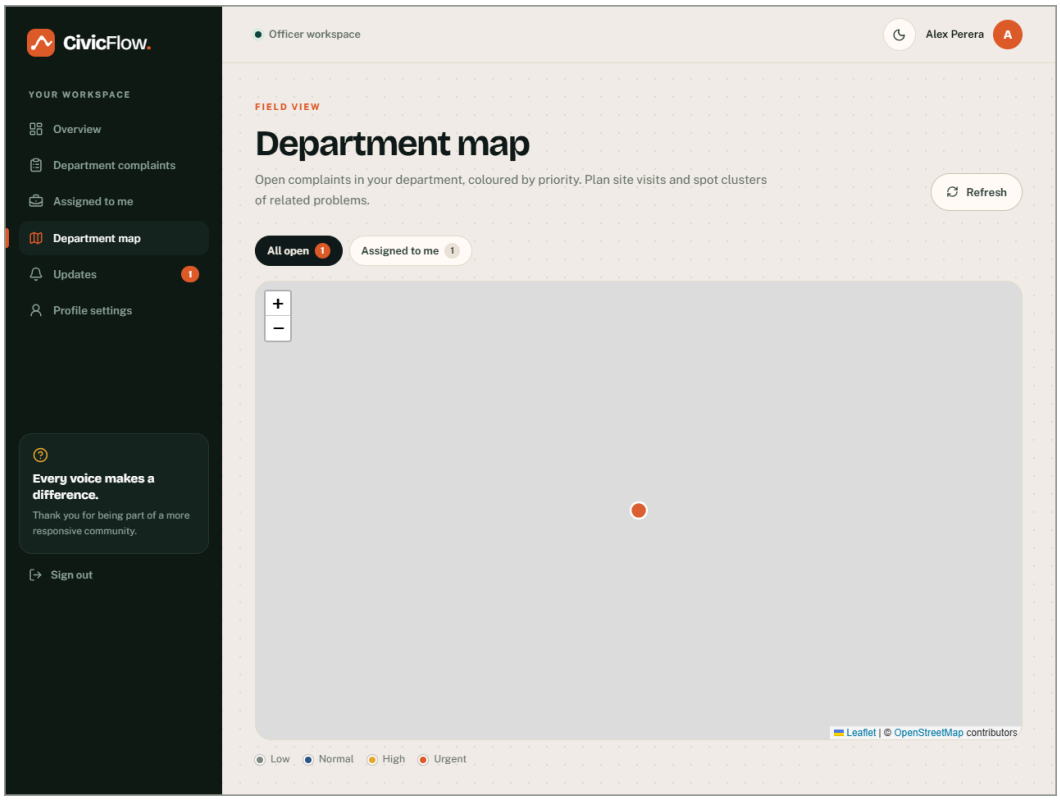


Figure 9.8: Department map of open complaints (Officer)

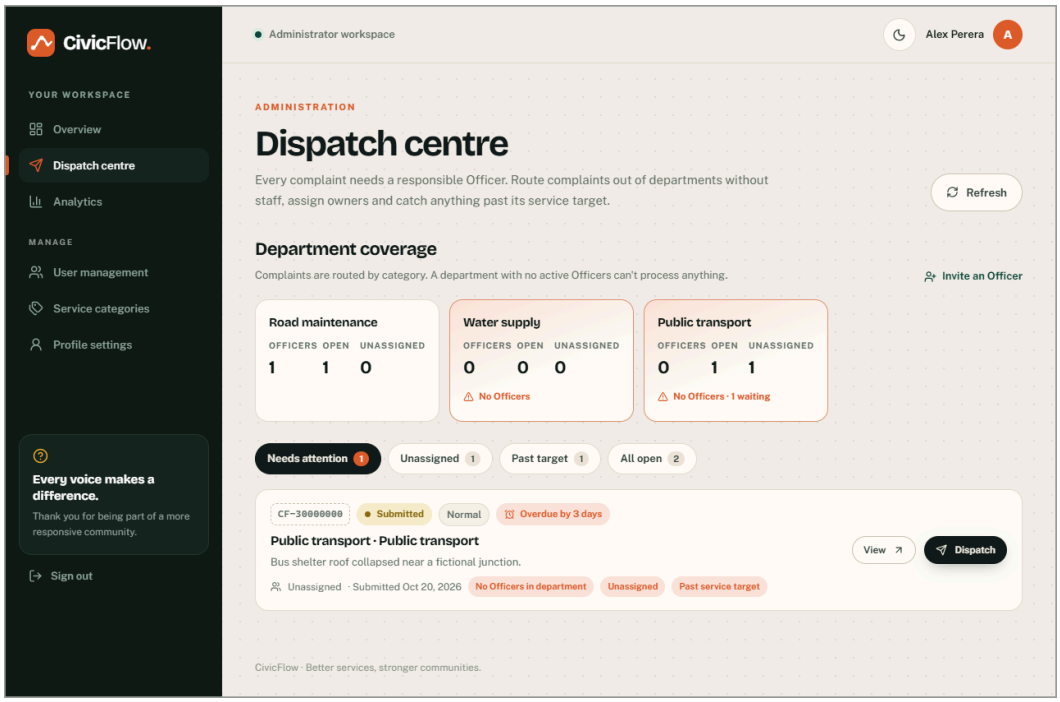


Figure 9.9: Dispatch centre (Administrator) showing department coverage and complaints that need attention

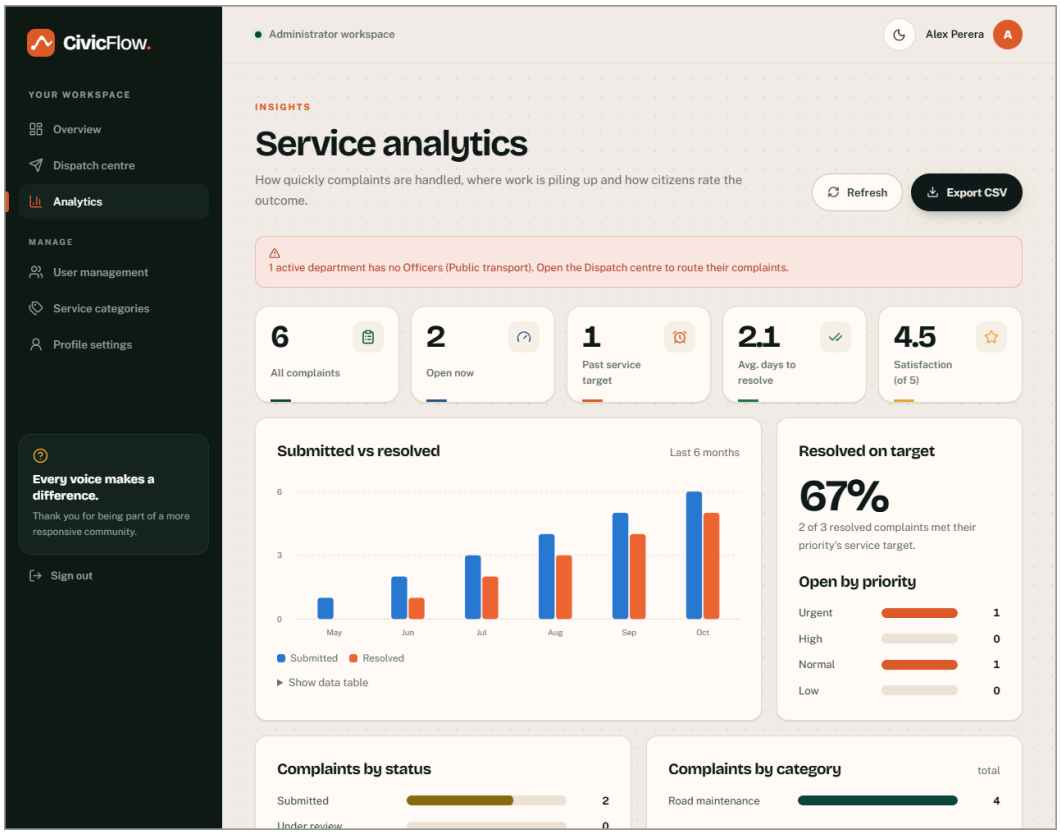


Figure 9.10: Service analytics (Administrator): targets, coverage, workload, satisfaction and the six-month trend, with CSV export

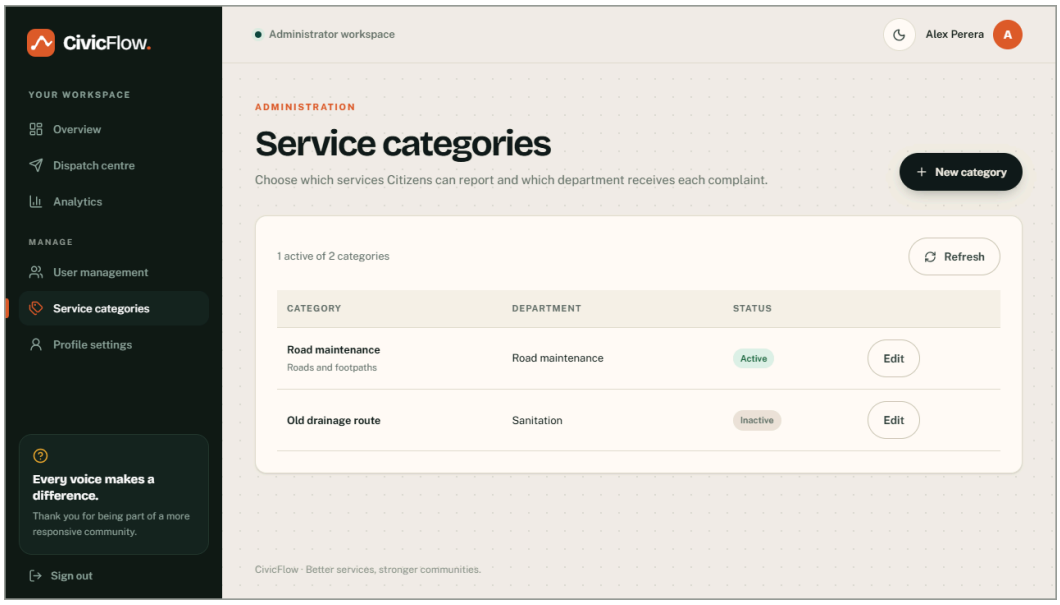


Figure 9.11: Service category management (Administrator)

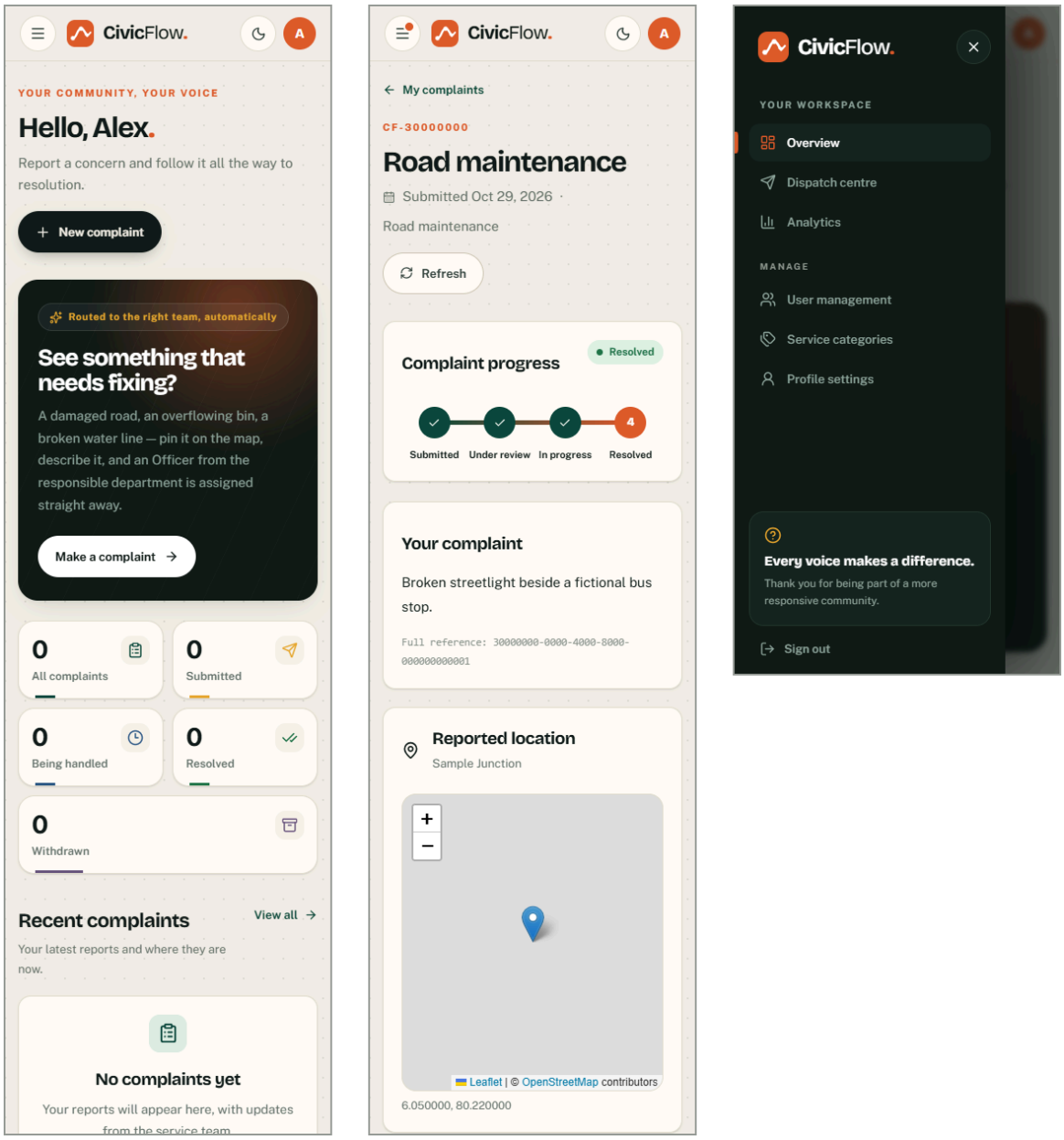


Figure 9.12: Citizen overview, feedback form and slide-out menu on a 360 px wide phone

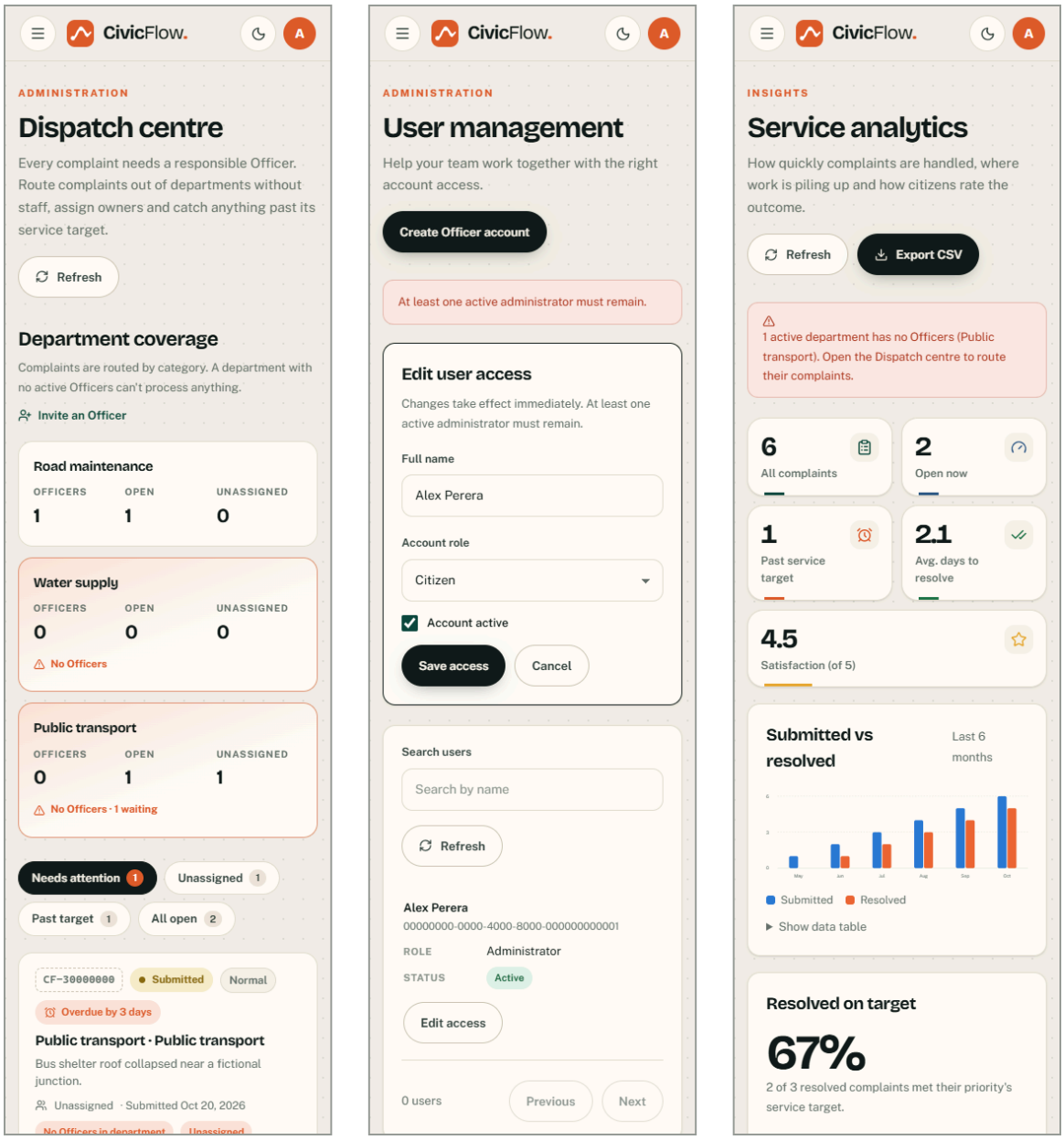


Figure 9.13: Dispatch centre, user management and analytics on a 360 px wide phone

9.4 Input validation

Every input is checked twice. Zod schemas in the browser give the user immediate feedback, and CHECK constraints and function checks in the database make the final decision. Our tests check that both layers use the same limits.

Table 9.3: Input validation in two layers

Field	Client rule (Zod)	Database rule
Complaint description	trimmed, 10–5,000	CHECK + RPC check
Location text	trimmed, 1–300	CHECK + RPC check
Coordinates	finite, lat −90…90, lon −180…180	CHECK + RPC check
Withdrawal reason	trimmed, 10–2,000, confirmation	RPC check + table constraint

Field	Client rule (Zod)	Database rule
Progress remark / resolution	1–2,000 / 10–5,000	RPC checks + CHECK
Evidence	JPEG/PNG/PDF magic bytes, ≤ 5 MiB, ≤ 5 files	Declared type/size CHECK, bucket MIME/size limits, quota under lock, stored metadata verified at finalize
Feedback	integer 1–5, comment ≤ 2,000	RPC check + CHECK
Category	name 1–100, description ≤ 500, department required	RPC check + CHECK + unique name
Password	≥ 12 characters at registration	Supabase Auth minimum length 12

9.5 Error handling

- Database functions return fixed error codes such as not_authorized, invalid_input, stale_version and feedback_exists. Each feature turns these codes into plain messages. Unknown errors show a general message and never reveal server details.
- If someone else changed the complaint first (HTTP 409), the user's input is kept and they are asked to refresh. This request is never retried automatically.
- Creating and withdrawing a complaint use idempotency keys, so the request can be sent again safely if the response is lost.
- When only part of an action succeeds, the user is told exactly what happened, for example "Your complaint is saved, but the file has not uploaded", with a Retry button.
- If a database migration is missing, the user sees an explanation instead of a crash. Authorisation errors are still treated as refusals.

9.6 Coding standards and version control

- Strict TypeScript with no any; database types generated from the migrations and checked for drift in CI.
- ESLint with zero warnings; snake_case in SQL, camelCase/PascalCase in TypeScript; feature folders named after the domain.
- SQL functions are SECURITY DEFINER with an empty search_path, fully qualified names, explicit EXECUTE grants and stable error codes.
- No string-built HTML: all user text is rendered as text by React.
- Git history on GitHub (https://github.com/razim2003/civic-flow) with feature branches per increment (for example sprint-4/feedback-inbox-categories, sprint-5/dispatch-and-experience, ui/responsive-layout) merged to main only when CI is green. Applied migrations are never edited; fixes are new migrations.

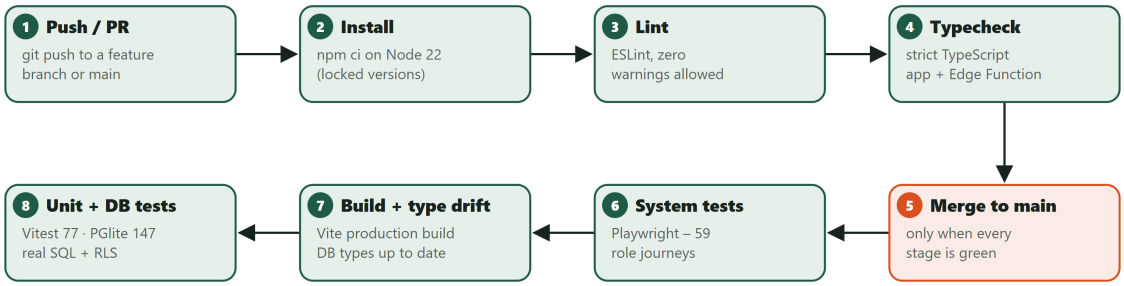


Figure 9.14: Continuous-integration pipeline that runs on every push

10 Testing and Quality Assurance

10.1 Test plan

Testing had three aims: to show that every requirement works as specified, that the backend itself refuses unauthorised users (hiding a button is not enough), and that every defect is fixed and tested again. The plan was based on risk. The biggest risks were data leaking between citizens or departments, skipping steps in the workflow, lost updates and invalid data. We tested these in the database with real SQL run as ordinary users without special privileges, and again through the user interface.

Table 10.1: Test levels

Level	What is tested	Tool	Environment
Unit	Pure validation rules, error translation, service-level targets, analytics maths, session handling, Edge Function handler	Vitest	Node / jsdom
Integration (database)	All 15 migrations, grants, RLS, RPCs, constraints, rollbacks and concurrency, run as anon / authenticated users with fixture identities	Vitest + PGlite (PostgreSQL 17 in-process)	Fresh disposable database per test file
Integration (HTTP)	Real Supabase Auth JWTs + PostgREST + PostgreSQL in containers; concurrent race conditions	Node script + Docker	Disposable local containers
System	Complete role journeys through the real UI in a browser, with every backend response controlled	Playwright (Microsoft Edge / Chromium)	Local dev server
Acceptance	Acceptance criteria of the major user stories, checked by mapped system tests, plus a smoke test on the deployed system	Playwright + manual checklist	Local and deployed URL

Entry criteria: the code compiles and lint passes. Exit criteria: all automated suites pass, no high-severity defect is open, and every Must requirement has at least one passing positive test and one passing negative test. All test data is fictional, for example "Alex Perera", "Sample Road" and @example.test e-mail addresses.

10.2 Execution results

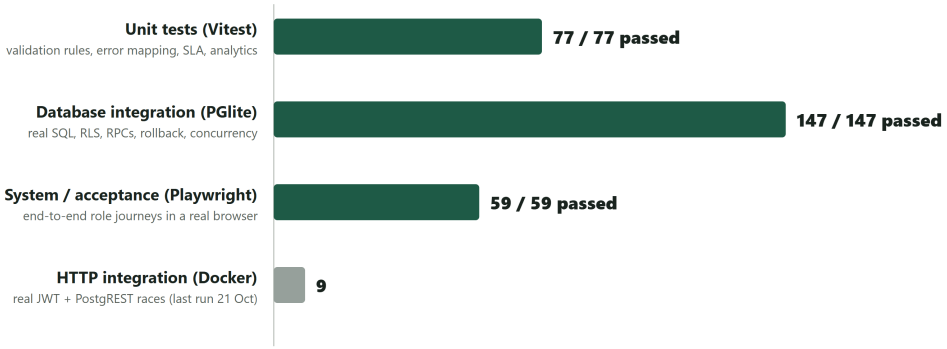


Figure 10.1: Automated test results by level (latest run, 30 October 2026)

Table 10.2: Execution summary

Suite	Files	Tests	Passed	Failed
Lint / typecheck / production build	—	—	✓	—
Unit (Vitest)	13	77	77	0
Database integration (PGlite, 15 migrations + demo data)	9	147	147	0
System (Playwright, Edge)	5	59	59	0
HTTP integration (Docker)	1	9	9 (21 Oct)	0
Total executed in the latest run	27	283	283	0

The documented test cases contain 139 entries: 57 of them include positive checks and 89 include negative checks (misuse or invalid input). We also ran a layout audit that rendered 24 key screens at 360, 414, 768 and 1280 px and checked automatically for sideways scrolling and content spilling out of its box. After the Sprint 5 fixes it found no problems.

10.3 Representative test cases

The table below shows a selection of test cases from all levels. Appendix B has the complete list with preconditions, expected results and actual results.

Table 10.3: Representative test cases

ID	Req	±	Preconditions / input	Expected result	Actual	Status
TC-AUTH-02	CF-01	+/−	Submit an empty form, then valid data	Field errors first; signup request carries only display_name (no role); "Check your email" shown	As expected	Pass
TC-AUTH-04	CF-03	−	Citizen types /admin/users	"Administrator access required"; no admin data requested	As expected	Pass
TC-DB-FND-02	CF-03, NFR-05	+	Inspect catalog	RLS enabled on every application table	As expected	Pass

ID	Req	±	Preconditions / input	Expected result	Actual	Status
TC-SUB-01	CF-04, 06, 16, 18	+	Citizen submits a valid complaint; first upload fails, retry; download; open detail	Reference shown; request has no citizen_id/status; upload retry reuses the same key; private download; history shows "Supporting evidence added"	As expected	Pass
TC-SUB-02	CF-04	−	Blank coordinates; map tiles unavailable	Validation error; map click selects; numeric entry still works	As expected	Pass
TC-TRK-03	CF-07, NFR-11	−	Open another citizen's reference; description containing HTML	"Complaint unavailable"; HTML shown as text	As expected	Pass
TC-DB-EV-03	CF-16	−	Declared size 0 / > 5 MiB; MIME text/html	Rejected	As expected	Pass
TC-OFF-01	CF-09–12	+	Officer reviews, assigns, prioritises, starts, adds progress, resolves with confirmation	Each step "Complaint updated"; expected versions 1→6 sent; complaint closed with resolution text	As expected	Pass
TC-OFF-03	NFR-06	−	Another user changed the complaint (409)	Remark kept; "Refresh and review"; retry after refresh uses the new version	As expected	Pass
TC-DB-OFF-05	CF-11, NFR-06	−	Skip/repeat transitions; stale version; no-ops	invalid_transition; PT409 stale_version; no-op adds no event	As expected	Pass
TC-DB-OFF-08	NFR-06	−	Notification insert forced to fail during review	Status/version and history unchanged	As expected	Pass
TC-FB-02	CF-19	−	Server answers feedback_exists	"Feedback was already submitted…"; form stays; no false success	As expected	Pass
TC-NTF-01	CF-13	+	Two unread, one read update; mark one; filter unread; mark all	Badge "2 unread" → "1 unread" → none; HTML remark shown as text; exact RPC payloads	As expected	Pass
TC-CAT-02	CF-20	−	Server answers category_in_use	Explanation shown; input kept	As expected	Pass
TC-DSP-01	CF-24	+/−	Admin opens the Dispatch centre with a complaint in an unstaffed department; dispatches with no reason, then with a reason	Coverage card shows "No Officers"; row flagged unstaffed, unassigned and past target; reason error with no request; then exactly one RPC with version 1 and a success message	As expected	Pass
TC-UX-03	NFR-03, NFR-04	+	1280 px, then 768 px and 360 px. Open the menu, press Escape, then open each admin page from the menu	Desktop shows the sidebar with no menu button. Tablet/phone hide the sidebar links behind Open menu (aria-expanded). Focus moves to Close menu and returns to the button on Escape. Choosing a page closes the menu. No page is wider than the screen	As expected	Pass
TC-DB-DSP-07	CF-24	−	Short reason, mismatched route, foreign/citizen assignee, stale/null version, withdrawn	reason_required, category_unavailable, invalid_assignee, PT409, invalid_input, complaint_closed	As expected	Pass

10.4 Acceptance testing

Table 10.4: Acceptance test summary

Story	Acceptance criteria	Result
US-04/05/06 Submit complaint	AC1–AC4	Pass (TC-SUB-01…03, TC-DB-SUB-01…04)
US-07 Track complaint	AC1–AC3	Pass

Story	Acceptance criteria	Result
US-11 Update status	AC1–AC4	Pass
US-12 Resolve	AC1–AC4	Pass
US-13 Updates inbox	AC1–AC3	Pass
US-14 Manage users	AC1–AC4	Pass
US-18 Feedback	AC1–AC3	Pass
US-19 Categories	AC1–AC4	Pass
US-20 Withdrawal	AC1–AC2	Pass
Manual UAT on the deployed system	Smoke checklist (Section 11.6)	S1–S3 passed on the live URL (30 Oct 2026); signed-in checks S4–S12 are run by the team before the demonstration

10.5 Defects identified and corrected

We recorded 19 defects. For each one we found the root cause, fixed it and tested it again. The most important were DEF-01, an endless retry loop when two users edited the same complaint (found by the HTTP integration test); DEF-13, complaints in departments without Officers that nobody could handle (found while using the hosted system and fixed through change request CR-006); and DEF-16 to DEF-19, layout problems such as feedback options spilling out of their boxes and the sidebar squeezing the page on tablets (found by the layout audit).

Table 10.5: Defect log

ID	Found	How found	Severity	Description	Root cause	Correction	Re-test
DEF-01	Sprint 3 (2026-10-21)	HTTP integration test (concurrent review)	High	A stale-version conflict kept the request in an endless retry loop	PostgREST 14 automatically retries SQLSTATE 40001	Raise PT409 (HTTP 409). A new corrective migration 20261021000100 replaced the withdrawal function; Officer helpers use PT409	TC-INT-03/04, TC-DB-OFF-05 pass
DEF-02	Sprint 2 (2026-10-19)	Hosted preflight / manual demo	High	Citizen category dropdown was empty	Category migration 006 not yet applied to the hosted DB	Applied the reviewed migration; added an explicit "missing setup" state instead of a silent empty list	TC-SUB-05 pass
DEF-03	Sprint 2	Playwright	Medium	Duplicate React sibling key warning in the history list	Two events rendered with the same key	Keys use stable event IDs	Browser suite pass
DEF-04	Sprint 2	Playwright	Medium	Race between refreshing a complaint and submitting a withdrawal could use an old version	Refresh and submit not serialised	Controls disabled while refreshing; in-flight guard	TC-WD-02 pass
DEF-05	Sprint 1/3	Operator review	Medium	supabase config push reset hosted Auth settings (TOTP, OTP length, email frequency) to defaults	CLI pushes the whole local config	Values restored immediately; local config.toml now records them	Verified by later push (no drift)
DEF-06	Sprint 3	Playwright	Low (test)	Officer return-navigation test clicked before the detail page loaded	Test timing	Test waits for the detail return link	TC-OFF-09 pass
DEF-07	Sprint 3	Playwright (cold start)	Low (test)	First redirect assertion exceeded 5 s on a cold Vite start	Test timeout too short for first compile	Wait for login readiness (15 s bound)	TC-AUTH-01 pass
DEF-08	Sprint 3	Unit run on Windows	Low (tooling)	Unit worker start-up timeouts	Too many jsdom workers	Pure tests run in Node; worker count bounded	Unit suite pass

ID	Found	How found	Severity	Description	Root cause	Correction	Re-test
DEF-09	Sprint 4 (2026-10-29)	Production build warning	Medium	The app shell imported the notifications module both statically and lazily, pulling complaint pages into the first download	The unread-badge hook was in the same module as the inbox page	Separate lightweight notifications/badge entry and zod-free unread API	Build with no warnings; TC-NTF-01 pass
DEF-10	Sprint 4	Database regression suite	n/a (expected)	foundation.test.ts failed: feedback and notifications became readable	The guard intentionally asserted "closed until policies ship"	Guard updated to check authorised reads with writes still denied	119/119 pass
DEF-11	Sprint 4	Local set-up	Low (environment)	vitest could not start: missing Rolldown native binding	Dependencies installed with Node 20 (project requires ≥ 22.18)	Reinstalled with Node 22; requirement documented	All suites pass
DEF-12	Sprint 4	Screenshot review	Low (test)	Inbox fixture showed "Date unavailable"	Malformed fixture timestamp T010:00	Fixture fixed. The app's graceful fallback for bad dates is correct behaviour.	TC-NTF-01 pass
DEF-13	Sprint 5 (2026-10-30)	Hosted use / read-only data check	High	4 of 5 open complaints sat in departments with no Officers and could never be handled; the Administrator could not act	Routing assumed every department is staffed; no assignment at submission; Administrator was read-only	CR-006: auto-assignment, Administrator dispatch, coverage flags (migration 015)	TC-DB-DSP-01…10, TC-DSP-01 pass
DEF-14	Sprint 5	Screenshot review	Medium	Analytics bars below the fold stayed empty until scrolled into view, so printouts and screenshots under-reported data	Bars animated on scroll-into-view	Bars animate on mount; scroll reveals kept only for decorative sections	TC-DSP-02 pass; screenshots re-reviewed
DEF-15	Sprint 5	Screenshot review	Low	Sign-in route illustration clipped its end labels; ticket notches misaligned with the perforation	Coordinates too close to the viewBox edge; notch tied to card bottom	Route moved inward; notches anchored to the perforated line	Visual re-check
DEF-16	Sprint 5 (2026-10-30)	User report + layout audit	Medium	Feedback rating options spilled out of their boxes at every width above 414 px	Rating cards had a 150 px minimum, narrower than the radio, five stars and label combined	One aligned option per row (radio, label, stars); stars drop below the label on very small phones	TC-FB-01, TC-UX-04
DEF-17	Sprint 5	Layout audit	Medium	On tablets the full sidebar stayed open, squeezing pages into ~500 px: two thin detail columns, stat cards cut off, long words breaking	The sidebar only collapsed below 760 px, and then into wrapped link chips	Below 1100 px the sidebar is an accessible slide-out menu opened from a sticky header. Detail pages use one column below 1180 px, and stat cards auto-fit	TC-UX-03, TC-UX-04
DEF-18	Sprint 5	Layout audit	Medium	User management and Service categories scrolled sideways on phones (up to 134 px)	Wide no-wrap tables	Admin tables become stacked, labelled rows on phones	TC-UX-03, TC-UX-04

ID	Found	How found	Severity	Description	Root cause	Correction	Re-test
DEF-19	Sprint 5	Layout audit	Low	Ticket notches extended outside the card; heading links ("Invite an Officer") were squeezed into a thin column on phones	Notches drawn as full circles and clipped; heading rows could not wrap	Half-circle notches drawn inside the card; heading rows wrap and their links do not	TC-UX-04

10.6 Limitations of the test evidence

- The browser tests replace the backend with controlled responses, so they check the interface and the exact requests it sends. Authorisation is checked separately by the database suite, using real SQL as anonymous and signed-in users.
- PGlite uses fixture Auth and Storage schemas; the Storage HTTP service (actual file bytes) was not integration-tested.
- The Docker HTTP suite last ran on 21 October 2026, before the Sprint 4–5 SQL; the same locking pattern is covered by the database suite.
- We did not test with real citizens or Officers; usability was judged by the team.

11 Deployment

CivicFlow is live at https://civic-flow-rho.vercel.app. The frontend is a static build served from Vercel's CDN, and the backend is the Supabase project xapqfnumcpjckhpdyuxz (PostgreSQL 17, Auth, Storage and one Edge Function).

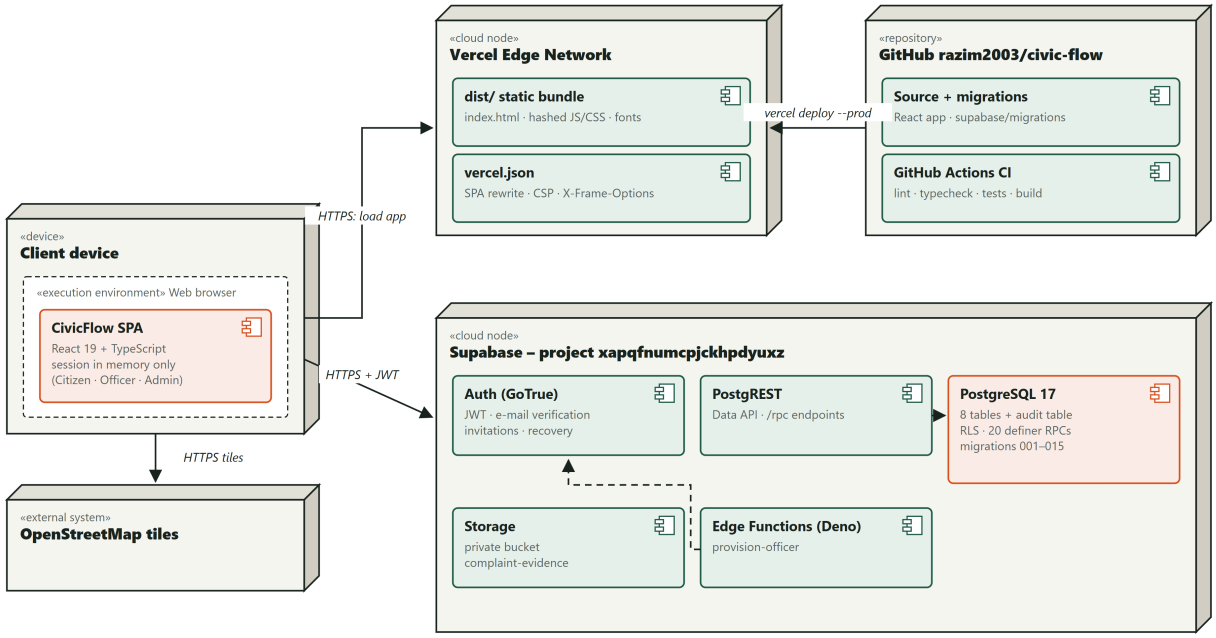


Figure 11.1: UML deployment diagram of the production system

11.1 Deployed architecture and database configuration

- Frontend: a static bundle with hashed file names, served by Vercel. No server code runs on Vercel and the browser holds no secrets. vercel.json adds rewrites for deep links and the security headers Content-Security-Policy, X-Frame-Options DENY, nosniff and Referrer-Policy.
- Backend: Supabase. The database schema comes only from the 15 migrations in the repository. Before applying each migration to the hosted database we reviewed the target, the SQL, the expected effects and the validation and recovery steps, and we compared data checksums before and after.
- The browser needs only two public values, VITE_SUPABASE_URL and VITE_SUPABASE_PUBLISHABLE_KEY. They are safe to publish because access is controlled by Row Level Security.
- The Officer-invitation Edge Function is the only place that uses the privileged key. It runs on the server and uses the CIVICFLOW_APP_URL secret so that invitation links open the live site.

11.2 System requirements

Table 11.1: System requirements

Component	Requirement
End users	A current Chrome, Edge, Firefox or Safari; JavaScript enabled; internet access. Works on phones (≥ 375 px wide).
Developer machine	Node.js ≥ 22.18 with npm 10; Git; optional Docker Desktop (for test:integration) and the Supabase CLI
Hosting	A Vercel account linked to the GitHub repository; a Supabase project

11.3 Configuration

Table 11.2: Configuration settings

Setting	Where	Value / note
VITE_SUPABASE_URL	.env.local (local) / Vercel Environment Variables	https://xapqfnumcpjckhpdyuxz.supabase.co
VITE_SUPABASE_PUBLISHABLE_KEY	same	The project's publishable key (sb_publishable_…). Never use the secret or service-role key.
Auth → URL configuration	Supabase dashboard	Site URL = the deployed URL. Redirect allow-list: <deployed>/auth/callback, <deployed>/reset-password, <deployed>/accept-invite, plus the local 127.0.0.1:5173 equivalents.
Auth → providers	Supabase dashboard	Email with confirmation on; minimum password length 12; anonymous sign-ins off
CIVICFLOW_APP_URL	Supabase Edge Function secrets	The deployed origin, so Officer invitation links point to it
SMTP	Supabase Auth	Custom SMTP is required to invite Officers at arbitrary email addresses. The default sender only delivers to project-team addresses.
Security headers	vercel.json (already in the repo)	CSP, X-Frame-Options: DENY, nosniff, Referrer-Policy

11.4 Installation and deployment steps

1. Local installation: git clone the repository, run npm ci (Node.js 22.18 or later), copy .env.example to .env.local and set the two public values, then npm run dev and open http://127.0.0.1:5173.
2. Verification: npm run lint, npm run typecheck, npm run test, npm run test:db, npm run build and npx playwright test.
3. Database: review each new migration, take a read-only baseline, apply it to the Supabase project only after approval, then verify grants, RLS, anonymous refusal and unchanged checksums.
4. Frontend: create the Vercel project, add the two public environment variables for Production and Preview, and run vercel deploy --prod (or connect the GitHub repository for automatic deployments).
5. Post-deployment: set the Supabase Auth Site URL and redirect URLs (/auth/callback, /reset-password, /accept-invite) and the CIVICFLOW_APP_URL secret to the production URL, then run the smoke test.

11.5 Deployment verification (smoke test)

Table 11.3: Smoke-test and acceptance checklist on the deployed URL

#	Check	Expected	Result
S1	Open / and /workspace signed out	Redirect to sign-in	Pass 2026-10-30: / shows the landing page; /workspace redirects to /login
S2	Deep link /complaints/new reloads	SPA loads (no 404) and asks to sign in	Pass 2026-10-30: HTTP 200, then redirected to sign-in; no console errors
S3	Response headers	CSP, X-Frame-Options: DENY, nosniff present	Pass 2026-10-30: CSP, X-Frame-Options: DENY, nosniff and Referrer-Policy present
S4	Register a demo Citizen; confirm email; sign in	Citizen workspace	Run by the team with the fictional demo accounts before the demonstration
S5	Submit a complaint with a map location and a small PNG	Reference shown; file downloadable	Run by the team with the fictional demo accounts before the demonstration
S6	Officer (demo) reviews, assigns, starts, adds progress, resolves	Each step in history	Run by the team with the fictional demo accounts before the demonstration
S7	Citizen sees the Updates badge, opens the inbox, marks it read	Badge clears	Run by the team with the fictional demo accounts before the demonstration
S8	Citizen gives feedback (rating 4)	Read-only feedback shown; Officer can see it	Run by the team with the fictional demo accounts before the demonstration
S9	Admin creates a category; Citizen form shows it	Category appears	Run by the team with the fictional demo accounts before the demonstration
S10	Admin overview counts	Match the demo data	Run by the team with the fictional demo accounts before the demonstration
S11	Citizen opens another user's complaint URL	"Complaint unavailable"	Run by the team with the fictional demo accounts before the demonstration
S12	Page load timing (DevTools, broadband)	NFR-02 targets met	Run by the team with the fictional demo accounts before the demonstration

For the demonstration we loaded a fictional Sri Lankan data set into the hosted database: ten citizens, Officers for the departments that had none, and 36 complaints from places such as Nugegoda, Kandy, Jaffna, Galle and Trincomalee. The data was created through the normal workflow functions, so every screen shows realistic content. The demo accounts cannot sign in, and a tested script removes the data.

12 Maintenance and Change Request

Once the first version was working, we treated every change as a maintenance task and followed the same steps each time: record the request, analyse its impact, classify and estimate it, get approval, implement it on a branch with a new migration, run the full regression suite, update every affected document, and apply it to the hosted system only after explicit approval.

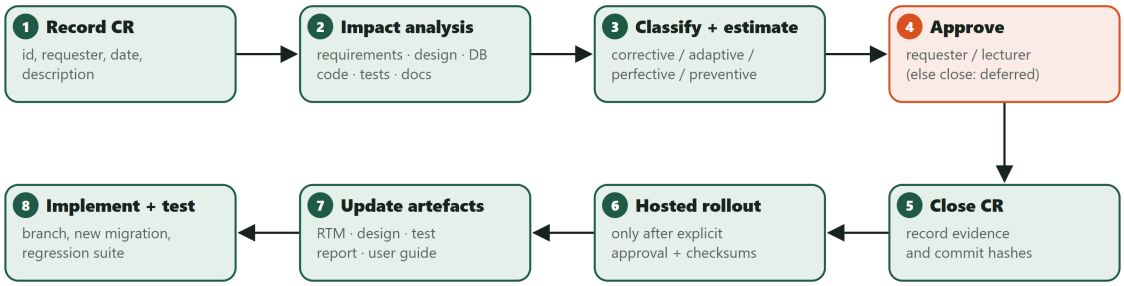


Figure 12.1: Maintenance process used for every change request

12.1 Change request register

Table 12.1: Change request register

CR	Title	Requested by / date	Type	Status
CR-001	Citizen can withdraw an unresolved complaint (CF-22)	Project owner, 2026-10-19	Perfective	Implemented, deployed to DB, tested
CR-002	Stale-version conflicts must not loop (PT409)	Found by testing, 2026-10-21	Corrective	Implemented, applied, tested
CR-003	Administrator-created Officer accounts (CF-14)	Project owner, 2026-10-20	Perfective (+ preventive security)	Implemented, applied, tested
CR-004	"Use my current location" and closer default map (CF-04)	Project owner, 2026-10-21	Perfective (usability)	Implemented, tested
CR-005	Lecturer-provided change request	Lecturer, date	to classify	Awaiting request
CR-006	Every complaint must reach a responsible Officer (auto-assignment, dispatch, service targets) and product completion/redesign	Project owner after hosted testing, 2026-10-30	Corrective (coverage defect) + perfective	Implemented, applied to the hosted database, tested and deployed

Lecturer change request

The lecturer-provided change request (CR-005) had not been received when this report was prepared. When it arrives it will be handled with the same process and template: description, affected requirements, impact analysis across requirements, design, database, code, tests and documentation, an estimate, classification and the implementation and test evidence.

12.2 Main change request CR-006: every complaint must reach a responsible Officer

Table 12.2: CR-006 change request

Field	Entry
Requested by / date	Project owner after hosted testing, 30 October 2026
Description of the change	Complaints were routed to a department by category and only that department's Officers could act. Departments without Officers silently collected complaints: the hosted data showed 4 of 5 open complaints stuck this way, and the Administrator could see them but had no permission to act. Every complaint must reach a responsible Officer.
Classification	Corrective for the coverage defect (the system failed its own objective O3, "enable officers to manage … complaints"); perfective for the service targets, analytics and dispatch tools added with it
Affected requirements	CF-09, CF-13, CF-21; new CF-23 (automatic assignment), CF-24 (Administrator dispatch), CF-25 (service-level targets); NFR-03
Affected stories and use cases	New US-22, US-23, US-24; new UC-15 Dispatch complaint and UC-16 Auto-assign Officer; UC-03 changed

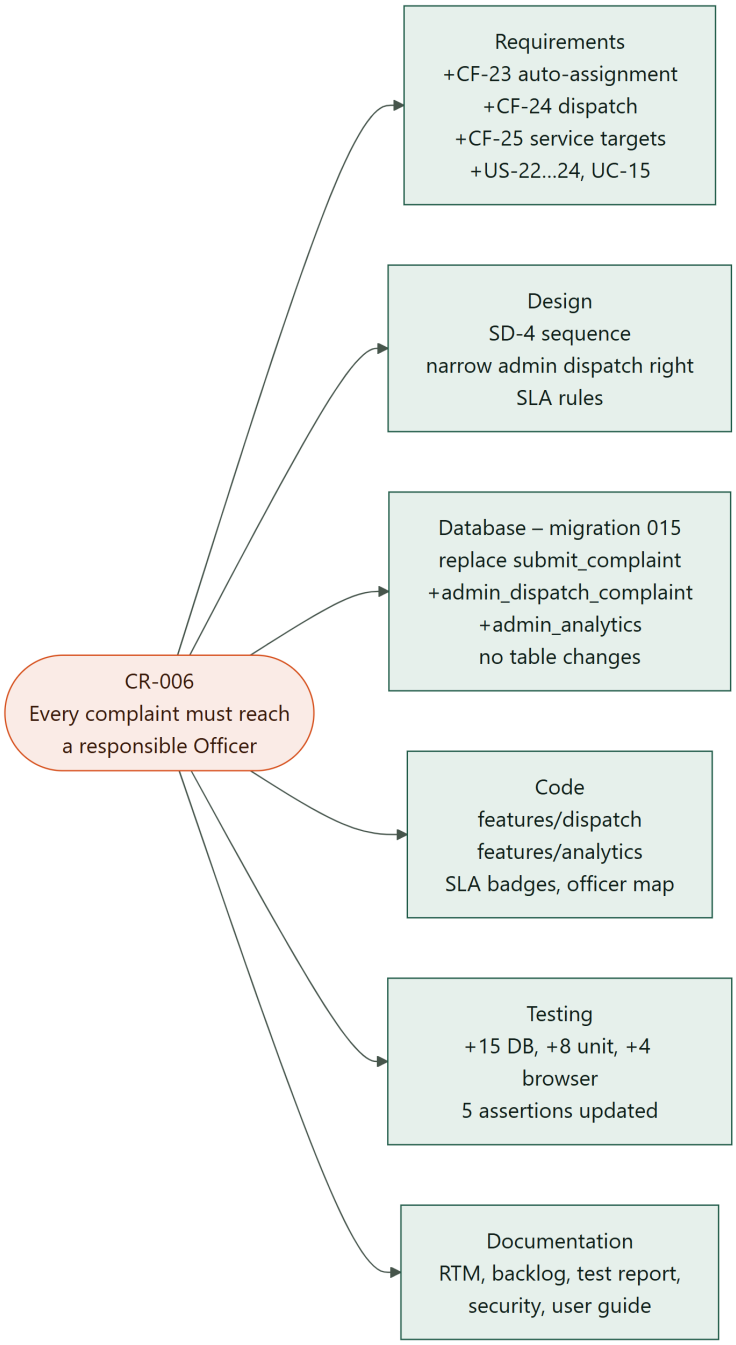


Figure 12.2: CR-006 impact across SDLC artefacts

Table 12.3: CR-006 impact analysis and estimate

Area	Affected items	Change made	Effort (pts)
Requirements	Backlog, FR list, user stories, use cases, RTM	3 new functional requirements, 3 user stories, UC-15/16; RTM rows added	2
Design	Architecture, sequence diagrams, UI	SD-3 sequence; Administrator gets a narrow, audited dispatch right (no processing rights); SLA rules; dispatch centre and analytics screens	3

Area	Affected items	Change made	Effort (pts)
Database	submit_complaint, record_officer_event	Migration 015 (new, never editing applied ones): workload-balanced assignment, admin_dispatch_complaint, admin_analytics, community_impact; no table changes and no data changed at install	6
Code	complaints, officer, admin	New dispatch and analytics features, SLA badges, Officer map	6
Testing	Database, unit, browser suites	+15 database, +8 unit, +4 browser tests; 5 assertions updated because the assigned Officer is now also notified	3
Documentation & security	Test report, security, user guide	Threat T11 and requirement SR-12 added; user guide and deployment record updated	1
Total estimate		About one sprint for a pair of developers	21

12.2.1 Modification plan and verification

1. Create branch sprint-5/dispatch-and-experience.
2. Write migration 015 and its 15 database tests first (test-first for the security-sensitive SQL).
3. Implement the dispatch and analytics features, SLA badges and Officer notifications.
4. Run the full regression suite: 77 unit, 138 database and 58 browser tests passed at that point, with CI green.
5. Apply migration 015 to the hosted database after explicit approval. All existing data checksums were identical before and after, anonymous calls to the new functions were refused (HTTP 401), and the live analytics confirmed the three unstaffed departments.
6. The four stuck complaints were not moved automatically: the Administrator dispatches them with a reason, so each citizen sees why their complaint moved.

12.2.2 Other change requests handled

- CR-001 Citizen withdrawal (perfective): a citizen can withdraw an unresolved complaint with a recorded reason; records, evidence and history are kept.
- CR-002 Non-retrying conflict code (corrective): the stale-version error was changed from SQLSTATE 40001 to PT409 (HTTP 409) after testing showed PostgREST retried 40001 indefinitely.
- CR-003 Administrator-created Officer accounts (perfective and preventive): Officers can no longer self-register; an audited invitation flow was added.
- CR-004 Location usability (perfective): an optional "Use my current location" button and a closer default map view.
- Responsive-layout maintenance (corrective, DEF-16–19): a slide-out menu for tablets and phones, an aligned feedback form and stacked admin tables.

13 Security

CivicFlow stores personal data (names, e-mail addresses, complaint locations and descriptions, and evidence files) and has three privilege levels, so we planned security from Sprint 1 instead of adding it at the end.

Table 13.1: Assets and their sensitivity

Asset	Sensitivity
Citizen identity (name, email) and complaint location/description	Personal data: high
Evidence files (photos, PDFs)	Personal data, may show people or homes: high
Workflow integrity (status, assignment, resolution, history)	Accountability: high
Administrator/Officer privileges	Privilege: critical
Service configuration (categories, departments)	Medium

13.1 Threats and risk assessment

11 threats were identified using the STRIDE categories. Likelihood (L) and impact (I) are rated from 1 (low) to 3 (high); risk = L × I. The residual risk is the rating after the implemented controls.

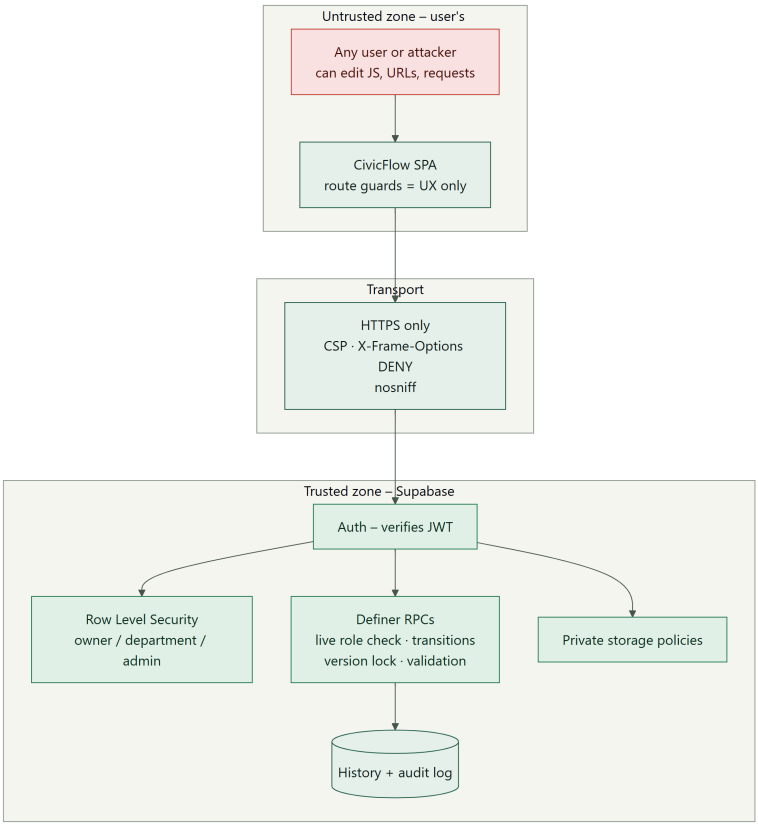


Figure 13.1: Trust boundary: every decision is made on the server side

Table 13.2: Threats and risk assessment

ID	Threat (STRIDE)	Scenario	L	I	Risk	Controls	Residual
T1	Elevation of privilege	A Citizen sends role: administrator in signup metadata or runs UPDATE profiles SET role=… from the browser console	3	3	9	Signup trigger hard-codes Citizen; no browser UPDATE grant; role changes only through the audited admin RPC, which checks the live Administrator	1
T2	Information disclosure (IDOR)	A Citizen changes the complaint ID in the URL or REST query to read another person's complaint, history, evidence or feedback	3	3	9	RLS on every table via can_read_complaint (owner / same-department Officer / Admin); child tables inherit; Storage policies check the parent	1
T3	Tampering (workflow bypass)	An Officer or Citizen calls PostgREST directly to set status='resolved', skip steps or edit another department's complaint	2	3	6	Direct DML revoked; narrow RPCs with transition rules, department/assignee checks, row locks and expected version	1
T4	Spoofing / stale privilege	A deactivated or demoted user keeps using an unexpired JWT	2	3	6	Every policy and RPC re-reads the live profile (is_active, role, department); no authorisation from JWT claims	1
T5	Tampering / XSS	A malicious description, remark, filename or comment contains <script> or onerror to steal sessions	2	3	6	React text rendering only; no dangerouslySetInnerHTML; no string-built map popups; CSP script-src 'self'; sessions kept in memory	1
T6	Malicious upload / storage abuse	Upload of HTML/SVG/executables, huge files, overwriting someone else's file, or uploading to another path	2	2	4	Private bucket; MIME allow-list (JPEG/PNG/PDF) and 5 MiB limit in the bucket and RPC; server-reserved UUID paths; no UPDATE/DELETE policy; 5-file quota; stored metadata verified at finalize	2 (content isn't malware-scanned)
T7	Weak authentication	Short or reused passwords; account enumeration via recovery	2	2	4	12-character minimum; email verification; generic recovery response; Supabase Auth rate limits	2
T8	Secret leakage	A service-role key or DB password bundled into the frontend or committed to Git	1	3	3	Only VITE_ public URL and publishable key; the env validator rejects sb_secret_ keys; .env* git-ignored; the service key only inside the Edge Function runtime	1
T9	Denial of service / spam	Mass complaint submission or repeated requests	2	2	4	Auth required for all writes; bounded inputs and pages; idempotent submission; platform rate limits	3 (no app-level rate limit)
T11	Tampering (CSV injection) / disclosure via analytics	A category name like =HYPERLINK(...) runs when an exported CSV is opened; non-admins call the analytics RPC	1	2	2	CSV cells are quoted and formula prefixes neutralised; admin_analytics checks a live active Administrator; community_impact returns aggregates only	1
T10	Repudiation	An Officer or Admin denies making a change	2	2	4	Append-only complaint_events with actor and time; private admin_audit_events with before/after values	1

13.2 Security requirements

Table 13.3: Security requirements

ID	Requirement	Threats	Verified by
SR-01	Every protected read and write requires an authenticated and active profile.	T4	TC-DB-FND-03, TC-DB-ADM-04, TC-DB-NTF-04
SR-02	Roles can only be changed by an active Administrator through an audited operation. Signup always creates a Citizen.	T1	TC-DB-FND-01, TC-DB-FND-04, TC-DB-ADM-01, TC-INT-02
SR-03	Citizens can read only their own complaints and children. Officers can read only their department's.	T2	TC-DB-SUB-06, TC-TRK-03, TC-DB-FB-05, TC-INT-01
SR-04	Browsers have no direct INSERT/UPDATE/DELETE on application tables. Business changes go only through validated RPCs.	T3	TC-DB-OFF-03, TC-DB-FB-05, TC-DB-CAT-06
SR-05	Workflow changes enforce valid transitions, assignee rules and optimistic locking.	T3	TC-DB-OFF-02, TC-DB-OFF-05, TC-INT-04
SR-06	User-supplied text is never interpreted as HTML.	T5	TC-TRK-03, TC-NTF-01
SR-07	Evidence is private, type- and size-limited, stored at server-chosen paths and never overwritten.	T6	TC-DB-EV-01…06
SR-08	Passwords are at least 12 characters, and email ownership is verified.	T7	TC-U-AUTH-01, Auth configuration
SR-09	No secret credential is present in source, the build or browser configuration.	T8	TC-U-AUTH-01 ("rejects a secret key …"), .gitignore
SR-10	All administrative and workflow changes are attributable.	T10	TC-DB-ADM-02, TC-DB-CAT-01, TC-DB-OFF-01
SR-11	The last active Administrator cannot be removed, even concurrently.	T1	TC-DB-ADM-03, TC-INT-08
SR-12	Dispatch is limited to active Administrators, is version-checked, audited and notified; analytics are Administrator-only; aggregates never identify citizens.	T1, T3, T11	TC-DB-DSP-04…12, TC-U-ANL-01

13.3 Implemented security controls

The guidelines ask for at least three implemented controls. We implemented 8, and each one can be demonstrated.

Table 13.4: Implemented controls and how to demonstrate them

#	Control	Where	How to demonstrate
C1	Row Level Security (role, ownership and department authorisation)	supabase/migrations/…0200, …0400, …200100 (can_read_complaint, read policies)	Sign in as Citizen B and open Citizen A's complaint URL: "Complaint unavailable". Run npm run test:db and show "isolates Citizens and departments…" passing.

#	Control	Where	How to demonstrate
C2	Protected workflow RPCs (no direct DML; transitions, assignee, version)	… 20261020000100_officer_processing .sql, …0200_officer_resolution.sql	Send PATCH /rest/v1/complaints?id=eq.<id> with a signed-in Officer's token (DevTools → Network → copy as cURL, change it to PATCH {"status":"resolved"}): 403 permission denied (also asserted by the HTTP integration test). A stale tab shows "This complaint changed while you were viewing it".
C3	Private evidence storage	… 0500_private_complaint_evidence.s ql	A public object URL (/storage/v1/object/public/complaint-evidence/…) is refused because the bucket isn't public. The download works only when signed in as an authorised user.
C4	Fail-closed live authorisation	private.active_role, checks in every RPC	Deactivate an Officer in User management: their next action is refused without signing them out.
C5	Input validation in two layers	Zod model/*.ts + SQL CHECK/RPC	Submit a 5-character description: client error. The same request sent directly gets invalid_input.
C6	XSS defences	React text rendering, CSP in vercel.json	A complaint containing <img src=x onerror=alert(1)> displays as text (TC-TRK-03, TC-NTF-01).
C7	Secure session handling	shared/lib/supabase/client.ts (persistSession:false), AuthProvider	Reloading the page signs you out; no token is stored in localStorage (TC-AUTH-06).
C8	Audit trails	complaint_events, private.admin_audit_events	Every step appears in the complaint history. Category changes write audit rows (TC-DB-CAT-01).

13.4 Security decisions

- Authorisation happens in the database. The user controls the browser, so a check that exists only in React can be bypassed. All checks are in PostgreSQL and the interface only reflects them.
- SECURITY DEFINER functions bypass Row Level Security, so we hardened them: an empty search_path, fully qualified names, no dynamic SQL, EXECUTE granted only to signed-in users, and an explicit check of the caller's current role.
- The privileged service key is used in exactly one place, the invitation Edge Function, and only after it has verified that the caller is a live Administrator.
- Sessions are kept in memory only. Users must sign in again after refreshing the page, but their tokens are less exposed on shared computers.
- Accepted residual risks for the prototype: no malware scanning of uploads, no application-level rate limiting beyond the platform's, and the default Supabase e-mail sender. Enabling Supabase leaked-password protection is recommended before public use.

14 Project Management and Individual Contributions

14.1 Team coordination

- Roles: each member owned one module end to end (Section 6.5) and reviewed another member's work at the end of each sprint.
- Ceremonies: sprint planning at the start of each sprint, short progress check-ins, and a sprint review and retrospective whose outcomes were recorded in the sprint documents (docs/sprint-*.md).
- Integration: modules integrate only through shared database contracts and small public feature interfaces, so members could work in parallel without breaking each other's code; CI protected main.
- Change control: database migrations and hosted rollouts required explicit approval; scope changes were recorded as change requests.

14.2 Individual contributions

As Section 13 of the guidelines requires, every member worked on requirements, design, implementation and testing, and not only on documentation or the presentation.

Table 14.1: Contribution matrix by SDLC activity

Member	Requirements	Design	Implementation	Testing
TG/2023/1761 M.S.M. Sharfan	Admin, data and coverage requirements (CF-14, 18, 20, 21, 23, 24); NFR-05/06	ER and database design, access matrix, RLS policies, architecture, SD-3	Migrations and protected functions; user and category management; admin dashboard and analytics; dispatch centre (CR-006); hosted database operations	Database integration suites (foundation, dispatch, sprint 4, demo data)
TG/2023/1731 H.M. Amadi Nethma	Stakeholder analysis; citizen FRs; user stories and acceptance criteria	Use case diagram; Citizen screens and wireframes	Registration, login, profile; complaint form with map; My complaints; withdrawal; Updates inbox and feedback UI	Authentication and citizen browser tests; validation unit tests
TG/2023/1710 M.F. Rishma	Category, priority, evidence and service-target requirements (CF-05, 06, 09, 10, 16, 25)	Class / domain model; SD-1 submit sequence; evidence storage design	Category routing; complaint details; private evidence upload (reserve → upload → finalize); assignment and priority rules; SLA badges	Complaint and evidence database tests; SLA unit tests
TG/2023/1757 N.M. Razim Mohamed	Officer use cases UC-05–07 and stories US-08–12	Complaint state machine; activity diagram; SD-2 resolve sequence; Officer screens	Officer queues and filters; review, assign, priority, start, progress, transfer and resolve; Officer map; project repository set-up	Officer database tests; Officer browser journeys
TG/2023/1776 V. Kayaththiry	Non-functional and security requirements (NFR-01–11, SR-01–12)	Test plan; threat model and trust boundary; deployment architecture	Error handling and messages; security headers and environment validation; CI pipeline; Vercel deployment configuration	System and HTTP integration testing; defect log; responsive-layout audit; smoke test; CR impact analysis

14.3 Contribution log

Table 14.2: Contribution log (major tasks)

Sprint	Member	SDLC stage	Task completed	Evidence
Planning	All	Definition, Agile, requirements	Project proposal: problem, stakeholders, backlog, user stories, FR/NFR, use cases, RTM	grp15 proposal (Milestones 1–3)
1	1761, 1731	Design / implementation	Database foundation with RLS; registration, login, roles; admin user management	Migrations 001–003; features/auth, features/admin
1	1776	Implementation / testing	CI workflow, environment validation, test harness	.github/workflows/ci.yml; supabase/tests/harness.ts
2	1731, 1710	Implementation / testing	Complaint form with map, categories and routing, idempotent submission, private evidence, withdrawal	Migrations 004–008, 011; features/complaints, features/evidence
3	1757	Design / implementation / testing	Officer queues and the complete processing workflow with stale-write protection	Migrations 009–010; features/officer; officer.test.ts
3	1776	Testing / maintenance	Found the PostgREST retry-loop defect with HTTP integration tests; drove CR-002	scripts/test-postgres-http.mjs; DEF-01
4	1761, 1731	Implementation / testing	Feedback, Updates inbox, category management, monitoring	Migrations 012–014; sprint4.test.ts; sprint4.spec.ts
5	1761, 1710	Maintenance / implementation	CR-006 auto-assignment, dispatch centre, service targets, analytics	Migration 015; features/dispatch, features/analytics; dispatch.test.ts
5	1757, 1731	Design / implementation	Redesign with motion and dark mode; Officer map; landing page	src/shared/ui, features/landing, sprint5.spec.ts
5	1776, 1757	Testing / deployment / security	Responsive audit and fixes; security assessment; Vercel production deployment and smoke test	tests/e2e/capture.ts; vercel.json; DEF-16–19

14.4 Version-control evidence

All work is in the GitHub repository https://github.com/razim2003/civic-flow. The history shows the increments of every sprint, merged through feature branches (for example sprint-4/feedback-inbox-categories, sprint-5/dispatch-and-experience, demo/sri-lanka-data, ui/responsive-layout and release/vercel-production) with CI passing before each merge. A Git identity shows who pushed a commit, not necessarily who designed or wrote each part; where members worked together on one machine, each member's part is recorded in the contribution log above.

15 Conclusion

CivicFlow meets the goals we set in the proposal. A Citizen can report a public service problem once and follow it until it is fixed. Officers have a clear workflow with service targets, and the system stops anyone from skipping its steps. Administrators can manage access and services, make sure every complaint has an owner, and see how well each department performs.

The project also showed us how the output of one SDLC stage feeds the next. The stakeholder needs became objectives and prioritised requirements. The requirements became user stories with acceptance criteria and a traceability matrix. The design turned them into an architecture, UML models and a database whose security rules enforce the requirements, and the implementation was checked by 283 automated tests that trace back to them. Problems found during real use, such as complaints stuck in departments without staff and layout faults on tablets, were handled as maintenance through change requests and defect fixes, and every affected document was updated.

15.1 Lessons learned

- Keeping authorisation in the database and testing it as real users meant we could show that the security rules work, instead of assuming that they do.
- Short sprints that ended with a demonstration showed problems early: the empty category list in Sprint 2, the retry loop in Sprint 3 and the coverage gap in Sprint 5.
- Testing the hosted system with realistic data found a defect (fixed by CR-006) that unit tests could not find, because it depended on how the departments were staffed.
- Automated checks help with layout too: the layout audit found problems that we had missed when checking by hand.

15.2 Future work

- Validate the system with real citizens and Officers, and measure page-load performance on the deployed system (NFR-02).
- Add e-mail or SMS notifications through a custom SMTP or SMS provider, and application-level rate limiting.
- Malware scanning of uploaded evidence and persistent "remember me" sessions after a security review.
- A public, privacy-preserving map of resolved issues and multilingual support (Sinhala and Tamil).

16 References

1. Faculty of Technology, University of Ruhuna (2026) ICT2232 Software Engineering – Mini Project Guidelines.
2. Group 15 (2026) Public Service Complaint & Resolution System – Mini Project Proposal. University of Ruhuna.
3. Sommerville, I. (2016) Software Engineering. 10th edn. Harlow: Pearson.
4. Pressman, R.S. and Maxim, B.R. (2020) Software Engineering: A Practitioner's Approach. 9th edn. New York: McGraw-Hill.
5. Schwaber, K. and Sutherland, J. (2020) The Scrum Guide. Available at: https://scrumguides.org.
6. Object Management Group (2017) Unified Modeling Language (UML) Specification, version 2.5.1.
7. IEEE (1998) IEEE Std 830-1998: Recommended Practice for Software Requirements Specifications.
8. OWASP Foundation (2021) OWASP Top 10. Available at: https://owasp.org/Top10/.
9. Shostack, A. (2014) Threat Modeling: Designing for Security. Indianapolis: Wiley (STRIDE).
10. Supabase (2026) Supabase documentation: Auth, Database, Row Level Security and Storage. Available at: https://supabase.com/docs.
11. PostgreSQL Global Development Group (2026) PostgreSQL 17 Documentation. Available at: https://www.postgresql.org/docs/17/.
12. Meta Open Source (2026) React documentation. Available at: https://react.dev.
13. Vercel (2026) Vercel documentation. Available at: https://vercel.com/docs.
14. Microsoft (2026) Playwright documentation. Available at: https://playwright.dev.
15. OpenStreetMap Foundation (2026) Tile Usage Policy. Available at: https://operations.osmfoundation.org/policies/tiles/.

17 Appendices

17.1 Appendix A: User guide (summary)

Table 17.1: User guide by role

Role	How to use CivicFlow
Citizen	Create an account (name, e-mail, password of at least 12 characters) and confirm the e-mail. Choose Make a complaint, pick a category, describe the issue, mark the location on the map or type coordinates, and submit; attach photos or PDFs if needed. Follow progress in My complaints and the Updates inbox. Withdraw an unresolved complaint with a reason, or rate the service once it is resolved.
Officer	Accept the invitation e-mail and set a password. Use Department complaints and Assigned to me with filters. Open a complaint and use Process complaint: begin review, assign, set priority, start work, add progress and resolve. Due and overdue badges show the service target; the department map shows locations.
Administrator	Dispatch centre: see departments with no Officers and complaints that need attention, then dispatch with a reason. Analytics: service targets, coverage, workload, satisfaction and the six-month trend, with CSV export. User management: edit access or create Officer accounts. Service categories: add, edit, route and deactivate categories.
All users	On phones and tablets the menu is behind the ☰ button. You stay signed in only while the page is open. Use the moon/sun button to switch between light and dark themes.

17.2 Appendix B: Complete test cases

Legend: + positive test, − negative test (misuse or invalid input). The actual results come from the automated runs recorded in the repository test report (docs/sdlc/07-test-report.md), and each case can be re-run with the automated test named in that report.

Table 17.2: Authentication and access (CF-01…03, CF-15)

ID	Req	±	Preconditions / input	Expected result	Actual	Status
TC-AUTH-01	CF-03	−	No session; open /admin/users	Redirected to sign-in; no admin navigation	As expected	Pass
TC-AUTH-02	CF-01	+/−	Submit an empty form, then valid data	Field errors first; signup request carries only display_name (no role); "Check your email" shown	As expected	Pass
TC-AUTH-03	CF-02, 03	+	Sign in as Citizen, Officer, Administrator; sign out	Each lands in its own workspace; sign-out clears the workspace	As expected (3 cases)	Pass
TC-AUTH-04	CF-03	−	Citizen types /admin/users	"Administrator access required"; no admin data requested	As expected	Pass
TC-AUTH-05	CF-03	−	Signed-in account with no profile row	Fails closed with retry/sign-out; no workspace	As expected	Pass
TC-AUTH-06	CF-02	+	Sign in, reload page	Must sign in again (session in memory only)	As expected	Pass
TC-AUTH-07	CF-02	−	Password recovery; expired reset link	Generic response (no account enumeration); expired link can't set a password	As expected	Pass
TC-AUTH-08	CF-15	+	Change display name	Name updates; role unchanged	As expected	Pass
TC-U-AUTH-01	CF-01, 02	−	Short passwords, blank names, bad email, secret key as public key, insecure URL	Rejected by client rules	As expected	Pass
TC-U-AUTH-02	CF-02	−	A profile response arrives after sign-out or account switch	Late data discarded; fails closed on error	As expected	Pass
TC-DB-FND-01	CF-01, 03	−	Signup with metadata role=administrator	Profile created as active Citizen; metadata ignored	As expected	Pass
TC-DB-FND-02	CF-03, NFR-05	+	Inspect catalog	RLS enabled on every application table	As expected	Pass
TC-DB-FND-03	NFR-04	−	anon role reads tables / updates profiles	Permission denied	As expected	Pass
TC-DB-FND-04	CF-03	−	Citizen runs UPDATE profiles SET role='administrator', deletes, reads audit	Permission denied	As expected	Pass
TC-RBAC-01	CF-03	−	Citizen opens /admin/categories	"Administrator access required"; no nav link	As expected	Pass

Table 17.3: Complaint submission and tracking (CF-04…07, 16…18)

ID	Req	±	Preconditions / input	Expected result	Actual	Status
TC-SUB-01	CF-04, 06, 16, 18	+	Citizen submits a valid complaint; first upload fails, retry; download; open detail	Reference shown; request has no citizen_id/status; upload retry reuses the same key; private download; history shows "Supporting evidence added"	As expected	Pass
TC-SUB-02	CF-04	−	Blank coordinates; map tiles unavailable	Validation error; map click selects; numeric entry still works	As expected	Pass
TC-SUB-03	CF-06	−	First submit response lost (503)	Form kept; retry sends the same request ID; one complaint	As expected	Pass
TC-SUB-04	CF-05	+	Open category dropdown	Six categories, Other last; real category UUID submitted	As expected	Pass
TC-SUB-05	CF-05	−	No categories configured	Explained; submission blocked; no fabricated IDs	As expected	Pass
TC-LOC-01	CF-04	+	Open the form; click "Use my current location"	Map opens over Southern Province with no pin; opt-in fix sets an editable pin and zooms in	As expected	Pass
TC-LOC-02	CF-04	−	Geolocation denied / unavailable / timeout / unsupported / cancelled / late	Chosen point preserved; submission allowed; late results ignored	As expected (6 cases)	Pass
TC-TRK-01	CF-07, 17, NFR-03	+	11 complaints; paginate, search, filter; mobile width	Correct pages and filters; only own complaints; fits 375 px	As expected	Pass
TC-TRK-02	CF-07, 12	+	Open own resolved complaint	Progress steps, resolution text and date, history	As expected	Pass
TC-TRK-03	CF-07, NFR-11	−	Open another citizen's reference; description containing HTML	"Complaint unavailable"; HTML shown as text	As expected	Pass
TC-U-CMP-01	CF-04	−	Invalid field values; zero coordinates	Invalid values rejected; 0,0 accepted as a value	As expected	Pass
TC-U-EV-01	CF-16	−	Spoofed MIME, unsupported, empty, oversized files	Rejected client-side (usability, not malware safety)	As expected	Pass
TC-DB-SUB-01	CF-04–06, 18	+	Citizen calls submit_complaint	Trimmed values, routed department, status Submitted, version 1, event and notification created	As expected	Pass
TC-DB-SUB-02	CF-06	+/−	Same request ID twice; same ID with changed input	Same row returned; changed input → request_reused	As expected	Pass
TC-DB-SUB-03	CF-04, NFR-07	−	Description < 10 or > 5,000 characters; invalid/NaN/∞ coordinates	invalid_input	As expected	Pass
TC-DB-SUB-04	CF-03, 04	−	Officer, Administrator, inactive Citizen, anonymous	not_authorized / permission denied	As expected	Pass
TC-DB-SUB-05	NFR-06	−	Notification insert forced to fail	Complaint and event rolled back	As expected	Pass
TC-DB-SUB-06	CF-07, 08	−	Citizen B and a foreign-department Officer read Citizen A's complaint/history	0 rows	As expected	Pass

ID	Req	±	Preconditions / input	Expected result	Actual	Status
TC-DB-SUB-07	CF-05	–	Inactive category or department	category_unavailable; historic reference still readable	As expected	Pass
TC-DB-EV-01	CF-16, NFR-11	+	Inspect bucket; reserve	Private bucket, 5 MiB, JPEG/PNG/PDF; server path citizen/complaint/evidence.ext	As expected	Pass
TC-DB-EV-02	CF-16	–	Foreign Citizen, Officer, Administrator, inactive reserve	Denied	As expected	Pass
TC-DB-EV-03	CF-16	–	Declared size 0 / > 5 MiB; MIME text/html	Rejected	As expected	Pass
TC-DB-EV-04	CF-16	–	Storage writes to foreign, unreserved or spoofed-owner paths	Denied by Storage policy	As expected	Pass
TC-DB-EV-05	CF-16	–	Sixth file	evidence_limit; expired reservations free slots	As expected	Pass
TC-DB-EV-06	CF-16	–	Upload after review began	Reserve, upload and finalize all refused	As expected	Pass

Table 17.4: Officer processing and resolution (CF-08…12)

ID	Req	±	Preconditions / input	Expected result	Actual	Status
TC-OFF-01	CF-09–12	+	Officer reviews, assigns, prioritises, starts, adds progress, resolves with confirmation	Each step "Complaint updated"; expected versions 1→6 sent; complaint closed with resolution text	As expected	Pass
TC-OFF-02	CF-08, 17	+	"Assigned to me" with a priority filter	Scoped to the department and the Officer; filter narrows the list	As expected	Pass
TC-OFF-03	NFR-06	–	Another user changed the complaint (409)	Remark kept; "Refresh and review"; retry after refresh uses the new version	As expected	Pass
TC-OFF-04	CF-09	+	Transfer to Water supply	Officer returned to queue; old department loses access	As expected	Pass
TC-OFF-05	CF-11	–	Complaint assigned to another Officer	Start/progress/resolve not offered	As expected	Pass
TC-OFF-06	CF-22, NFR-03	–	Withdrawn complaint, mobile	Readable; all processing disabled	As expected	Pass
TC-OFF-07	CF-08	–	Workflow RPC missing vs. authorisation denied	Missing: read-only notice, activates after setup. Denied: fails closed.	As expected (2 cases)	Pass
TC-OFF-08	CF-10	+	Set an unchanged priority	"No change was needed"; no new history claimed	As expected	Pass
TC-OFF-09	CF-17	+	Search and filter, open a detail, go back	Filters preserved; Clear resets all	As expected	Pass
TC-U-OFF-01	CF-09–12	–	Missing reasons, short remarks, tampered URL filters	Rejected or bounded client-side	As expected	Pass
TC-DB-OFF-01	CF-11, 12, 18	+	Full lifecycle via RPCs	Versions 1→7; 7 events; Citizen notifications; last is resolution	As expected	Pass

ID	Req	±	Preconditions / input	Expected result	Actual	Status
TC-DB-OFF-02	CF-03, 08	–	Citizen, other Citizen, foreign Officer, inactive Officer, Administrator call 7 RPCs	not_authorized for all	As expected (5×7)	Pass
TC-DB-OFF-03	NFR-05	–	Anonymous RPC; direct UPDATE complaints; call private helper	Permission denied	As expected	Pass
TC-DB-OFF-04	CF-09	–	Assign to Citizen/foreign/inactive/Admin/null; roster contents	invalid_assignee; roster shows only active same-department IDs and names	As expected	Pass
TC-DB-OFF-05	CF-11, NFR-06	–	Skip/repeat transitions; stale version; no-ops	invalid_transition; PT409 stale_version; no-op adds no event	As expected	Pass
TC-DB-OFF-06	CF-11, 12	–	Empty/long progress; resolution < 10 or > 5,000	progress_required / resolution_required; version unchanged	As expected	Pass
TC-DB-OFF-07	CF-12, 22	–	Any workflow RPC on a Resolved or Withdrawn complaint	complaint_closed	As expected	Pass
TC-DB-OFF-08	NFR-06	–	Notification insert forced to fail during review	Status/version and history unchanged	As expected	Pass
TC-DB-OFF-09	CF-09	+/–	Transfer with mismatched category, short reason, valid; after start	category_unavailable / reason_required / success with access moved / invalid_transition	As expected	Pass

Table 17.5: Withdrawal (CF-22)

ID	Req	±	Preconditions / input	Expected result	Actual	Status
TC-WD-01	CF-22	+	Owner gives a reason and confirms	Status Withdrawn; reason in history; still searchable	As expected	Pass
TC-WD-02	CF-22	–	Stale page	Reason kept; refresh required	As expected	Pass
TC-WD-03	CF-22	–	Response lost	Same intent retried safely	As expected	Pass
TC-WD-04	CF-22	–	Resolved / withdrawn complaint	Withdraw not offered	As expected	Pass
TC-U-WD-01	CF-22	–	No reason / no confirmation / final states	Rejected	As expected	Pass
TC-DB-WD-01	CF-22	+	Withdraw from each open state	Reason, actor and time recorded; assignment released; event and notification atomic	As expected (3 states)	Pass
TC-DB-WD-02	CF-22	–	Foreign / non-Citizen / inactive / anonymous	Denied without revealing whether the complaint exists	As expected	Pass
TC-DB-WD-03	CF-22	–	Short reason; invalid/stale version; resolved	Rejected; records unchanged	As expected	Pass
TC-DB-WD-04	CF-22	+	Identical retry after a lost response	Returns the existing withdrawal; no duplicate history	As expected	Pass

ID	Req	±	Preconditions / input	Expected result	Actual	Status
TC-DB-WD-05	NFR-06	–	Notification failure	All withdrawal effects rolled back	As expected	Pass

Table 17.6: Sprint 4: feedback, inbox, categories, monitoring (CF-13, 19, 20, 21)

ID	Req	±	Preconditions / input	Expected result	Actual	Status
TC-FB-01	CF-19	+/–	Submit with no rating; then "Good" + padded comment	"Choose a rating" and no request; then one call {rating 4, comment trimmed}; panel becomes read-only	As expected	Pass
TC-FB-02	CF-19	–	Server answers feedback_exists	"Feedback was already submitted…"; form stays; no false success	As expected	Pass
TC-U-FB-01	CF-19	–	Ratings 0, 6, 2.5, missing; comment 2,001 characters	Rejected; 1–5 and 2,000 accepted	As expected	Pass
TC-DB-FB-01	CF-19	+/–	Owner rates a resolved complaint; identical retry; different second attempt	Stored once (trimmed); retry returns it; second → feedback_exists	As expected	Pass
TC-DB-FB-02	CF-19, NFR-07	–	Ratings 0, 6, null; comment 2,001 characters; whitespace comment	invalid_feedback; whitespace stored as null	As expected	Pass
TC-DB-FB-03	CF-19	–	Complaint not resolved; withdrawn	feedback_closed	As expected	Pass
TC-DB-FB-04	CF-19, CF-03	–	Other Citizen, Officer, Administrator, inactive owner	not_authorized	As expected (4)	Pass
TC-DB-FB-05	NFR-05	–	Anonymous RPC; direct UPDATE/DELETE/INSERT; reads by each role	Denied; readable by owner, department Officer and Admin only	As expected	Pass
TC-NTF-01	CF-13	+	Two unread, one read update; mark one; filter unread; mark all	Badge "2 unread" → "1 unread" → none; HTML remark shown as text; exact RPC payloads	As expected	Pass
TC-U-NTF-01	CF-13	+/–	Event label and remark; missing event; unknown kind	Label used; fallback title; invalid kind rejected	As expected	Pass
TC-DB-NTF-01	CF-13, NFR-11	+/–	Owner and others list notifications; mark one	Owner sees 5 kinds in order; others see 0; one marked; re-marking is a no-op	As expected	Pass
TC-DB-NTF-02	CF-13	–	Mark another user's notification (by ID and "all")	0 rows changed; target stays unread	As expected	Pass
TC-DB-NTF-03	CF-13	+	Mark all	All of the caller's unread rows set read	As expected	Pass
TC-DB-NTF-04	CF-13	–	Null/empty/101 IDs; null flag; inactive; anonymous; direct UPDATE/DELETE	invalid_input / not_authorized / permission denied	As expected	Pass
TC-DB-NTF-05	CF-13	–	Inactive recipient reads	0 rows	As expected	Pass

ID	Req	±	Preconditions / input	Expected result	Actual	Status
TC-CAT-01	CF-20	+/−	Admin opens categories; creates with no name, then valid; edits an inactive one	Validation error; created and listed; inactive departments offered only for the existing route; exact RPC payloads	As expected	Pass
TC-CAT-02	CF-20	−	Server answers category_in_use	Explanation shown; input kept	As expected	Pass
TC-U-CAT-01	CF-20	−	Empty/long name, long description, missing department	Rejected	As expected	Pass
TC-DB-CAT-01	CF-20	+	Create, no-op, update	Two audit events (created, updated) with before/after; no-op not audited	As expected	Pass
TC-DB-CAT-02	CF-20, NFR-10	−	Inactive category	Hidden from Citizens; submission → category_unavailable	As expected	Pass
TC-DB-CAT-03	CF-20	−	Duplicate name (different case); bounds; unknown/inactive department	category_name_taken, invalid_input, department_unavailable	As expected	Pass
TC-DB-CAT-04	CF-20	−	Change department of a referenced category	category_in_use; rename still allowed	As expected	Pass
TC-DB-CAT-05	CF-03	−	Citizen, Officer, inactive Admin, anonymous	not_authorized / permission denied	As expected (4)	Pass
TC-DB-CAT-06	NFR-05	−	Direct UPDATE categories; read audit table	Permission denied	As expected	Pass
TC-MON-01	CF-21	+	Admin overview with 1 resolved complaint, ratings 5 and 4	All 1, Open 0, Average 4.5; no processing buttons	As expected	Pass
TC-U-MON-01	CF-21	+	Counts and ratings	Open excludes final states; average to 1 decimal; none when no feedback	As expected	Pass
TC-UX-01	NFR-03	+	375×812 viewport; detail and inbox	No horizontal overflow	As expected	Pass

Table 17.7: Dispatch, coverage and service targets (CR-006: CF-23, 24, 25, 21)

ID	Req	±	Preconditions / input	Expected result	Actual	Status
TC-DSP-01	CF-24	+/−	Admin opens the Dispatch centre with a complaint in an unstaffed department; dispatches with no reason, then with a reason	Coverage card shows "No Officers"; row flagged unstaffed, unassigned and past target; reason error with no request; then exactly one RPC with version 1 and a success message	As expected	Pass
TC-DSP-02	CF-21, 25	+	Admin opens Analytics; exports CSV	Unstaffed warning; 67% on target; trend chart; CSV downloaded with dated name	As expected	Pass
TC-DSP-03	CF-13, 23, 25	+	Officer with an auto-assigned complaint	Updates badge "1 unread"; inbox link goes to the Officer detail path; map shows 1 marker; queue shows a "Due in" badge	As expected	Pass
TC-UX-02	NFR-03	+	Landing page; toggle theme	Hero and CTA shown; theme switches; no horizontal overflow	As expected	Pass

ID	Req	±	Preconditions / input	Expected result	Actual	Status
TC-UX-03	NFR-03, NFR-04	+	1280 px, then 768 px and 360 px. Open the menu, press Escape, then open each admin page from the menu	Desktop shows the sidebar with no menu button. Tablet/phone hide the sidebar links behind Open menu (aria-expanded). Focus moves to Close menu and returns to the button on Escape. Choosing a page closes the menu. No page is wider than the screen	As expected	Pass
TC-UX-04	NFR-03	+	Layout audit of 24 screens × 4 widths	No horizontal scroll and no overflowing content (see 7.2)	As expected	Pass
TC-U-SLA-01	CF-25	+/−	Targets and states at fixed times	24/72/168/336 h; on track / due soon / overdue; closed and invalid dates ignored	As expected	Pass
TC-U-DSP-01	CF-24	+/−	Attention flags; input bounds; messages	Unstaffed/unassigned/overdue flags; reason 10–2,000; refusals explained	As expected	Pass
TC-U-ANL-01	CF-21, SR-12	+/−	Summary maths; CSV with =HYPERLINK	Totals and rates correct; cells quoted; formula neutralised	As expected	Pass
TC-DB-DSP-01	CF-23	+	Two Officers; two submissions	One complaint each (balanced); assignee notified; event auto_assigned	As expected	Pass
TC-DB-DSP-02	CF-23	−	Only Officer inactive	Unassigned; citizen notified only	As expected	Pass
TC-DB-DSP-03	CF-23	+	Identical retry	Same row, same assignee, version 1	As expected	Pass
TC-DB-DSP-04	CF-13	+	Officer assigns a teammate, then themselves	Teammate notified; self-assignment not notified	As expected	Pass
TC-DB-DSP-05	CF-24	+	Dispatch stuck complaint to Water supply with an Officer	Route, assignee and version 2; transferred event by admin with reason; citizen and Officer notified; Officer can read it	As expected	Pass
TC-DB-DSP-06	CF-24	+	Same-department reassignment; repeat	assigned event; repeat is a no-op	As expected	Pass
TC-DB-DSP-07	CF-24	−	Short reason, mismatched route, foreign/citizen assignee, stale/null version, withdrawn	reason_required, category_unavailable, invalid_assignee, PT409, invalid_input, complaint_closed	As expected	Pass
TC-DB-DSP-08	CF-24	−	In-progress complaint moved or unassigned	invalid_transition; Officer swap within the department allowed	As expected	Pass
TC-DB-DSP-09	CF-24, CF-03	−	Citizen, Officer, inactive Admin, anonymous	not_authorized / permission denied	As expected (4)	Pass
TC-DB-DSP-10	NFR-06	−	Notification failure during dispatch	Route and version unchanged	As expected	Pass
TC-DB-DSP-11	CF-21, 25	+/−	Analytics as admin; as citizen, Officer, inactive admin	Coverage, overdue, workload, 6-month trend; others refused	As expected	Pass
TC-DB-DSP-12	CF-21, NFR-11	+/−	Community impact as citizen, inactive, anonymous; private helper	Only 6 aggregate keys; inactive/anonymous refused; helper not executable	As expected	Pass

Table 17.8: Sri Lankan demonstration data (supabase/demo)

ID	Req	±	Preconditions / input	Expected result	Actual	Status
TC-DB-DEMO-01	NFR-11	+	Empty database; run the seed	10 Citizens and 6 Officers who cannot sign in (.test e-mail, no password); Other public services left unstaffed	As expected	Pass
TC-DB-DEMO-02	CF-04…12, 19, 22, 23	+	Seed through the workflow RPCs	36 complaints: 21 resolved, 4 in progress, 3 under review, 5 submitted, 3 withdrawn; 1 transfer; 33 auto-assigned; 19 ratings; 3 unassigned in the unstaffed department	As expected	Pass
TC-DB-DEMO-03	NFR-06	+	Backdated history	Events 1..version with no gaps, in time order; first and last events match created/resolved/withdrawn times; every event notifies the citizen; times within 180 days; coordinates inside Sri Lanka; feedback after resolution	As expected	Pass
TC-DB-DEMO-04	CF-21, 25	+	Analytics after seeding; Officer reviews a seeded complaint	6 months of submissions, overdue > 0, some late resolutions; seeded complaint still works with the normal RPC	As expected	Pass
TC-DB-DEMO-05	NFR-06	+/−	Existing Road Officer; re-run; remove while a real complaint is assigned to a demo Officer; remove again	Existing Officer used (no demo Road Officer); re-run changes nothing; removal refused with demo_data_in_use; then removes only demo rows and keeps real ones	As expected	Pass

Table 17.9: Administration (CF-14)

ID	Req	±	Preconditions / input	Expected result	Actual	Status
TC-ADM-01	CF-14	−	Citizen opens Officer / Officer-provisioning routes	Refused	As expected	Pass
TC-ADM-02	CF-14	−	Demote the last admin	Backend refusal shown	As expected	Pass
TC-ADM-03	CF-14	+/−	Invite an Officer; email delivery refused	Department required; no password/role fields; refusal shown without false success or retry	As expected (2)	Pass
TC-U-INV-01	CF-14	−	Edge Function: no session, non-admin, foreign origin, bad input, inactive department, provider failures	Refused before any invitation; no promotion on partial failure	As expected (9)	Pass
TC-DB-ADM-01	CF-14	−	Non-admin calls admin_update_user	not_authorized	As expected	Pass
TC-DB-ADM-02	CF-14	+	Promote to Officer without and with a department	Requires an active department; audit row written	As expected	Pass
TC-DB-ADM-03	CF-14	−	Demote the last admin	last_administrator; change rolled back	As expected	Pass
TC-DB-ADM-04	NFR-04	−	Deactivate a user who has a valid claim	Reads and writes refused immediately	As expected	Pass

Table 17.10: HTTP integration with real JWTs (last executed 2026-10-21)

ID	Scenario (from the Officer verification record)	Result 2026-10-21
TC-INT-01	Owner / department / inactive / role denial through PostgREST with real user JWTs	Pass
TC-INT-02	Signup metadata privilege escalation through Auth	Pass
TC-INT-03	Stale withdrawal returns HTTP 409 immediately (no retry loop)	Pass
TC-INT-04	Concurrent review by two Officers: one succeeds, one gets 409	Pass
TC-INT-05	Resolution racing withdrawal: exactly one final state	Pass
TC-INT-06	Transaction rollback through HTTP	Pass
TC-INT-07	Transfer removes old-department access	Pass
TC-INT-08	Two Administrators demote each other concurrently: at least one remains	Pass
TC-INT-09	Only the assignee can start (403 for teammate); short resolution rejected (400); submission and owner reads work over HTTP	Pass

17.3 Appendix C: Repository, deployment and demonstration

Table 17.11: Key links and commands

Item	Value
Source code	https://github.com/razim2003/civic-flow
Deployed application	https://civic-flow-rho.vercel.app
SDLC evidence pack	docs/sdlc/ (README, 01–11, user guide, screenshots)
Run locally	npm ci, then npm run dev (Node.js ≥ 22.18)
Run all checks	npm run lint, npm run typecheck, npm test, npm run test:db, npm run build, npx playwright test
Layout audit	LAYOUT_AUDIT=<folder> npx playwright test
Demo data	supabase/demo/sri-lanka-demo.sql (remove with remove-sri-lanka-demo.sql)

Table 17.12: Demonstration script for the final evaluation

Step	What to show
1	Problem and stakeholders (Chapters 2–3); backlog and one user story with acceptance criteria (Chapter 6)
2	Scrum process and sprint history; architecture and sequence diagram SD-1 (Chapter 8)
3	Live demo on the deployed URL: a Citizen submits a complaint, the automatically assigned Officer processes and resolves it, the Citizen sees the update and rates it, and the Administrator views the analytics and dispatches a stuck complaint
4	Run npm run test:db and show a database test refusing a direct UPDATE (Chapter 10)
5	Change request CR-006 and its impact analysis (Chapter 12)
6	Security: open another citizen's complaint URL and show the refusal; show a <script> description rendered as text (Chapter 13)